

Computational Analysis of Educational Data: A Field Guide Using R

Wei Wang, Mete Akcaoglu, Joshua Rosenberg, Shaun Kellogg

July 16, 2026

Table of contents

Preface	vii
Getting Started	1
How the Full Book Connects	3
Additional Resources:	4
Positron and RStudio	5
1.1 Overview	5
1.2 Why Positron?	5
1.3 The Four Essential Panes	5
1.4 R Projects	7
1.5 Essential Keyboard Shortcuts	8
1.7 Extensions and Customization	9
1.8 What Makes Positron Stand Out	11
1.9 Version Control with Git and GitHub	12
1.10 Summary	16
R	17
2.1 Overview	17
2.2 Three Ways to Write R Code	17
2.3 Console vs. Source: Exploration vs. Preservation	22
2.4 When You Get Stuck	23
2.5 Summary	24
Tidyverse	27
3.1 Overview	27
3.2 What is in the Tidyverse?	27
3.3 Installing and Loading the Tidyverse	28
3.4 Working with Real Data	28
3.5 Essential dplyr Functions	29
3.6 The Pipe Operator: <code> ></code> and <code>%>%</code>	33
3.7 Data Visualization with ggplot2	34
3.8 Summary	38

Data Science Methods	41
Overview	43
The Analytical Scope of This Section	43
Setting Up the Computational Environment	44
A General Computational Workflow	45
Ethics, Transparency, and Reproducibility	45
Transition to Analytical Chapters	46
Text Data	47
4.1 Overview	47
4.2 Accessing Text Data	47
4.3 Methods for Analyzing Text Data	59
4.3.2 Dictionary-based Analysis	65
4.3.3 Clustering-Based Analysis	69
4.4 Summary	76
Network Data	79
5.1 Overview	79
5.2 Accessing SNA Data	79
5.3 Network Management & Measurement in Social Network Analysis	83
5.4 Case Study: Hashtag Common Core	95
5.5 Summary	104
Numeric Data	107
6.1 Overview	107
6.2 Accessing Big Data (Broadening the Horizon)	107
6.3 Logistic Regression & Machine Learning	114
6.4 Random Forests & Learning Analytics	121
6.5 Summary	132
LLM Methods	135
Overview	137
Cloud vs Local LLMs	141
7.1 Overview	141
7.2 Cloud-Based LLMs: Capabilities and Setup	141
7.3 Local LLMs: Privacy-Preserving and Offline Analysis	145
7.4 Cloud vs. Local LLMs: Choosing the Right Tool	148
7.5 Practical Setup Checklist	148
7.6 Summary	149

Positron Assistant and Databot	151
8.1 Overview	151
8.2 Positron Assistant in the Research Workflow	151
8.3 Getting Started	153
8.4 Using Positron Assistant	161
8.5 Meet Databot (Experimental): Your EDA Accelerator	164
8.6 Using These Tools in Research Practice	167
8.7 Working Responsibly with AI Helpers	168
8.8 Summary	170
Local LLMs for Text	171
9.1 Overview	171
9.2 What is a Local LLM?	171
9.3 What Can Local LLMs Do in Educational Research?	172
9.4 LM Studio as a Local Model Environment	172
9.5 Putting It All Together: A Real Example	174
9.6 Summary	208
Local LLMs for Images	209
10.1 Overview	209
10.2 Images	209
10.3 Analyzing Images	210
10.3.2 Image Classification	211
10.4 Summary	220
Communication and Conclusion	221
Communication Matters	225
11.1 Overview	225
11.2 Common Ways to Communicate Computational Educational Research	225
11.3 Choosing an Appropriate Communication Strategy	231
11.4 Case Study: Communicating a Text Analysis with Quarto .	232
11.5 Benefits of Web-Based Communication	236
11.6 Ethical and Responsible Communication	236
11.7 Data Availability and Reproducibility Statement	237
11.8 Summary	237
Conclusion	239
12.1 What this book contributes	239
12.2 Methods are complementary, not competitive	240

Table of contents

12.3 Responsible use in educational research 240

12.4 A practical path forward 241

12.5 Final reflection 241

Colophon **243**

 About the Authors 243

References **245**

Preface



Welcome to *Computational Analysis of Educational Data: A Field Guide Using R!*

Why this book?

Conventionally, as educational researchers, we start the research process by generating research questions based on our previous knowledge and theories in the field. This is the normative view of how the research process should work. But we are limited in how we see the world: we do not know what we do not know. For example, someone who does not know that social media posts can be collected and analyzed as data to capture the state of the world will most likely not develop research questions about such data. This reveals that it is also epistemologically possible for our observations of the world to guide our research questions — that discovering what can serve as data shapes what we can ask as research questions (Salganik, 2019).

In practice, the research process does not always proceed in the orderly, linear fashion suggested by the conventional model. Once you start seeing what can be data in the world, it shapes your ideas of what is “researchable.” Here is a simple model that we propose showing the reciprocal nature of the research process, which is central to our aims in this book:

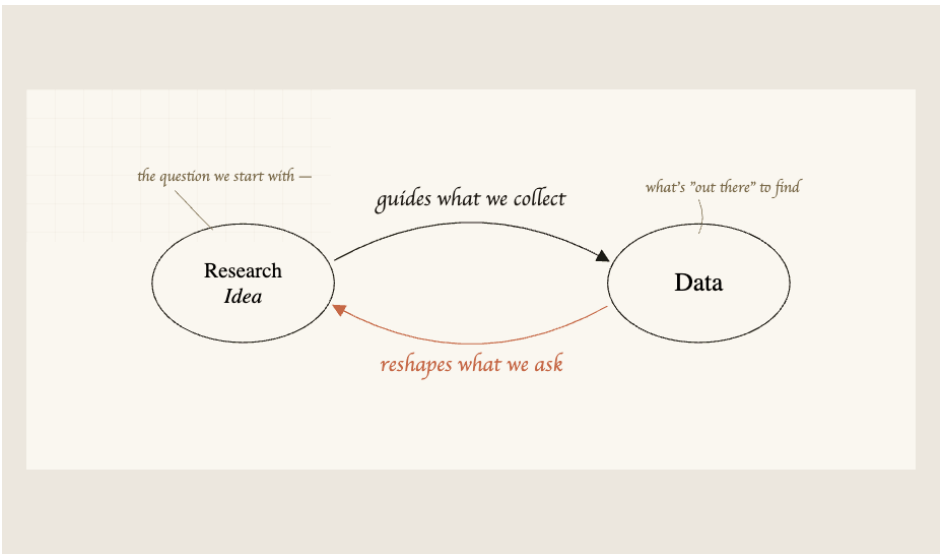


Figure P.1. Research-data relationship (Generated using Gemini Pro based on an earlier drawing by the authors)

We are writing this book because in the past few years what we described above has started happening for us. We have published work on social media data, social network analysis, natural language processing, and machine learning that was only possible after we learned what kind of data already existed around us (Estrellado et al., 2020). We thought other educational researchers may benefit from a resource that not only tells them what is available as data but also guides them through concrete examples of going through this research journey along with us. We hope that along this journey you will develop your own research questions, and maybe even replicate some of the studies we imagined in this book.

A second distinctive feature of this book is the “field guide” process we use to work through research design and reporting. Although there are many books on R (or Positron), they usually cover the data science aspects of the work, leaving the academic writing and reporting to individuals. Although researchers are expert in writing up conventional methods or results, these emerging data analysis methods (thanks to new ways data has become available) require a new guidebook. For each computational research method, we follow this process:

1. Start with what makes good data for that analysis (and how to capture it)
2. See what the data looks like (what it *has to have*, and what it can have)
3. Formulate sample research questions based on the resources provided by the data and the type of analysis.
4. Go through the analyses in R
5. Provide a sample write up for Methods
6. Provide a sample write up for Results

What we mean by computational methods

By *computational methods*, we mean research approaches that leverage the power of computing to collect, process, and analyze data at a scale or speed not possible through traditional means. This includes methods that fall under the umbrella of data science — such as text analysis, social network analysis, and machine learning — as well as the emerging use of generative AI and large language models as research tools (Estrellado et al.,

2020; Romero & Ventura, 2010, 2020; Siemens, 2013). These approaches are sometimes grouped under educational data science, learning analytics, or educational data mining, depending on the community and tradition (Krumm et al., 2018; Lang et al., 2022).

Computational social science is an ally field that has advanced many of the methods we draw on (Lazer et al., 2009). But we focus specifically on educational research, where the data, questions, and stakes have distinctive features — among them, the prevalence of sensitive student data, the importance of equity and context in interpretation, and the goal of understanding, describing, predicting, and impacting learning and teaching.

LLMs in computational analysis

As this book developed, another major shift emerged in our own work: large language models (LLMs) became part of everyday computational analysis. We do not see LLMs as a replacement for the methods in this book (e.g., text mining, network analysis, or machine learning). Instead, we treat them as an additional methodological layer that can support coding, interpretation, synthesis, and communication.

In practice, this means we use LLMs in at least three ways: as assistants for writing and debugging code, as tools for structured analysis workflows under researcher supervision, and as analytic partners for qualitative and multimodal data. Across all three uses, our approach remains the same as the rest of this field guide: start with a clear research question, make data and analytic decisions explicit, and validate outputs before drawing conclusions.

Because LLM-assisted analysis introduces new opportunities and risks, we also emphasize responsible use throughout the book. We encourage readers to evaluate LLM-supported work through three core criteria: correctness, transparency, and reproducibility. In other words, LLMs can help us see more in the data, but they should never reduce the rigor of how we do research.

This Book Vs. The AI Timeline

We did not write this book in a quiet methodological era. We started with a strong computational methods plan, thought we were almost done,

and then watched the AI landscape change faster than our chapter drafts. More than once.

The comic below captures that very real author experience: every time we felt “finally, this version is stable,” a new model release arrived and politely informed us that stability is a temporary concept.

Instead of treating this as a disaster, we used it as a design principle. Keep the foundations (text, network, numeric methods), integrate new AI workflows where they are genuinely useful, and make responsible use explicit at every step.

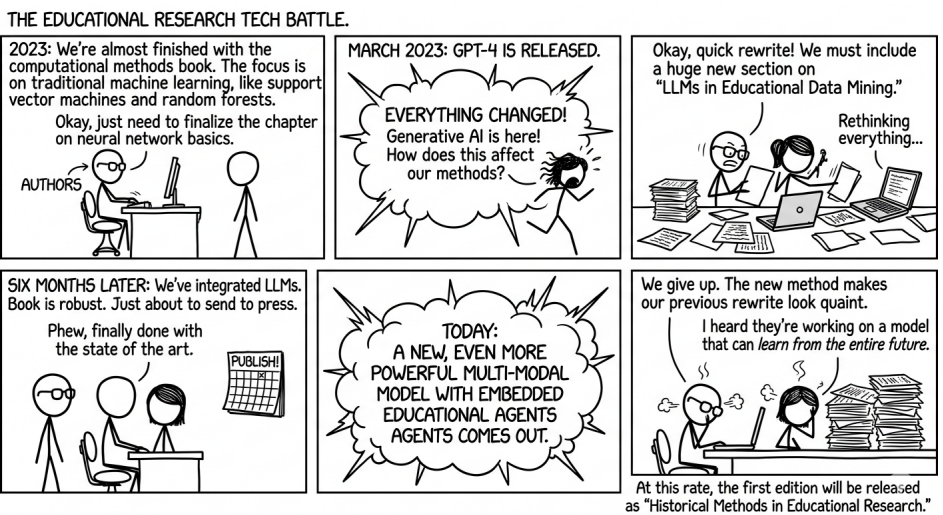


Figure. A comic-style reflection of how this book evolved alongside rapid AI changes (generated with Gemini, based on the authors’ storyline).

Data and reproducibility note

This book is designed to be reproducible with openly available project files and datasets. Most executable chapter workflows use data stored in the repository under `data/`, so readers can run analyses locally without relying on hidden inputs.

Some snippets are intentionally included as illustrative templates (for example, showing a generic data-loading or network-construction pattern). These are pedagogical examples and are not part of the core executed analysis chain.

For a formal data-availability and reproducibility statement, see Chapter 11.

Funding and source acknowledgment

Selected chapters in this book include materials adapted from resources developed as part of the Learning Analytics in STEM Education Research (LASER) Institute. The LASER Institute was supported by the National Science Foundation and contributed to the development of instructional materials, examples, and research-based approaches that informed portions of this work.

We gratefully acknowledge the National Science Foundation's support for the LASER Institute and the contributions of the LASER Institute leadership team, instructors, scholars, and broader community. Any opinions, findings, conclusions, or recommendations expressed in this book are those of the authors and do not necessarily reflect the views of the National Science Foundation.

To learn more about the LASER Institute, visit <https://fi.ncsu.edu/projects/laser-institute>.

Who is this book for?

This book will be beginner-friendly but not for the absolute beginner. We will dedicate the initial chapters to take you to resources that will help you get started. But, to make sense of this book, you should have basic research design knowledge, basic statistical knowledge, and a basic understanding of R and RStudio. At the same time, this book will not be for experts or expertise. There are already many great resources that delve into the topics that we cover (e.g., Silge's book on using text data for machine learning).

We imagine that this book can become a part of doctoral coursework for future researchers, opening the doors for new ways to look at the world for research and data. Likewise, senior and junior academics/researchers would benefit greatly from this book to help them expand their research agendas.

For researchers like ourselves, we think this book can serve as a fun summer reading to rejuvenate and get excited about new research. At the same

time, we also envision this book as a guidebook to keep on the side and frequently refer to, as researchers write up their work using these new methods.

This book will provide new ways to look at the world and formulate RQs. It will guide through the research process for each new method (including, friendly data organization tips, template for writing up and sharing this. We hope that you join us in this journey and this work will help open up new doors/embark on a journey of using computational research methods.

Organization of the Book

The book is organized around four sections. Within these sections, there are specific chapters, which serve as field guide “entries” or “cases”. While the section overviews (the first chapter within each section) introduce the techniques or methodologies used in the rest of the section’s chapters, the chapters are intended to address a specific, narrow case, where we provide a sample write-up for researchers in writing their research questions, methods, results (and discussions) sections based on the analyses.



Getting Started

If you are coming to computational educational research from a traditional research background, the technical setup can feel like the hardest part. This section is designed to lower that barrier and help you build a practical foundation before moving into method-heavy chapters (Estrellado et al., 2020).

Think of this section as the minimum viable toolkit for the rest of the book. The goal is not to teach everything about R or Positron, but to give you enough structure to reproduce examples, adapt workflows, and work confidently with your own data (Healy, 2018; Wickham & Golemund, 2017).

The three chapters in this section establish that foundation. Specifically, they cover:

- **Positron and RStudio** introduces your working environment, project organization, and workflow habits that support reproducibility.
- **R** covers core file types, execution patterns, and practical help-seeking strategies for common coding roadblocks.
- **Tidyverse** builds the day-to-day data workflow vocabulary for reading, cleaning, transforming, and visualizing data in R.

By the end of this section, you should have a functioning analysis environment and a clear workflow for moving from data to interpretable output.

How the Full Book Connects

This book follows a four-part progression:

- **Getting Started** builds the technical and workflow baseline.
- **Data Science Methods** applies text, network, and numeric approaches to core research tasks.
- **LLM Methods** introduces cloud/local LLM workflows and AI-assisted analysis with explicit attention to responsible use.
- **Communication** focuses on sharing computational research transparently and reproducibly.

Across all sections, we prioritize the same principles: methodological clarity, reproducibility, and responsible interpretation of results.

Additional Resources:

- R for Data Science: <https://r4ds.had.co.nz/> and R for Data Science (2e): <https://r4ds.hadley.nz/>

Positron and RStudio

1.1 Overview

You need an environment for writing and running R code. We are going to focus on **Positron**, Posit’s newer IDE built on the foundation of VS Code. If you are already comfortable with RStudio, everything in this chapter applies there too. The fundamental concepts are identical, and the R code in this book works the same way in both.

1.2 Why Positron?

Positron represents where computational educational research workflows are heading: multi-language support (R and Python in the same environment), modern editor features inherited from VS Code, and a more extensible architecture (i.e., plugins and extension). If you are coming from VS Code, Positron will feel immediately familiar. If you are new to both, you are learning a tool that is designed for the future of data science.

That said, **RStudio remains great**. It is mature, widely documented, and has a loyal community. Some researchers prefer its more opinionated interface. Everything you need is visible and organized for R-first workflows. If you are already using RStudio and prefer it, nothing in this book requires switching.

The instructions below work in both environments unless otherwise noted. When we say “Positron,” assume the same applies to RStudio.

1.3 The Four Essential Panes

Both Positron and RStudio organize your workspace into four main areas (though the layout is customizable):

- **Source/Editor** (top-left): Where you write and edit your code files (.R scripts, .qmd documents)
- **Console** (bottom-left): Where R actually executes code and shows results in real-time
- **Environment/Variables** (top-right): Shows the objects (data, functions, etc.) currently loaded in memory
- **Files/Plots/Help** (bottom-right): Navigate your project files, view visualizations, and access documentation

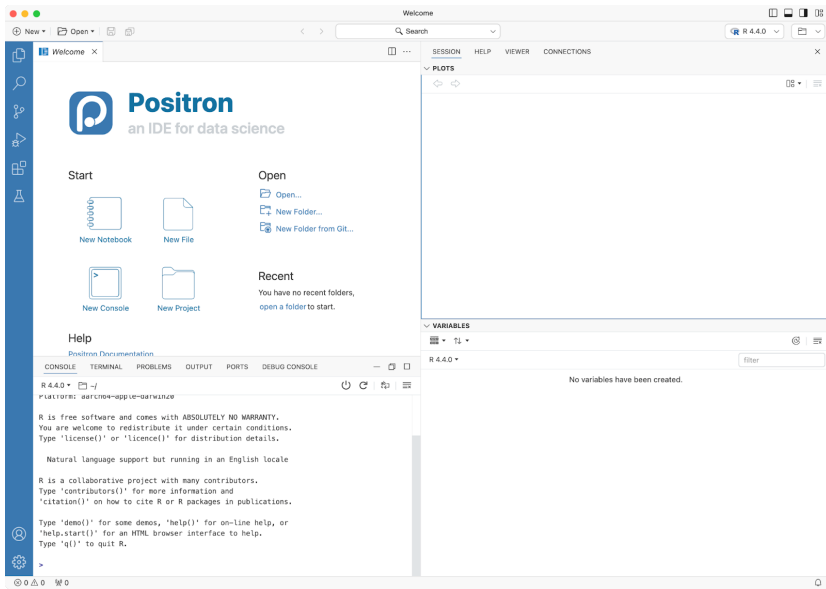


Figure 1.1. Positron interface and four-pane layout.

1.3.1 Understanding Each Pane

Source/Editor Pane: This is where you spend most of your time writing code. The editor provides syntax highlighting (different colors for functions, strings, and comments), autocomplete suggestions, and the ability to work with multiple files in tabs. You can split the pane to view two files side-by-side, and for Quarto documents, you can preview the rendered output alongside your code.

Console Pane: When you run code, it executes here and displays results immediately. Press the up arrow to cycle through previously run commands—useful when you want to re-run or modify something. The

prompt `>` means R is ready for input; a `+` means R is waiting for you to complete a command (often because a closing parenthesis is missing). You can clear the console output with `Cmd/Ctrl + L`, though this does not delete your objects from memory. Note that in the console pane, you have also **Terminal**. Terminal is different than the console in that the **Console runs R code and displays R output**, serving as your direct interface with the R language interpreter, while the **Terminal provides access to your computer's system shell** (bash on Mac/Linux, Command Prompt/PowerShell on Windows) where you can run system commands like file navigation, git version control, and package installation at the operating system level. In practical terms, you would use the Console for running R scripts, testing R functions, and viewing R output, whereas you would use the Terminal for navigating directories with `cd`, managing git repositories, installing system dependencies, and running command-line tools. Both are useful, but they serve fundamentally different purposes: the Console speaks R, while the Terminal speaks your operating system's language.

Environment/Variables Pane: This shows every object currently loaded in R—datasets, variables, functions you've created. Click on a dataset name to open it in a spreadsheet-style viewer. The display also shows object types (data frame, list, numeric vector) and basic information like the number of rows and columns. You can remove objects using the broom icon, which clears your workspace—useful when starting fresh.

Files/Plots/Help Pane: The Files tab lets you navigate your project directory, create new folders, and open files. The Plots tab shows your visualizations with forward/back arrows to review previous plots and an Export button to save images. The Help tab displays R documentation when you run commands like `?mean` or search for functions. In RStudio, there is also a Packages tab for installing and loading packages; Positron handles packages differently but with similar functionality.

The defaults work fine for now. As you get comfortable, you might adjust panel sizes or move things around, but the essential logic stays the same: code on the left, context on the right.

1.4 R Projects

Always work in an R Project. This is the single most important RStudio habit for reproducibility.

An R Project is a folder that contains all files for a particular analysis—data, scripts, outputs—and sets your working directory automatically. This means your code will work on any computer without changing file paths.

To create a new project:

- 1. File → New Project
- 2. Choose “New Directory” or “Existing Directory”
- 3. Name your project (e.g., “my-first-css-analysis”)
- 4. Click “Create Project”

Notice the `.Rproj` file in your folder? That is your project file. Double-click it to open Positron or RStudio with the correct working directory already set.

1.5 Essential Keyboard Shortcuts

RStudio Keybindings in Positron. If you are coming from RStudio and want to keep using familiar shortcuts, install the **RStudio Keymap** extension in Positron. Open the Extensions panel (View → Extensions), search for “RStudio Keymap,” and click Install. This maps common RStudio shortcuts to their Positron equivalents so you do not have to retrain your muscle memory.

Keyboard shortcuts can speed up your workflow considerably. You do not need to memorize them all at once—start with a few that feel natural, and others will follow as you get more comfortable.

Table 1.1. Essential keyboard shortcuts in Positron and RStudio.

Action	Win/Linux	macOS
Run line or selection	Ctrl+Enter	Cmd+Enter
New document	Ctrl+Shift+N	Cmd+Shift+N
Save file	Ctrl+S	Cmd+S
Find in file	Ctrl+F	Cmd+F
Comment/uncomment code	Ctrl+Shift+C	Cmd+Shift+C
Insert native pipe (<code> ></code>)	Ctrl+Shift+M	Cmd+Shift+M
Insert assignment (<code><-</code>)	Alt+-	Option+-
Restart R session	Ctrl+Shift+F10	Cmd+Shift+O

The run command (Ctrl/Cmd + Enter) is the one you will use most—it executes the line where your cursor is, or if you have selected multiple lines, it runs all of them. The comment shortcut is helpful when testing code: you can quickly comment out a section to see if removing it fixes an error.

Positron and RStudio have slightly different shortcuts for a few operations, particularly around navigating panes and restarting R. To see the full list specific to your environment, go to Help → Keyboard Shortcuts or press Alt/Option + Shift + K.

1.7 Extensions and Customization

Positron is built on VS Code, which means most VS Code extensions work in Positron. This gives you access to a large ecosystem of tools beyond what is built into the base IDE.

To access extensions, open the Extensions panel—usually visible in the left sidebar, or accessible via View → Extensions. From there, you can search for extensions by name or browse categories.

1.7.1 Useful Extensions for R Work

While Positron comes with solid R support out of the box, these extensions can enhance your workflow:

- **Rainbow CSV:** Colors columns in CSV files to make them easier to read
- **GitLens:** Shows detailed Git information inline (who changed what, when, and why)
- **Markdown Preview Enhanced:** Better previews for Markdown documents with extended syntax support

To install an extension, search for it by name in the Extensions panel, click on it, and press Install. Most extensions work immediately without restarting Positron.

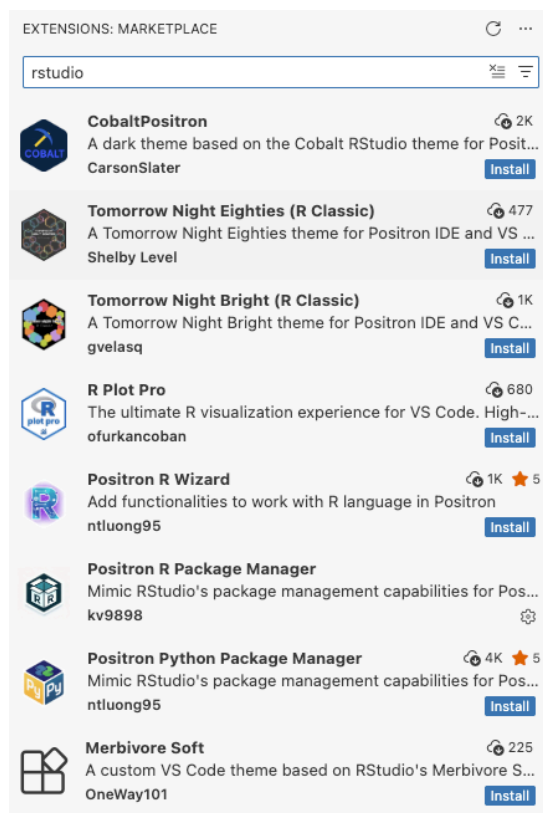


Figure 1.2. Positron extensions marketplace for R workflows.

1.7.2 RStudio's Approach

RStudio uses “addins” instead of extensions. These are R packages that add functionality to the IDE interface. The ecosystem is smaller than VS Code’s, but focused specifically on R workflows. If you are using RStudio, you can find addins through the Addins menu once you have installed packages that provide them.

1.7.3 Customization

Both Positron and RStudio let you customize themes, font size, and panel arrangements. The defaults work well to start, but if you find yourself squinting at text or want a different color scheme, explore Preferences/Settings → Appearance. Changes are a matter of personal pref-

erence rather than workflow improvement, so do not feel pressure to customize immediately.

1.8 What Makes Positron Stand Out

Beyond the basics, Positron offers several features that become valuable as your projects grow in complexity:

Multi-language support: You can work with R and Python in the same project without switching between different environments. If you later need to use Python libraries or call Python code from R (common in machine learning workflows), Positron handles both languages natively.

Terminal integration: The built-in terminal in Positron has better shell support than RStudio's terminal. This matters when you need to run command-line tools, manage Docker containers, or work with system utilities alongside your R code.

Remote development: Through SSH extensions, you can connect to remote servers and work on them as if they were local. Your code runs on the server, but Positron's interface stays responsive on your machine. This is useful when analyzing large datasets that do not fit on your laptop or when you need more computing power.

Performance: The editor itself is noticeably faster with large files—syntax highlighting, scrolling, and searching all feel snappier. This does not matter much for typical scripts, but when you are working with thousand-line codebases or large Quarto documents, the difference shows.

Modern editor features: IntelliSense provides context-aware code completion (it knows what functions are available on specific objects), bracket matching highlights matching parentheses and braces automatically, and a minimap gives you a bird's-eye view for navigating long files.

These features are there when you need them. For now, just know they exist—they are part of why Positron represents where these tools are heading.

1.9 Version Control with Git and GitHub

Git is how computational educational researchers version, share, and collaborate on their work. GitHub is the most popular platform for hosting Git repositories. If you’ve never used version control before, this section will get you set up with a workflow that integrates directly into your IDE.

1.9.1 Why Git Matters

Without version control, you end up with files like `analysis_final.R`, `analysis_final2.R`, `analysis_ACTUALLY_final.R`. Git tracks every change you make, so you can:

- Go back to any previous version without keeping multiple copies
- Experiment freely—if something breaks, you can always revert
- Collaborate with others without emailing files back and forth
- Share your work publicly for transparency and reproducibility

Most computational educational research projects live in Git repositories, and employers expect researchers to be comfortable with basic version control.

1.9.2 Git Integration

Once Git is configured, you will see version control features built into your environment:

- **In RStudio:** A “Git” tab appears in the Environment pane (top-right)
- **In Positron:** Git integration works through the Source Control panel (often on the left sidebar)

Either way, you can stage changes, write commit messages, and push to GitHub without leaving your editor. This is far friendlier than memorizing terminal commands.

1.9.3 Setting Up Git and GitHub: The Path of Least Resistance

Git authentication is notoriously confusing. We are going to use an R-native approach that minimizes pain: **Personal Access Tokens (PAT) via the `usethis` package**. This keeps you in familiar territory and works reliably for the kind of work you will do in this book.

1.9.3.1 Step 1: Install Required Packages

```
install.packages("usethis")
install.packages("gitcreds")
```

1.9.3.2 Step 2: Configure Your Git Identity

Tell Git who you are (one-time setup):

```
# Use your GitHub name and email
usethis::use_git_config(
  user.name = "Jane Doe",
  user.email = "jane@example.com"
)
```

Important: Use the same email address you will use (or already use) for your GitHub account.

1.9.3.3 Step 3: Create a GitHub Personal Access Token

```
usethis::create_github_token()
```

This opens GitHub in your browser. You will need to:

1. Log in to GitHub if you have not already
2. Leave the default scopes checked (at minimum: “repo” and “workflow”)
3. Set expiration to 90 days (good security practice, though you will regenerate when it expires)
4. Click “Generate token” at the bottom
5. **Copy the token immediately**—GitHub only shows it once

The token begins with a prefix such as `ghp_` followed by a long string of characters.

1.9.3.4 Step 4: Store Your Token in R

```
gitcreds::gitcreds_set()
```

When prompted, paste your token and press Enter. R stores it securely for future use.

1.9.3.5 Step 5: Verify Everything Worked

```
usethis::git_sitrep()
```

This diagnostic report should show:

- Your name and email
- Personal access token: <discovered>

If so, you are all set. If not, see troubleshooting below.

1.9.4 Alternative: SSH Keys (For the Long Haul)

SSH keys can be convenient—set them up once and never think about authentication again. But they are more complex to configure initially.

Interested? See *Happy Git with R*, Chapters 10-12. For now, the PAT method will serve you well.

1.9.5 Using Git in Practice

We will introduce Git commands as you need them throughout the book. For now, know that your IDE gives you a graphical interface for the most common operations:

- **Pull:** Download changes from GitHub (always do this before starting work)

- **Commit:** Save a snapshot of your changes with a descriptive message
- **Push:** Upload your commits to GitHub

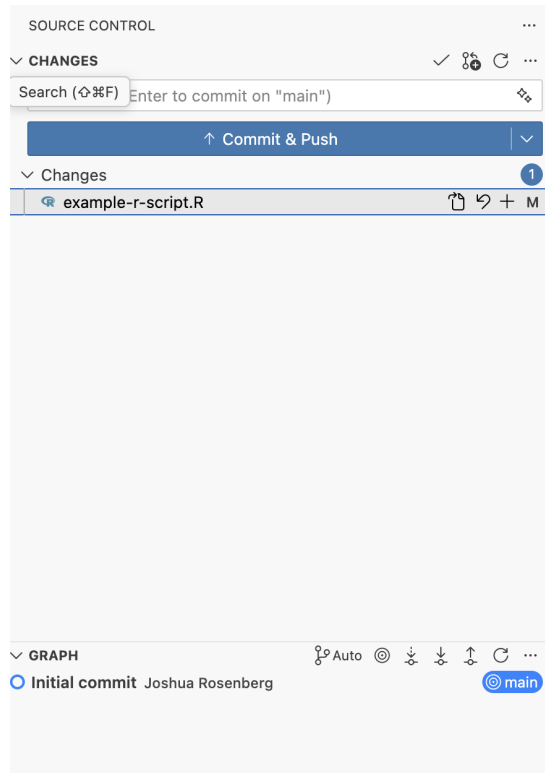


Figure 1.3. Source Control panel for Git operations.

You can also use terminal commands (`git status`, `git add`, `git commit`, `git push`) if you prefer, but the IDE interface handles 90% of everyday tasks.

The key habit: commit often with clear messages (“Add data cleaning script” rather than “changes”), and push regularly so your work is backed up and others can see your progress.

1.9.5.1 A Brief Note on Branches

So far we’ve discussed the basic workflow on a single branch (usually called `main`). Branches let you create parallel versions of your code for experi-

mentation or collaboration without affecting your main work.

Think of branches as separate workspaces: you can try a new analysis approach on a feature branch, and if it works out, merge it back into main. If it does not work, you can simply delete the branch—your main code remains untouched.

When to use branches:

- **Experimenting with new approaches:** Try different analytical methods without risking your working code
- **Collaborating on specific features:** Each person works on their own branch, then merges when ready
- **Maintaining different versions:** Keep a stable main branch while developing updates separately

Your IDE makes branch management visual. In Positron, the Source Control panel shows your current branch and lets you create, switch, and merge branches through menus. In RStudio, the Git pane has a branch dropdown for the same operations.

For now, working on the main branch is sufficient. We will cover branching in more detail when you need it for collaboration or when your projects become more complex.

1.10 Summary

This chapter gave you a working environment. When you are ready to explore more:

- **Positron Documentation:** Official docs for Positron features
- **RStudio User Guide:** Comprehensive guide to all RStudio capabilities
- **Happy Git and GitHub for the useR:** Everything Git-related for R users
- **GitHub Skills:** Interactive tutorials for Git and GitHub

Advanced features worth exploring later:

- **Code snippets:** Type abbreviations that expand to code templates
- **Debugger:** Step through code line-by-line when things break
- **Live Share** (Positron/Vs Code): Collaborate in real-time by sharing your workspace with others

R

2.1 Overview

We are not going to teach you RStudio comprehensively because excellent resources already exist for that purpose. We particularly recommend *R for Data Science*, *Data Science in Education Using R*, the RStudio primers, and *Happy Git and GitHub for the useR*.

Instead, this chapter covers just enough to reproduce our workflows and extend them to your own data science work. Think of this as the minimal viable setup for doing the work in this book.

2.2 Three Ways to Write R Code

Positron supports multiple file types for writing R code. You will encounter all three in data science work:

2.2.1 R Scripts (.R)

Plain text files containing R code, executed line-by-line or all at once. Great for data processing pipelines, functions, and code you will reuse.

Create one: File → New File → R Script

```
# This is an R script
# Lines starting with # are comments
data <- read.csv("mydata.csv")
summary(data)
```

Here is a more realistic example using student assessment data. Save this as `explore-data.R`:

```
library(tidyverse)
students <- read_csv(
  "data/oulad-students-and-assessments.csv"
)
summary(students$mean_weighted_score)
table(students$final_result)
```

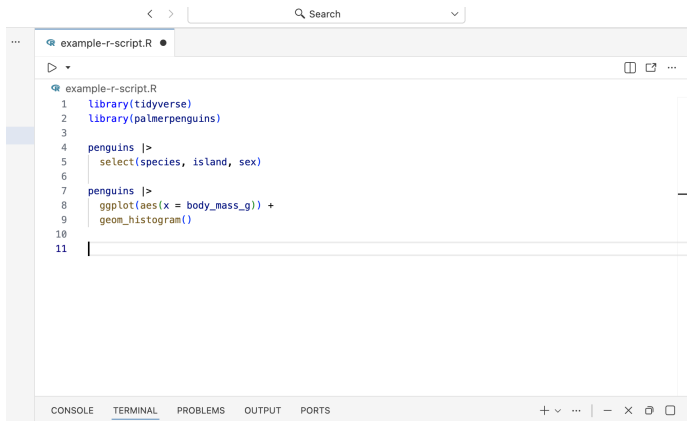


Figure 2.1. R script in the source pane with data-loading and summary commands.

This script loads the `tidyverse` package (more on this in the next chapter), reads a CSV file, and produces quick summaries. You can run it line-by-line using `Cmd/Ctrl + Enter`, or run the entire script at once. The output appears in the Console, but the script itself is saved for later use.

2.2.2 R Markdown (.Rmd)

Documents that weave together narrative text (in Markdown) and code chunks. When you “knit” an R Markdown file, it executes the code and generates a formatted document (HTML, PDF, or Word).

You should know R Markdown exists because you will encounter it in older resources and projects, but we are not using it in this book.

2.2.3 Quarto (.qmd)

Quarto is the successor to R Markdown, with better multi-language support (R, Python, Julia) and more consistent syntax. It is what we will use throughout this book.

Create one: File → New File → Quarto Document

The structure looks similar to R Markdown, but the rendering engine is more powerful:

```
---
title: "My Analysis"
format: html
---
```${r}
mean(1:10)
```
```

Click the “Render” button (or Ctrl/Cmd + Shift + K) to execute all code and generate your document.

Here is a more complete Quarto example analyzing student data:

```
---
title: "OULAD Student Analysis"
format: html
---
## Loading Data
```${r}
library(tidyverse)
students <- read_csv(
 "data/oulad-students-and-assessments.csv"
)
```

## Pass Rates by Gender
```${r}
students |>
 count(gender, pass)
```

The table summarizes pass outcomes for each gender.
```

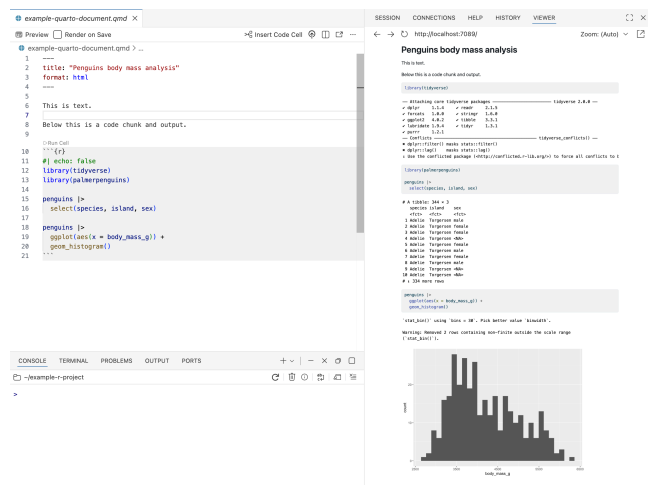


Figure 2.2. Rendered Quarto document combining narrative, code, and output.

When you render this document, Quarto executes each code chunk in order, captures the output, and weaves it together with your explanatory text. The result is an HTML document where readers see your code, your results, and your interpretation all together. This makes your analysis reproducible in that anyone with your data can re-run your Quarto file and get the same results.

Key difference for beginners: Think of R scripts as “just code” and Quarto as “code + explanation + output” in one document. Use scripts for behind-the-scenes work, Quarto for analysis you want to share or understand later.

2.2.4 Controlling Chunk Behavior with Options

Quarto lets you control what appears in your rendered document using chunk options. These are specified at the top of each code chunk using `#|` syntax.

Common options include:

- `#| echo: false` — Hide the code, show only the output
- `#| eval: false` — Show the code but do not run it
- `#| warning: false` — Suppress warning messages

- `#| message: false` — Suppress messages (like package loading notifications)
- `#| fig-width: 8` and `#| fig-height: 6` — Control plot dimensions in inches

Here is an example showing how to hide code while displaying a plot:

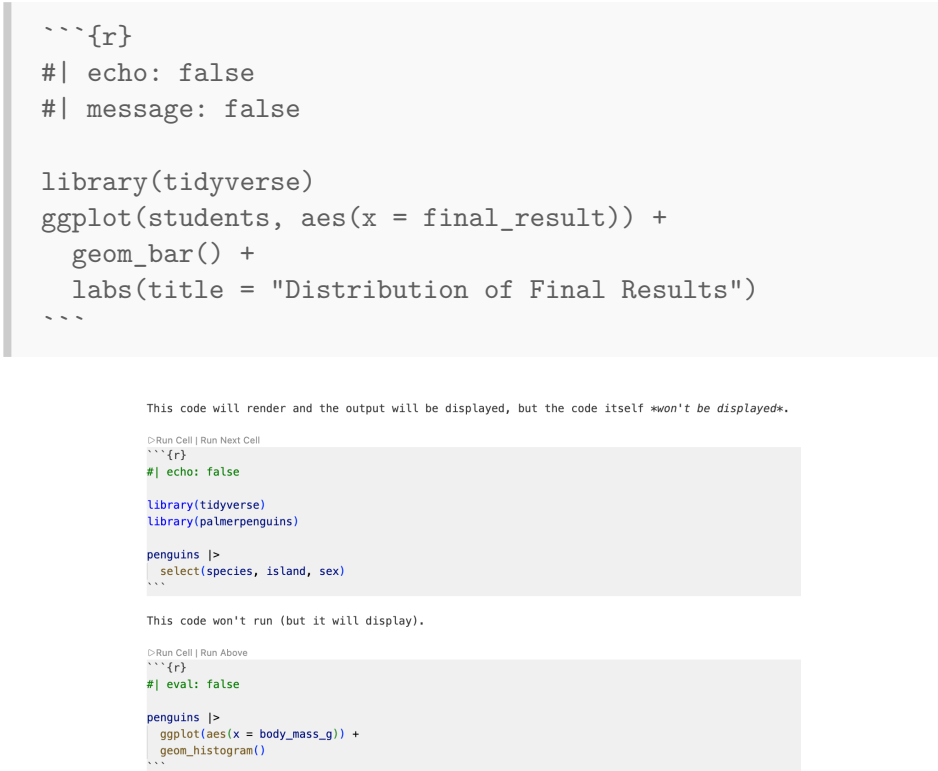


Figure 2.3. Quarto chunk options for controlling code visibility and execution output.

When rendered, readers see the plot but not the code that created it. This is useful for final reports where you want to emphasize results rather than implementation details.

You can also set options globally for the entire document by adding them to your YAML header. For example, to hide all code by default:

```

---
title: "My Analysis"
format: html
execute:
  echo: false
---
```

Individual chunks can override global settings. For more options, see the Quarto documentation on execution options.

2.3 Console vs. Source: Exploration vs. Preservation

A common pattern in data science work:

- **Console:** Quick exploration, testing ideas, checking output. Code here disappears when you close Positron.
- **Source** (scripts/Quarto): Code you want to keep, reproduce, or share. This is your permanent record.

Here is what this looks like in practice. Try this in the Console for a quick calculation:

```

# In Console - just exploring
mean(c(85, 92, 78, 95, 88))
[1] 87.6
```

That is useful for a quick check, but if you want to preserve this process for later:

```

# In a script - keeping a record
test_scores <- c(85, 92, 78, 95, 88)
average_score <- mean(test_scores)
print(average_score)
```

The second approach creates a record you can return to, modify, and share. If you later need to change the scores or calculate something else, the script preserves your workflow.

Early on, you might type everything in the Console. That is fine for learning! But as soon as you do something you want to remember, put it in a script or Quarto file.

2.4 When You Get Stuck

2.4.1 Reading Error Messages

When something goes wrong, R prints an error message in the Console (usually in red). These messages often look cryptic at first, but they are trying to help:

```
Error in mean(x) : object 'x' not found
```

This tells you exactly what went wrong: R could not find an object called `x`. Either you misspelled it, or you have not created it yet.

2.4.2 Getting Help on Functions

R has built-in documentation for every function:

```
?mean          # Opens help for the mean() function
# Search all help files for "regression"
??regression
```

Help files include descriptions, arguments, examples, and related functions.

2.4.3 Where Computational Social Scientists Get Help

When you are truly stuck, the R community is helpful:

- RStudio Community: Welcoming forum for all R questions
- Stack Overflow: Searchable archive of millions of questions

Before asking a question, try searching for your error message. Someone has likely encountered it before. Of course, turning to AI can also help, as we describe next.

2.5 Summary

This chapter covered the essential R skills needed to follow the workflows in this book: writing code in scripts, R Markdown, and Quarto documents; loading and inspecting data; computing basic summaries; producing simple visualizations; and reading error messages when things go wrong. These fundamentals carry through every subsequent chapter.

2.5.1 Large Language Models and AI Tools for Getting Unstuck

In Section 3, we cover using large language models as part of a research workflow. But LLMs are also useful for getting unstuck when you are learning and this is increasingly how working data scientists operate.

Two approaches work well:

1. **Ask LLMs directly:** ChatGPT, Claude, or your preferred model can explain error messages, suggest debugging approaches, and clarify confusing documentation. Copy your error message, describe what you were trying to do, and ask for help.
2. **Use AI tools directly in Positron:** AI coding assistants can work inside your editor to suggest code completions, explain what existing code does, and generate boilerplate. We will cover how to set these up in the next section.

The best programmers do not memorize every function—they know how to find answers quickly. LLMs have become part of that toolkit, alongside documentation, Stack Overflow, and experimentation. Use them when you are stuck, but also pay attention to *why* solutions work. That is how you build intuition.

2.5.1.1 Using AI Tools Directly in Positron

Positron, being built on VS Code, supports AI assistant extensions that can help you write code more efficiently and understand errors more quickly.

Positron Assistant

Positron includes a built-in AI chat panel called Positron Assistant. You can open it from the sidebar (look for the chat icon) or with the keyboard shortcut `Ctrl/Cmd + Shift + I`. Positron Assistant is context-aware—it can see your open files, your console history, and the variables in your environment, so you can ask questions about your code without copying and pasting everything into an external tool.

For example, if you have just loaded a dataset and are not sure how to reshape it, you can highlight the relevant code and ask the assistant to explain it, suggest next steps, or help you fix an error. You can also ask it to generate code from a plain-language description, like “make a bar chart of pass rates by gender.” The assistant will write code that fits the context of your current session, using the packages you have already loaded and the data you are working with.

Positron Assistant connects to a language model provider that you configure in your settings—options include Anthropic (Claude), OpenAI, and others. Some providers require a paid API key; check Positron’s documentation at <https://positron.posit.co/> for current setup instructions. Because the assistant runs inside Positron rather than in a separate browser tab, it reduces the friction of switching between your code and a help tool, which makes it especially useful when you are learning.

When to Use Inline AI vs. External LLMs

Different tools excel at different tasks:

- **Inline AI (Copilot):** Quick code completions, writing repetitive code, remembering function syntax, generating common patterns
- **External LLMs (ChatGPT/Claude):** Complex explanations, understanding concepts, debugging logic errors, learning new approaches

Many programmers use both: Copilot for day-to-day coding, external LLMs when they are truly stuck or learning something new.

Important Caution

AI suggestions are not always correct. Always review code before accepting it, especially for data analysis where subtle errors can lead to wrong conclusions. Use AI tools to speed up your work, but verify that the code does what you think it does.

Please see LLM Methods section (intro) for more information about the relationship between you, data, and LLMs.

Tidyverse

3.1 Overview

The tidyverse is a collection of R packages designed for data science with a shared philosophy: code should be readable, consistent, and focused on the data transformations you are actually trying to accomplish (Wickham et al., 2019; Wickham & Grolemund, 2017). If you have struggled with base R's sometimes cryptic syntax, the tidyverse may feel like a relief.

Most computational educational research workflows rely on tidyverse packages for data manipulation, visualization, and modeling. Learning this ecosystem means learning the tools your collaborators use, the examples you will find online, and the approaches that scale from quick exploration to publication-ready analysis (Wickham et al., 2019; Wickham & Grolemund, 2017).

3.2 What is in the Tidyverse?

The tidyverse is actually a meta-package that loads eight core packages when you run `library(tidyverse)`:

- **ggplot2**: Data visualization with a grammar of graphics
- **dplyr**: Data manipulation (filtering, selecting, summarizing, joining)
- **tidyr**: Reshaping data between wide and long formats
- **readr**: Reading rectangular data (CSV, TSV) efficiently
- **purrr**: Functional programming tools for iteration
- **tibble**: Modern reimagining of data frames
- **stringr**: String manipulation
- **forcats**: Working with categorical variables (factors)

Most of the work we do in this chapter relies heavily on **dplyr** and **ggplot2**, with the others playing supporting roles as needed.

3.3 Installing and Loading the Tidyverse

You install packages like any others in R.

Installation (one time):

```
install.packages("tidyverse")
```

Loading (at the start of each session or script):

```
library(tidyverse)
```

When you load the tidyverse, you will see a message listing which packages were attached and any conflicts (functions from tidyverse packages that mask base R functions). The conflicts are normal and intentional—tidyverse functions are generally preferable for data science work.

3.4 Working with Real Data

Let us load actual data to see the tidyverse in action. We will use student assessment data from the Open University Learning Analytics Dataset (OULAD), which contains information about student demographics and course outcomes.

```
student_file <- "data/oulad-students-and-assessments.csv"
students <- read_csv(student_file)

students |>
  select(gender, final_result, mean_weighted_score) |>
  glimpse(width = glimpse_width)
```

The `glimpse()` function shows you the structure of your data: how many rows and columns, what type each column is (character, numeric, etc.), and the first few values. This dataset contains information about students (gender, age, region, disability status) along with their course performance (final_result, pass status, mean_weighted_score).

This is the dataset we will use in the examples below. It represents the kind of educational data you will work with in computational educational

research: thousands of observations with both categorical and continuous variables.

3.5 Essential dplyr Functions

The dplyr package provides functions for the most common data manipulation tasks. Here are the ones you will use repeatedly.

3.5.1 filter() - Selecting Rows

Use `filter()` to keep only rows that meet certain conditions:

```
# Keep only students who passed
passed_students <- students |>
  filter(pass == 1)

# Multiple conditions with AND (comma means AND)
passed_females <- students |>
  filter(pass == 1, gender == "F")

# OR condition using |
good_outcomes <- students |>
  filter(pass == 1 | mean_weighted_score > 800)
```

This gives us 22,333 passed students from the original 32,593. Use `==` for equality tests, `>` and `<` for comparisons, and combine conditions with commas (AND) or `|` (OR).

3.5.2 select() - Choosing Columns

Use `select()` to pick which columns to keep or remove:

```
# Select specific columns
student_basics <- students |>
  select(id_student, gender, age_band, final_result)

# Remove columns with minus sign
student_no_dates <- students |>
```

```

select(-date_registration, -date_unregistration)

# Select by pattern
student_demographics <- students |>
  select(starts_with("id"), gender, region)

```

The `select()` function is useful when you have many columns but only need a few for analysis. Helper functions like `starts_with()`, `ends_with()`, and `contains()` make it easy to select groups of related columns.

3.5.3 `mutate()` - Creating or Modifying Columns

Use `mutate()` to add new columns or change existing ones:

```

# Create new column
students <- students |>
  mutate(high_achiever = mean_weighted_score > 800)

# Modify existing column
students <- students |>
  mutate(age_band = factor(age_band))

# Multiple new columns at once
students <- students |>
  mutate(
    score_category = case_when(
      mean_weighted_score > 800 ~ "High",
      mean_weighted_score > 650 ~ "Medium",
      TRUE ~ "Low"
    )
  )

```

The `case_when()` function inside `mutate()` handles conditional logic: if `mean_weighted_score` is over 800, assign “High”; if over 650, assign “Medium”; otherwise (`TRUE`), assign “Low”.

3.5.4 summarize() - Calculating Summaries

Use `summarize()` to collapse your data into summary statistics:

```
# Overall statistics
students |>
  summarize(
    mean_score = mean(mean_weighted_score, na.rm = TRUE),
    pass_rate = mean(pass, na.rm = TRUE),
    n_students = n()
  )
```

The `na.rm = TRUE` argument tells R to ignore missing values when calculating means. The `n()` function counts the number of rows.

3.5.5 group_by() - Operations by Group

Use `group_by()` before `summarize()` to calculate statistics for each group separately:

```
# Pass rates by gender
students |>
  group_by(gender) |>
  summarize(
    pass_rate = mean(pass, na.rm = TRUE),
    n = n()
  )
```

```
# Multiple grouping variables
students |>
  group_by(gender, disability) |>
  summarize(
    mean_score = mean(mean_weighted_score, na.rm = TRUE)
  )
```

After `group_by()`, subsequent operations happen separately for each group. This is one of the most powerful patterns in data analysis: split your data into groups, apply a function to each group, then combine the results.

3.5.6 arrange() - Sorting Rows

Use `arrange()` to reorder rows:

```
arrange_vars <- students |>
  transmute(id = id_student, result = final_result,
            score = mean_weighted_score)

# Sort by score (ascending by default)
arrange_vars |>
  arrange(score) |>
  slice_head(n = 3)

# Sort descending
arrange_vars |>
  arrange(desc(score)) |>
  slice_head(n = 3)

# Multiple sort keys
arrange_vars |>
  arrange(result, desc(score)) |>
  slice_head(n = 3)
```

This is useful when you want to see the highest or lowest values, or when you need data in a particular order for plotting or reporting.

3.5.7 count() - Quick Frequency Tables

Use `count()` as a shortcut for `group_by()` + `summarize(n = n())`:

```
# Count final results
students |>
  count(final_result)

# Count with percentages
students |>
  count(final_result) |>
  mutate(percent = n / sum(n) * 100)
```

The `count()` function is perfect for quickly understanding the distribution of categorical variables.

3.6 The Pipe Operator: `|>` and `%>%`

The tidyverse introduced a distinctive style: piping operations together to create readable data transformation pipelines. Instead of nesting functions inside each other, you “pipe” the output of one function into the next.

R now has a native pipe operator `|>` (introduced in R 4.1), while the tidyverse originally used `%>%` from the `magrittr` package. They work almost identically for most purposes.

Here is a concrete example using the `dplyr` functions we just learned:

```
# Without pipes (nested, hard to read)
summarize(
  group_by(
    filter(students, pass == 1),
    gender
  ),
  mean_score = mean(
    mean_weighted_score,
    na.rm = TRUE
  )
)
```

```
# With pipes (clear, step-by-step)
students |>
  filter(pass == 1) |>
  group_by(gender) |>
  summarize(
    mean_score = mean(
      mean_weighted_score,
      na.rm = TRUE
    )
  )
```

The piped version reads like instructions: “Take students, *then* keep only those who passed, *then* group by gender, *then* calculate mean scores.” Each

step is on its own line, making it easy to follow the logic.

We will use the native pipe `|>` throughout this book because it is now built into R, but you will encounter `%>%` in older code and documentation. For practical purposes, they are interchangeable in most tidyverse workflows.

How to read it: Think of `|>` as “then.” The pipe takes the result from the left and passes it as the first argument to the function on the right.

3.7 Data Visualization with ggplot2

The ggplot2 package uses a “grammar of graphics” approach where you build plots in layers. You start with your data, specify how variables map to visual properties (x-axis, y-axis, colors), then add geometric objects like points, bars, or lines.

3.7.1 Basic Structure

Every ggplot follows this template:

```
ggplot(data = DATA, aes(x = X_VAR, y = Y_VAR)) +  
  geom_FUNCTION()
```

The `aes()` function (short for “aesthetics”) maps your data columns to visual properties. The `geom_` functions specify what kind of plot to draw.

3.7.2 Bar Charts - Categorical Data

Use `geom_bar()` to visualize counts of categorical variables:

```
# Simple bar chart  
students |>  
  ggplot(aes(x = final_result)) +  
  geom_bar()
```

This automatically counts how many students have each final result (Pass, Fail, Distinction, Withdrawn) and displays the counts as bars.

```
# Stacked bar chart by group
students |>
  ggplot(aes(x = gender, fill = final_result)) +
  geom_bar()
```

The `fill` aesthetic colors the bars by `final_result`, creating stacked bars that show how outcomes differ between genders.

```
# Side-by-side bars
students |>
  ggplot(aes(x = gender, fill = final_result)) +
  geom_bar(position = "dodge")
```

Setting `position = "dodge"` puts bars next to each other instead of stacking them, making it easier to compare counts directly.

3.7.3 Histograms - Distributions

Use `geom_histogram()` to see the distribution of continuous variables:

```
# Distribution of scores
students |>
  ggplot(aes(x = mean_weighted_score)) +
  geom_histogram(binwidth = 50)
```

The `binwidth` argument controls how wide each bar is. Smaller values show more detail, larger values show broader patterns.

```
# Separate distributions by group
students |>
  filter(!is.na(mean_weighted_score)) |>
  ggplot(aes(
    x = mean_weighted_score,
    fill = factor(pass)
  )) +
  geom_histogram(
    binwidth = 50,
    position = "identity",
    alpha = 0.5
```

```
)
```

Using `position = "identity"` overlays the histograms, and `alpha = 0.5` makes them semi-transparent so you can see both distributions.

3.7.4 Boxplots - Comparing Distributions

Use `geom_boxplot()` to compare distributions across groups:

```
# Scores by final result
boxplot_data <- students |>
  select(result = final_result, score = mean_weighted_score)

ggplot(boxplot_data, aes(result, score)) +
  geom_boxplot()
```

Boxplots show the median (middle line), quartiles (box edges), and outliers (individual points). This makes it easy to see that Distinction students have higher median scores than Pass students, who score higher than those who Fail or Withdraw.

```
# With colors
ggplot(boxplot_data, aes(result, score, fill = result)) +
  geom_boxplot()
```

Adding `fill` colors each boxplot by the category, making them easier to distinguish.

3.7.5 Scatter Plots - Relationships

Use `geom_point()` to explore relationships between two continuous variables:

```
# Registration timing vs. scores
scatter_data <- students |>
  filter(!is.na(mean_weighted_score)) |>
  select(date_registration, mean_weighted_score)

ggplot(
```

```

scatter_data,
aes(date_registration, mean_weighted_score)
) +
geom_point(alpha = 0.3)

```

The `alpha` parameter makes points semi-transparent, which helps when many points overlap. This plot would show whether students who register earlier tend to score higher or lower.

```

# Add a trend line
ggplot(
  scatter_data,
  aes(date_registration, mean_weighted_score)
) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "lm")

```

Adding `geom_smooth(method = "lm")` fits a linear model and displays the trend line with a confidence band, making patterns easier to see.

3.7.6 Improving Plots with Labels and Themes

Good plots need clear labels:

```

students |>
  ggplot(aes(x = final_result)) +
  geom_bar(fill = "steelblue") +
  labs(
    title = "Distribution of Student Outcomes",
    x = "Final Result",
    y = "Number of Students"
  ) +
  theme_minimal()

```

The `labs()` function adds titles and axis labels. The `theme_minimal()` function applies a clean, simple theme. Other themes include `theme_bw()`, `theme_classic()`, and `theme_light()`.

3.7.7 Faceting - Small Multiples

Use faceting to create separate plots for different groups:

```
students |>
  ggplot(aes(x = final_result)) +
  geom_bar() +
  facet_wrap(~code_module) +
  scale_x_discrete(labels = abbreviate)
```

This creates a grid of small bar charts, one for each module. It is useful when you want to compare patterns across many categories.

```
# Grid by two variables
students |>
  filter(!is.na(mean_weighted_score)) |>
  ggplot(aes(x = mean_weighted_score)) +
  geom_histogram(binwidth = 50) +
  facet_grid(gender ~ disability)
```

`facet_grid()` creates a matrix of plots: rows for one variable (gender), columns for another (disability). This lets you see how score distributions vary across combinations of factors.

3.8 Summary

You've now seen core tidyverse functions in action with real data. The seven dplyr functions covered here—`filter()`, `select()`, `mutate()`, `summarize()`, `group_by()`, `arrange()`, and `count()`—handle the majority of everyday data manipulation tasks. The ggplot2 visualizations—bar charts, histograms, boxplots, scatter plots, and faceting—cover most exploratory analysis needs.

The subsequent chapters in this book use these functions extensively, and we will introduce additional tidyverse capabilities as they become relevant. You will continue learning by doing, working with real data and real research questions.

When you want to go deeper:

- *R for Data Science (2e)*: The definitive guide to the tidyverse

- Tidyverse documentation: Reference docs for all packages
- RStudio Cheatsheets: Visual quick references for dplyr, ggplot2, and more

The tidyverse community is large and welcoming. When you get stuck, someone has likely asked your question before.

Data Science Methods

Overview

This section introduces the core data science methods that form the basis of educational data analysis (Baker & Inventado, 2014; Lang et al., 2022). Across the next three chapters, you will learn how to analyze **textual**, **relational**, and **numeric** data — three major forms of information that appear in learning environments, institutional records, and educational research projects (Carolan, 2014; James et al., 2021; Silge & Robinson, 2017).

These chapters emphasize hands-on, transparent, and reproducible approaches using R. By engaging with real examples, you will gain practical experience in transforming raw educational data into interpretable results that inform theory and practice.

The Analytical Scope of This Section

| Chapter | Data Type | Analytical Focus | Example Research Question |
|------------------|--------------------------|---|--|
| Chapter 4 | Text Data (unstructured) | Natural language processing, tokenization, sentiment, topic modeling | “What themes emerge in student reflections or policy statements?” |
| Chapter 5 | Relational Data | Social network analysis: centrality, community detection, visualization | “How do students or instructors connect and collaborate in learning networks?” |
| Chapter 6 | Numeric / Big Data | Statistical modeling, regression, clustering, predictive analysis | “Which factors best predict academic outcomes or engagement?” |

These three perspectives together illustrate how computational techniques can capture different dimensions of learning—language, interaction, and measurement.

Setting Up the Computational Environment

Before exploring the examples, ensure that your R environment contains the essential packages used throughout this section.

```
install.packages(c(  
  # Core workflow and visualization  
  "tidyverse", "ggplot2", "readr", "stringr",  
  
  # Text analysis  
  "tidytext", "quanteda", "textdata",  
  
  # Network analysis  
  "igraph", "ggraph", "tidygraph",  
  
  # Numeric and machine learning tools  
  "caret", "cluster"  
))
```

Optional visualization and interaction packages:

```
install.packages(c("plotly", "RColorBrewer", "visNetwork"))
```

Tip: Use a consistent project structure so each chapter builds on the same foundation:

```
project/  
  data/          # datasets  
  scripts/       # reusable code  
  outputs/       # tables and processed files  
  figures/       # charts and visualizations
```

This structure promotes reproducibility and helps keep analysis pipelines organized.

A General Computational Workflow

Regardless of data type, the analytical logic follows a similar cycle:

1. **Load data** — read files from local or online sources.
2. **Clean data** — handle missing values, normalize text or numeric fields.
3. **Transform data** — create tokens, build networks, or scale variables.
4. **Analyze** — apply the method appropriate to the data form.
5. **Visualize and interpret** — generate plots and summaries to support interpretation.

```
library(tidyverse)

data <- read_csv("data/example.csv")

cleaned <- data |>
  mutate(across(
    everything(),
    str_squish
  )) |>
  drop_na()

summary(cleaned)
```

This five-step workflow—load, clean, transform, analyze, interpret—appears throughout all chapters in this section.

Ethics, Transparency, and Reproducibility

Educational data often include sensitive or identifiable information.

Responsible computational research requires attention to both ethical and methodological rigor.

- **Privacy:** Remove or anonymize all personal identifiers.
- **Transparency:** Keep analysis scripts in Quarto or R Markdown files for version control.
- **Reproducibility:** Record package versions and parameters used in each analysis.

- **Interpretability:** Combine quantitative patterns with contextual educational insight.

Ethical and transparent practices ensure that computational results remain credible, interpretable, and usable for improving learning.

Transition to Analytical Chapters

With the environment prepared and workflow established, you are ready to begin applying computational methods to real educational data:

- **Chapter 4 — Text Analysis:** Working with unstructured language to uncover patterns in meaning and discourse.
- **Chapter 5 — Network Analysis:** Examining connections and relationships among learners, instructors, or resources.
- **Chapter 6 — Numeric and Big Data:** Exploring structured data to identify trends and predictors in education.

The following chapter begins with text data, illustrating how natural language processing can transform qualitative information into structured, interpretable results.

Text Data

4.1 Overview

In educational research, analyzing text data is typically considered the work of qualitative researchers. Traditionally, qualitative research involves identifying patterns in non-numeric data, and this pattern recognition is done manually (Creswell & Poth, 2018). This process is time-intensive but can yield rich results — for example, through human coding of interview transcripts, policy documents, or classroom artifacts (Saldaña, 2021).

With the advent of new software, we can capture and analyze text data in ways that were previously not possible, such as web scraping, accessing social media APIs, or downloading large online documents. Given the increased scale and variety of available text data, analysis now benefits from computational approaches — including dictionary-based, frequency-based, and machine learning methods — that go beyond manual coding (Grimmer et al., 2022; Silge & Robinson, 2017). These methods allow educational researchers to ask new types of research questions, expanding the scope and depth of possible insights. For example, Rosenberg et al. (2021) used computational text analysis to examine public sentiment about science education policy across thousands of social media posts — a scale of analysis not feasible through traditional qualitative methods.

4.2 Accessing Text Data

Text can be found and collected in many different ways. For example, social media can serve as a rich source of text data (e.g., Reddit posts), and text created in classrooms—especially online (e.g., student discussion posts)—can also be analyzed. In this section, we will cover a few basic ways of accessing text data. Please note that although we cover some prominent ways, this is by no means an exhaustive list. Therefore, please refer to the additional resources section at the end of the chapter to dive deeper.

4.2.1 Web Scraping (Unstructured or API)

What is Web Scraping?

Web scraping refers to the automated process of extracting data from web pages. It is particularly useful when dealing with extensive lists of websites that would be tedious to mine manually. A typical web scraping program follows these steps:

1. Loads a webpage.
2. Downloads the HTML or XML structure.
3. Identifies the desired data.
4. Converts the data into a format suitable for analysis, such as a data frame.

In addition to text, web scraping can also be used to download other content types, such as audio-visual files.

Is Web Scraping Legal?

Web scraping is not governed by a single universal rule, so researchers should separate platform policy questions from legal and ethical risk assessment (Salganik, 2019). In practice, Terms of Service define platform-level permissions (for example, whether automated collection is allowed), while `robots.txt` provides crawler guidance; neither should be treated as a complete legal determination. Legal exposure depends on jurisdiction and context (for example, public vs. access-restricted content, personal data handling, and whether technical barriers are bypassed). For educational research, a practical standard is to document collection decisions, minimize personally identifiable data, and align scraping procedures with institutional review and data-governance requirements (Salganik, 2019).

Reading a Web Page into R

Once permissions are confirmed, the first step in web scraping is to download the webpage's source code into R. In R, a great package to do this is `rvest`.

```
# Install and load the rvest package
install.packages("rvest")
library(rvest)
```

To demonstrate, we will scrape a simple Wikipedia page. Static pages, which lack interactive elements like JavaScript, are simpler to scrape. You can view the page's HTML source in your browser by selecting **Developer Tools > View Source**.

```
# Load the webpage
wikipedia_page <- read_html("https://en.wikipedia.org")

# Verify that the webpage loaded successfully
wikipedia_page
```

Parsing HTML

The next challenge is extracting specific information from the HTML structure. HTML files have a “tree-like” format, allowing us to target particular sections. Use your browser’s “Developer Tools” to inspect elements and locate the data. Right-click the desired element and select **Inspect** to view its structure.

To isolate data sections within the HTML structure, identify the **XPath** or **CSS selectors**. For instance:

```
# Extract specific section using XPath
section_of_wikipedia <- html_node(
  wikipedia_page,
  xpath = '//*[@id="mw-content-text"]/div/table'
)
head(section_of_wikipedia)
```

To convert the extracted section into a data frame, use `html_table()`:

```
# Convert the extracted data into a table
health_rankings <- html_table(section_of_wikipedia)
# Display the first two columns
head(health_rankings[, 1:2])
```

Parsing with CSS Selectors

For more complex web pages, CSS selectors can be an alternative to XPath. Tools like **Selector Gadget** can help identify the required CSS selectors.

For example, to scrape event information from Duke University's main page:

```
# Load the webpage
duke_page <- read_html("https://www.duke.edu")

# Extract event information using CSS selector
duke_events <- html_nodes(
  duke_page,
  css = "li:nth-child(1) .epsilon"
)
html_text(duke_events)
```

Scraping with Selenium

For tasks involving interactive actions (e.g., filling search fields), use **RSelenium**, which enables automated browser operations.

To set up Selenium, install the **Java SE Development Kit** and **Docker**. Then, start Selenium in R:

```
# Install and load RSelenium
install.packages("RSelenium")
library(RSelenium)

# Start a Selenium session
rD <- rsDriver()
remDr <- rD$client
remDr$navigate("https://www.duke.edu")
```

To automate data entry, identify the CSS selector for the search box and input the query:

```
# Find the search box element and enter a query
search_box <- remDr$findElement(
  using = "css selector",
  value = "fieldset input"
)
search_box$sendKeysToElement(
  list("data science", "\uE007")
) # Enter key
```

Web Scraping within a Loop

To scrape multiple pages, embed the code within a loop to automate tasks across different URLs. Since each site may have a unique structure, generalized scraping can be time-intensive and error-prone. Implement error handling to manage interruptions.

When to Use Web Scraping

Web scraping is appropriate if:

- **Page structure is consistent across sites:** For example, a government site with date suffixes but a uniform layout.
- **Manual data collection is prohibitive:** For extensive text or embedded tables.

When feasible, consider alternatives like APIs or data-entry services (e.g., Amazon Mechanical Turk) for better efficiency and legal compliance.

What is an API?

An Application Programming Interface (API) is a set of protocols that allows computers to communicate and exchange information. A common type is a **REST** API (Representational State Transfer), where clients make stateless HTTP requests (for example, **GET** and **POST**) to resource URLs and receive structured responses, often in JSON. APIs provide standardized access to data, services, and functionalities, making them essential in software development.

When to Use an API

APIs are commonly used for:

- **Integrating with Third-Party Services:** APIs connect applications to services like payment gateways or social media.
- **Accessing Data:** APIs retrieve data from systems or databases (e.g., real-time weather data).
- **Automating Tasks:** APIs automate processes within applications, such as email marketing.
- **Building New Applications:** APIs allow developers to build new apps or services (e.g., a mapping API for navigation).
- **Streamlining Workflows:** APIs enable seamless communication and data exchange across systems.

Using Reddit API with RedditExtractor

Reddit is a social media platform featuring a complex network of users and discussions, organized into “subreddits” by topic. `RedditExtractor`, an R package, enables data extraction from Reddit to identify trends and analyze interactions.

```
# Install and load RedditExtractor
install.packages("RedditExtractor")
library(RedditExtractor)

# Access data from the GenAI subreddit
GenAI_reddit <- find_thread_urls(
  subreddit = "GenAI",
  sort_by = "new",
  period = "day"
)
view(GenAI_reddit)
```

Researchers can search online (or GitHub) to find packages that are specifically designed to capture other platforms, like the Reddit example here.

4.2.2 Audio Transcripts (Zoom, etc.)

Audio transcripts are a rich source of text data, especially useful for capturing spoken content from meetings, interviews, or webinars. Many platforms, such as Zoom, provide automated transcription services that can be downloaded as text files for analysis. By processing these transcripts, researchers can analyze conversation themes, sentiment, or other linguistic features. Here's how to access and prepare Zoom transcripts for analysis in R.

Key Steps for Extracting Text Data from Audio Transcripts

1. Access the Zoom Transcript

- Log in to your Zoom account.
- Navigate to the “Recordings” section.
- Select the recording you wish to analyze and download the “Audio Transcript” file.

2. Import the Transcript into R

Once the file is downloaded, you can load it into R for analysis. Depending on the file format (usually a .txt file with tab or comma delimiters), use `read.table()`, `read.csv()`, or functions from the `readr` package to load the data.

```
# Load the transcript into R
transcript_data <- read.table(
  "path/to/your/zoom_transcript.txt",
  sep = "\t",
  header = TRUE
)
```

Adjust the `sep` parameter based on the delimiter used in the transcript file (typically `\t` for tab-delimited files).

3. Data Cleaning (if necessary)

Clean up the text data to remove unnecessary characters, standardize formatting, and prepare it for further analysis.

• Remove Unwanted Characters

Use `gsub()` to eliminate special characters and punctuation, keeping only alphanumeric characters and spaces.

```
# Remove special characters
pattern <- "[^a-zA-Z0-9 ]"
clean_text <- gsub(
  pattern, "", transcript_data$text
)
transcript_data$text <- clean_text
```

- **Convert Text to Lowercase**

Standardize text to lowercase for consistency in text analysis.

```
# Convert text to lowercase
transcript_data$text <- tolower(transcript_data$text)
```

4.2.3 PDF

PDF files are a valuable source of text data, often found in research publications, government documents, and industry reports. We'll explore two main methods for extracting text from PDFs:

1. **Extracting from Local PDF Files:** This method involves accessing and parsing text from PDF files stored locally, providing tools and techniques to efficiently retrieve text data from offline documents.
2. **Downloading and Extracting PDF Files:** This approach covers downloading PDFs from online sources and extracting their text. This method is useful for scraping publicly available documents from websites or databases for research purposes.

3. **PDF Data Extractor (PDE)**

For more advanced PDF text extraction and processing, you can use the PDF Data Extractor (PDE) package. This package provides tools for extracting text data from complex PDF documents, supporting additional customization options for text extraction. PDE is a R package that easily extracts information and tables from PDF files. The `PDE_analyzer_i()` performs the sentence and table extraction while the included `PDE_reader_i()` allows the user-friendly visualization and quick-processing of the obtained results.

Steps for Extracting Text from Local PDF Files

1. Install and Load the `pdftools` Package

Start by installing and loading the `pdftools` package, which is specifically designed for reading and extracting text from PDF files in R.

```
install.packages("pdftools")  
library(pdftools)
```

2. Read the PDF as a Text File

Use the `pdf_text()` function to read the PDF file into R as a text object. This function returns each page as a separate string in a character vector.

```
txt <- pdf_text("path/to/your/file.pdf")
```

3. Extract Text from a Specific Page

To access a particular page from the PDF, specify the page number in the text vector. For example, to extract text from page 24:

```
page_text <- txt[24] # page 24
```

4. Extract Rows into a List

If the page contains a table or structured text, use the `scan()` function to read each row as a separate element in a list. The `textConnection()` function converts the page text for processing.

```
rows <- scan(  
  textConnection(page_text),  
  what = "character",  
  sep = "\n"  
)
```

5. Split Rows into Cells

To further parse each row, split it into cells by specifying the delimiter, such as whitespace (using `"\\s+"`). This converts each row into a list of individual cells.

```
# Example with the 24th row
row <- unlist(strsplit(
  rows[24],
  "\\s+"
))
```

Steps for Downloading and Extracting Text from PDF Files

1. Download the PDF from the Web

Use the `download.file()` function to download the PDF file from a specified URL. Set the mode to `"wb"` (write binary) to ensure the file is saved correctly.

```
link <- paste0(
  "http://www.singstat.gov.sg/docs/",
  "default-source/default-document-library/",
  "publications/publications_and_papers/",
  "cop2010/census_2010_release3/",
  "cop2010sr3.pdf"
)
download.file(link, "census2010_3.pdf", mode = "wb")
```

2. Read the PDF as a Text File

After downloading, read the PDF into R as a text object using the `pdf_text()` function from the `pdftools` package. Each page of the PDF will be stored as a string in a character vector.

```
txt <- pdf_text("census2010_3.pdf")
```

3. Extract Text from a Specific Page

Access the desired page (e.g., page 24) by specifying the page number in the character vector.

```
page_text <- txt[24] # Page 24
```

4. Extract Rows into a List

Use the `scan()` function to split the page text into rows, with each row representing a line of text in the PDF. This creates a list where each line from the page is an element.

```
rows <- scan(
  textConnection(page_text),
  what = "character",
  sep = "\n"
)
```

5. Loop Through Rows and Extract Data

Starting from a specific row (e.g., row 7), loop over each row. For each row:

- Split the text by spaces ("\\s+") using `strsplit()`.
- Convert the result to a vector with `unlist()`.
- If the third cell in the row is not empty, store the second cell as `name` and the third cell as `total`, converting it to a numeric format after removing commas.

```
name <- c()
total <- c()

for (i in 7:length(rows)) {
  row <- unlist(strsplit(rows[i], "\\s+"))
  if (!is.na(row[3])) {
    name <- c(name, row[2])
    value <- gsub(",", "", row[3])
    total <- c(total, as.numeric(value))
  }
}
```

4.2.4 Survey, Discussions, etc.

Surveys and discussion posts are valuable sources of text data in educational research, providing insights into participants' perspectives, opinions, and experiences. These data sources often come from open-ended survey responses, online discussion boards, or educational platforms. Extracting and preparing text data from these sources can reveal recurring themes, sentiment, and other patterns that support both quantitative and qualitative analysis. Below are key steps and code examples for processing text data from surveys and discussions in R.

Key Steps for Processing Survey and Discussion Text Data

1. Load the Data

Survey and discussion data are typically stored in spreadsheet formats like CSV. Begin by loading this data into R for processing. Here, `readr` is used for reading CSV files with `read_csv()`.

```
# Install and load necessary packages
install.packages("readr")
library(readr)

# Load data
survey_data <- read_csv("path/to/your/survey_data.csv")
```

2. Extract Text Columns

Identify and isolate the relevant text columns. For example, if the text data is in a column named “Response,” you can create a new vector for analysis.

```
# Extract text data from the specified column
text_data <- survey_data$Response
```

3. Data Cleaning

Prepare the text data by cleaning it, removing any unnecessary characters, and standardizing the text. This includes removing punctuation, converting text to lowercase, and handling extra whitespace.

- **Remove Unwanted Characters**

Use `gsub()` from base R to remove any non-alphanumeric characters, retaining only words and spaces.

```
# Remove special characters
text_data <- gsub("[^a-zA-Z0-9 ]", "", text_data)
```

- **Convert to Lowercase**

Standardize the text by converting all characters to lowercase.

```
# Convert text to lowercase
text_data <- tolower(text_data)
```

- **Remove Extra Whitespace**

Remove any extra whitespace that may be left after cleaning.

```
# Remove extra spaces
text_data <- gsub("\\s+", " ", text_data)
```

4. Tokenization and Word Counting (Optional)

If further analysis is needed, such as frequency-based analysis, split the text into individual words (tokenization) or count the occurrence of specific words. Here, `dplyr` is used to organize the word counts.

```
# Install and load necessary packages
install.packages("dplyr")
library(dplyr)

# Tokenize and count words
word_count <- strsplit(text_data, " ") %>%
  unlist() %>%
  table() %>%
  as.data.frame()
```

4.3 Methods for Analyzing Text Data

In the following section, we will provide a “recipe” for the educational researcher interested in these new methods of analyzing text data to get you from the initial stages of getting the data to running the analyses and the write up. Often left out is also a research question that suits or requires a method. Since we have a data and method-centric approach here, we will backtrack and also provide a research question, so that you can model after it in your own work. Finally, we will provide a sample results and discussions section.

4.3.1 Frequency-Based Analysis

The purpose of the frequency-based approach is to count the number of words as they appear in a text file, whether it is a collection of tweets, documents, or interview transcripts. This approach aligns with the frequency-coding method (e.g., Saldaña, 2021) and can supplement human coding

by revealing the most/least commonly occurring words, which can then be compared across dependent variables.

Case Study: Frequency-Based Analysis of GenAI Usage Guidelines in Higher Education

As AI writing tools like ChatGPT become more prevalent, educators are working to understand how best to integrate them within academic settings, while many students and instructors remain uncertain about acceptable use cases. Our research into AI usage guidelines from the top 100 universities in North America aims to identify prominent themes and concerns in institutional policies regarding GenAI.

4.3.1.1 Sample Research Questions

To investigate the nature of AI use policies within higher education institutions, our research questions are:

- **RQ1:** What are the most frequently mentioned words in university GenAI writing usage policies?
- **RQ2:** Which keywords reflect common concerns or focal points related to GenAI writing usage in academic settings?

4.3.1.2 Sample Methods

Data Source

The dataset consists of publicly available AI policy texts from 100 universities in the USA. The data was collected from institutional websites, including syllabus guidelines and academic integrity pages, and saved as a CSV file for analysis.

Data Analysis

To analyze the data, we employed a natural language processing (NLP) approach, focusing on word frequency and visualization. We used the `tidytext` package to tokenize the policy text and the `ggplot2` and `wordcloud` packages to represent the results visually.

4.3.1.3 Analysis

Step 1: Load Required Libraries

Install and load libraries for data processing, visualization, and word cloud generation.

```
# Install necessary packages if not already installed
# install.packages(c(
#   "tibble", "dplyr", "tidytext", "ggplot2", "viridis",
#   "tm", "wordcloud", "wordcloud2", "webshot"
# ))

# Load libraries
library(readr)
library(tibble)
library(dplyr)
library(tidytext)
library(ggplot2)
library(viridis)
library(tm)
library(wordcloud)
library(wordcloud2)
library(webshot)
```

Step 2: Load Data

Read the CSV file containing policy texts from the top 100 universities.

```
# Load the dataset
policy_path <- "data/University_GenAI_Policy_Stance.csv"
university_policies <- read_csv(policy_path)
```

Step 3: Tokenize Text and Count Word Frequency

Process the text data by tokenizing words, removing stop words, and counting word occurrences.

```
# Tokenize, remove stop words, and count words
word_frequency <- university_policies |>
  # Tokenize the 'Stance' column
  unnest_tokens(word, Stance) |>
  # Remove common stop words
  anti_join(stop_words) |>
  # Count and sort words by frequency
  count(word, sort = TRUE)

print(word_frequency, n = 6, width = output_width)
```

Step 4: Create a Word Cloud

Generate a word cloud to visualize frequent words in an exploratory way. Word clouds are useful for quick pattern scanning, but they are less precise for comparing magnitudes across terms. For quantitative comparison, bar charts are usually more interpretable (Wilke, 2019).

```
# Create and display the GenAI usage Stance wordcloud
set.seed(1234) # For reproducibility
wordcloud(
  words = word_frequency$word,
  freq = word_frequency$n,
  scale = c(4, 0.5),
  random.order = FALSE,
  min.freq = 10,
  colors = brewer.pal(8, "Dark2"),
  rot.per = 0.35
)
```

Step 5: Visualize Top 12 Words in University Policies

Select the top 12 most frequent words and create a bar chart to visualize the distribution.

```
# Select the top 12 words
top_words <- word_frequency |>
  slice(1:12)
```

```

# Generate the bar chart
policy_word_chart <- ggplot(
  top_words,
  aes(x = reorder(word, n), y = n, fill = word)
) +
  geom_bar(stat = "identity") +
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Top Words in University GenAI Policies",
    x = NULL,
    y = "Frequency",
    caption = "Source: University AI Policy Text",
    fill = "Word"
  ) +
  scale_fill_viridis(discrete = TRUE) +
  geom_text(
    aes(label = n),
    vjust = 0.5,
    hjust = -0.1,
    size = 3
  )

# Print the bar chart
print(policy_word_chart)

```

The methods used above can be written up as follows:

Sample Methods

To examine the lexical content of university generative AI policy stances, the text data were first tokenized into individual words and cleaned by removing stop words. Word occurrences were then counted across the corpus to produce a frequency table of the most common terms, following standard tidy text and text mining procedures for corpus-level term analysis.

The resulting word-frequency distribution was visualized in two complementary ways. First, a word cloud was generated using the *wordcloud* package to provide a qualitative overview

of term prominence, with larger words representing higher frequency and a fixed random seed used to ensure reproducibility. Second, the twelve most frequent words were displayed in a horizontal bar chart using *ggplot2*, which offers a clearer quantitative comparison of term counts than the word cloud alone. The bar chart was enhanced with frequency labels and a viridis fill scale to improve readability and distinguish terms visually.

This frequency-based approach provided a straightforward descriptive summary of the policy corpus and helped identify the most salient vocabulary used in institutional GenAI guidance.

4.3.1.4 Results and Discussion

- **RQ1:** What are the most frequently mentioned words in university GenAI writing usage policies?

The results of the frequency analysis showed that keywords such as “assignment,” “student,” and “writing” were among the most commonly mentioned terms across AI policies at 100 universities. This emphasis reflects a focus on using AI tools to support student learning and enhance teaching content. The frequent mention of these words suggests that institutions are considering the role of AI in academic assignments and course design, indicating a strategic commitment to integrating AI within educational tasks and student interactions.

- **RQ2:** Which keywords reflect common concerns or focal points related to GenAI writing usage in academic settings?

The analysis of the top 12 frequently mentioned terms highlighted additional focal points, including “tool,” “academic,” “instructor,” “integrity,” and “expectations.” These terms reveal concerns around the ethical use of AI tools, the need for clarity in academic applications, and the central role of instructors in AI policy implementation. Keywords like “integrity” and “expectations” emphasize the importance of maintaining academic standards and setting clear guidelines for AI use in classrooms, while “instructor” underscores the influence faculty members have in shaping AI-related practices. Together, these terms reflect a commitment to transparent policies that support ethical and effective AI integration, enhancing the academic experience for students.

4.3.2 Dictionary-based Analysis

4.3.2.1 Purpose

The purpose of dictionary-based analysis in text data is to assess the presence of predefined categories, like emotions or sentiments, within the text using lexicons or dictionaries. This approach allows researchers to quantify qualitative aspects, such as positive or negative sentiment, based on specific words that correspond to these categories.

Case:

In this analysis, we examine the stance of 100 universities on the use of GenAI by applying the Bing sentiment dictionary. By analyzing sentiment scores, we aim to identify the general tone in these policies, indicating whether the institutions' attitudes toward GenAI are predominantly positive or negative.

4.3.2.2 Sample Research Questions

- **RQ:** What is the dominant sentiment expressed in GenAI policy texts across universities, and is it primarily positive or negative?

4.3.2.3 Analysis

Step 1: Install and Load Necessary Libraries

First, install and load the required packages for text processing and visualization.

```
# Install necessary packages if not already installed
# install.packages(c(
#   "tidytext", "tidyverse", "dplyr", "ggplot2", "tidyr"
# ))

# Load libraries
library(tidytext)
library(tidyverse)
```

```
library(dplyr)
library(ggplot2)
library(tidyr)
```

Step 2: Load and Prepare Data

Load the GenAI policy stance data from a CSV file. We use the same data as in Section 2.3.

```
# Load the dataset
policy_path <- "data/University_GenAI_Policy_Stance.csv"
university_policies <- read_csv(policy_path)

# Tokenize, remove stop words, and count words
word_frequency <- university_policies |>
  # Tokenize the 'Stance' column
  unnest_tokens(word, Stance) |>
  # Remove common stop words
  anti_join(stop_words) |>
  # Count and sort words by frequency
  count(word, sort = TRUE)

print(word_frequency, n = 6, width = output_width)
```

Step 3: Tokenize Text Data and Apply Sentiment Dictionary

Tokenize the policy text data to separate individual words. Then, use the Bing sentiment dictionary to label each word as positive or negative.

```
# Apply the Bing dictionary to policy words
sentiment_scores <- word_frequency |>
  # Join with Bing sentiment lexicon
  inner_join(get_sentiments("bing")) |>
  pivot_wider(
    names_from = sentiment,
    values_from = n,
    values_fill = 0
  ) |>
```

```
# Calculate net sentiment score
mutate(sentiment_score = positive - negative)

print(sentiment_scores, n = 6, width = output_width)
```

Step 4: Create a Density Plot for Sentiment Distribution

Visualize the distribution of sentiment scores with a density plot, showing the prevalence of positive and negative sentiments across university policies.

```
# Generate a density plot of sentiment scores
density_plot <- ggplot(
  sentiment_scores,
  aes(x = sentiment_score, fill = sentiment_score > 0)
) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(
    values = c("red", "green"),
    name = "Sentiment",
    labels = c("Negative", "Positive")
  ) +
  labs(
    title = "University AI Policy Sentiment",
    x = "Sentiment Score",
    y = "Density",
    caption = "Source: University Policy Text"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    axis.text = element_text(size = 14),
    axis.title = element_text(face = "bold", size = 14),
    plot.caption = element_text(size = 12)
  )

# Print the plot
print(density_plot)
```

The methods used above can be written up as follows:

Sample Methods

To assess the emotional valence of university policy statements on generative artificial intelligence, the text data were first tokenized into individual words and cleaned by removing stop words. Word frequencies were then computed across the corpus to identify the most commonly used terms in the policy stance texts. This approach follows standard tidy text practices for descriptive text mining and term-frequency analysis.

Sentiment analysis was conducted using the **Bing** sentiment lexicon implemented in the *tidytext* package. The Bing lexicon classifies words as either positive or negative, making it well suited for binary polarity assessment in policy discourse. Word-frequency data were joined with the Bing dictionary using `inner_join()`, and the sentiment categories were reshaped into a wide format so that positive and negative counts could be compared directly. A net sentiment score was then calculated as the difference between positive and negative word counts, with positive values indicating more favorable tone and negative values indicating more unfavorable tone.

This dictionary-based method provides a transparent and reproducible way to quantify sentiment in textual policy documents. It is particularly useful for exploratory analysis because it captures broad polarity patterns while remaining easy to interpret and replicate across documents.

4.3.2.4 Results and Discussion

- **RQ:** What is the dominant sentiment expressed in GenAI policy texts across universities, and is it primarily positive or negative? The dictionary-based sentiment analysis reveals the prevailing sentiments in university policies on GenAI usage. Using the Bing lexicon to assign positive and negative scores, the density plot illustrates the distribution of sentiment scores across the 100 institutions.

The results indicate a balanced perspective with a slight tendency toward positive sentiment, as reflected by a higher density of positive scores. This analysis provides insights into the varying degrees

of acceptance and caution universities adopt in their AI policy frameworks, demonstrating the diverse stances that shape institutional AI guidelines.

4.3.3 Clustering-Based Analysis

Clustering-based analysis involves grouping similar text documents or text segments into clusters based on their underlying topics or themes. This approach is particularly useful for identifying dominant themes in text data, such as university AI policy documents.

4.3.3.1 Purpose

Purpose: The goal of clustering-based analysis is to uncover latent themes in text data using unsupervised machine learning techniques. Topic modeling is one popular method for clustering documents into groups based on their content.

Case: Using the GenAI policy texts from 100 universities, we apply Latent Dirichlet Allocation (LDA) to identify dominant themes in these policy documents. This analysis will help categorize policies into overarching themes, such as academic integrity, student support, and instructor discretion.

4.3.3.2 Sample Research Questions

- **RQ1:** What are the prominent themes present in university policies regarding GenAI usage Stance?
- **RQ2:** How do these themes reflect the key concerns or opportunities for integrating GenAI in higher education?

4.3.3.3 Analysis

Step 1: Install and Load Necessary Libraries

Install and load the required libraries for text processing and topic modeling.

```
# Install necessary packages
# install.packages(c(
#   "dplyr", "tidytext", "topicmodels", "ggplot2"
# ))

# Load libraries
library(dplyr)
library(tidytext)
library(topicmodels)
library(ggplot2)
```

Step 2: Prepare the Data

Load the data and create a document-term matrix (DTM) for topic modeling.

```
# Load the dataset
policy_path <- "data/University_GenAI_Policy_Stance.csv"
university_policies <- read_csv(policy_path)

# Count word frequencies by document.
# We add a unique ID for each policy.
university_policies <- university_policies |>
  mutate(policy_id = row_number())

word_counts <- university_policies |>
  unnest_tokens(word, Stance) |>
  anti_join(stop_words) |>
  count(policy_id, word, sort = TRUE)

# Create Document-Term Matrix
gpt_dtm <- word_counts |>
  cast_dtm(
    document = policy_id,
    term = word,
    value = n
  )

gpt_dtm
```

```

# Define range of k values
k_values <- c(2, 3, 4, 5)

# Initialize a data frame to store perplexities
perplexities <- data.frame(
  k = integer(),
  perplexity = numeric()
)

# Calculate perplexity for each k
for (k in k_values) {
  # Fit LDA model
  lda_model <- LDA(
    gpt_dtm,
    k = k,
    control = list(seed = 1234)
  )
  # Calculate perplexity
  perplexity_score <- perplexity(lda_model, gpt_dtm)
  perplexities <- rbind(
    perplexities,
    data.frame(k = k, perplexity = perplexity_score)
  )
}

# Plot perplexity vs number of topics
ggplot(perplexities, aes(x = k, y = perplexity)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Perplexity vs Number of Topics",
    x = "Number of Topics (k)",
    y = "Perplexity"
  ) +
  theme_minimal()

```

Step 3: Fit the LDA Model

Fit an LDA model with $k = 3$ topics.

```
# Fit the LDA model
lda_model <- LDA(
  gpt_dtm,
  k = 3,
  control = list(seed = 1234)
)

# View model results
gpt_policy_topics_k3 <- tidy(lda_model, matrix = "beta")

print(gpt_policy_topics_k3, n = 6, width = output_width)
```

Step 4: Visualize Topics

Extract the top terms for each topic and visualize them.

```
# Visualizing top terms for each topic
gpt_policy_ap_top_terms_k3 <- gpt_policy_topics_k3 |>
  group_by(topic) |>
  slice_max(beta, n = 10) |>
  ungroup() |>
  arrange(topic, -beta)

gpt_policy_ap_top_terms_k3 |>
  mutate(term = reorder_within(term, beta, topic)) |>
  ggplot(
    aes(beta, term, fill = factor(topic))
  ) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ topic, scales = "free") +
  scale_y_reordered() +
  labs(
    title = "Top Terms per Topic (LDA)",
    subtitle = "Analysis of University GenAI Policies",
    x = "Beta (Probability of term belonging to topic)",
    y = NULL
  ) +
  theme_minimal()
```

The methods used above can be written up as follows:

Sample Methods

Data Preparation

Text data were obtained from the *University GenAI Policy Stance* dataset, which contains institutional policy statements related to the adoption and regulation of generative artificial intelligence (GenAI). Each policy document was assigned a unique identifier to facilitate document-level analysis. Text pre-processing was conducted in **R** using the *tidytext* framework (Silge & Robinson, 2016). The preprocessing steps included tokenization into unigrams, removal of stop words using the *tidytext* stop word lexicon, and aggregation of word counts by document. The resulting term frequencies were converted into a **document-term matrix (DTM)**, which served as the input for topic modeling.

Topic Modeling Procedure

Latent Dirichlet Allocation (LDA) was employed to identify latent themes within the policy texts (Blei et al., 2003), implemented via the *topicmodels* package (Grün & Hornik, 2011). To determine the optimal number of topics, a range of models with $k = 2, 3, 4$, and 5 topics was fitted, and each model's **perplexity score**—a standard measure of model fit in probabilistic topic modeling—was computed. A line plot of perplexity values against topic numbers was used to identify the point of diminishing returns. The model with three topics ($k = 3$) exhibited a balanced trade-off between interpretability and model fit and was selected for further analysis.

Model Fitting and Interpretation

An LDA model with $k = 3$ topics was fitted using a fixed random seed to ensure replicability. The resulting topic-term probabilities (β) were extracted and organized to identify the most representative words for each topic. The top ten terms per topic were visualized using *ggplot2*, allowing for intuitive comparison of thematic content across topics. This visualization facilitated qualitative interpretation of the emergent themes and enabled labeling based on the semantic coherence of high-probability terms.

Computational Environment

All analyses were conducted using **R version 4.x** within the **tidyverse** ecosystem. The following packages were utilized: *dplyr* for data manipulation, *tidytext* for tokenization and text processing, *topicmodels* for LDA modeling, and *ggplot2* for visualizations. The analysis was executed on a standard workstation environment to ensure reproducibility and computational efficiency.

4.3.3.4 Results and Discussion

Research Question 1:

What are the prominent themes present in university policies regarding GenAI usage?

Answer:

The topic modeling analysis revealed three distinct themes in the university GenAI policies:

1. **Theme 1: Student-Centric Guidelines and Ethical Considerations**
 - Key terms: `students`, `integrity`, `tools`, `instructors`, `assignment`
 - This theme emphasizes student usage of GenAI in academic settings, with a focus on ethics (`integrity`) and guidelines for instructors to manage assignments involving AI tools.
2. **Theme 2: Academic Standards and Faculty Expectations**
 - Key terms: `students`, `academic`, `faculty`, `honor`, `expectations`
 - This theme focuses on maintaining academic integrity and clarifying expectations for faculty and students regarding GenAI usage in assignments and assessments.
3. **Theme 3: Policy-Level Governance and Technology Integration**
 - Key terms: `ai`, `tools`, `policy`, `learning`, `generative`

- This theme revolves around institutional policies on AI integration, highlighting broader governance strategies and how generative AI (like GenAI) fits into learning environments.

Research Question 2:

How do these themes reflect the key concerns or opportunities for integrating GenAI in higher education?

Answer:

The identified themes reflect both concerns and opportunities:

1. Concerns:

Theme 1: Highlights the ethical challenges, such as ensuring academic integrity when students use AI tools in their coursework. Institutions are keen on setting clear guidelines for both students and instructors to avoid misuse.

Theme 2: Underlines the potential for conflict between maintaining academic standards (**honor, expectations**) and leveraging AI to support learning. This shows a cautious approach to integrating AI while upholding traditional values.

Theme 3: Raises policy-level questions on AI governance, such as whether existing institutional frameworks are adequate to regulate emerging generative AI technologies.

2. Opportunities:

Theme 1: Presents a chance to redefine how students interact with AI tools to foster responsible and innovative usage, particularly for assignments and creative tasks.

Theme 2: Encourages collaboration between faculty and administration to develop robust expectations and support systems for integrating AI in the classroom.

Theme 3: Offers a strategic opportunity for universities to lead in AI adoption by establishing comprehensive policies that guide AI's role in education and research.

Discussion:

The topic modeling results suggest that universities are navigating a complex landscape of opportunities and challenges as they incorporate GenAI into academic contexts. While student-centric policies aim to balance innovation with ethical considerations, institutional-level themes signal the need for governance frameworks to ensure responsible AI use. These findings indicate that higher education institutions are positioned to play a pivotal role in shaping the future of generative AI in learning, provided they address the ethical, pedagogical, and policy challenges identified in this analysis.

References:

Saldaña, J. (2021). Coding techniques for quantitative and mixed data. *The Routledge reviewer's guide to mixed methods analysis*, 151-160.

4.4 Summary

This chapter demonstrated three approaches to computational text analysis — frequency-based analysis, sentiment analysis, and topic modeling — each applied to GenAI usage guidelines from U.S. universities. Together, these methods illustrate how educational researchers can systematically analyze large text corpora using R. Readers wishing to deepen their understanding can explore the following resources:

- **Text Mining and Topic Modeling**

- Silge, J., & Robinson, D. (2017). *Text Mining with R: A Tidy Approach* (free online book; Chapters 2–3 on sentiment and 3–4 on topic modeling).
- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). *Latent Dirichlet Allocation* (foundational paper on LDA).

- **R-Based Text Analysis**

- CRAN documentation for `tidytext`, `topicmodels`, `dplyr`, and `ggplot2`, which together form the core toolkit used in this analysis.

- Online tutorials titled “topic modeling with tidytext and LDA” or “word frequency and word clouds in R” for step-by-step implementation.

- **Sentiment and Lexicon-Based Methods**

- Liu, B. (2012). *Sentiment Analysis and Opinion Mining* (introductory overview of dictionary-based and machine-learning approaches).
- Documentation for the Bing sentiment lexicon and VADER, which underpin the polarity-score methods used here.

- **Practice-Oriented Extensions**

- Open-source R notebooks or blog posts on “university policy text analysis,” “sentiment analysis of policy documents,” or “topic modeling of educational texts” to see closely related applied examples.

Network Data

5.1 Overview

Social network analysis (SNA) is the process of investigating social structures through the use of networks and graph theory (Borgatti et al., 2013; Wasserman & Faust, 1994). It maps and measures relationships and flows between people, groups, organizations, computers, or other information/knowledge processing entities — capturing not just individual attributes but the connections between them. In educational research, SNA has been applied to understand collaboration structures, examine the spread of information and influence, and evaluate the effects of instructional interventions on learner interactions (Carolan, 2014).

These applications range from analyzing face-to-face classroom dynamics to large-scale online learning environments. For example, Kellogg & Edelmann (2015) examined discussion networks in a massively open online course, and Rosenberg & Staudt Willet (2021) explored how social influence models can be advanced within learning analytics contexts. Used this way, SNA can identify patterns and trends in how networks form and operate, predict future behavior, and inform the design of interventions that improve learning environments.

Portions of this chapter adapt instructional and research resources developed through the NSF-supported Learning Analytics in STEM Education Research (LASER) Institute. See the Preface for the full acknowledgment and disclaimer.

5.2 Accessing SNA Data

Social Network Analysis (SNA) relies on relational data—information about **connections** (edges) between entities (nodes) such as students, teachers, or organizations. Compared to traditional survey or tabular data, SNA requires **pairwise relational information**. In education,

this could include “who collaborates with whom,” “who talks to whom,” or digital traces of discussion and collaboration in online platforms.

5.2.1 Types and Sources of SNA Data

There are several common sources and structures for SNA data in educational research contexts:

- **Survey-based Network Data:** Collected via roster or name generator questions, e.g., “List the classmates you discuss assignments with.”
- **Behavioral/Observational Data:** Derived from logs of actual interactions, e.g., forum replies, emails, classroom seating.
- **Archival or Digital Trace Data:** Extracted from digital platforms such as MOOCs, LMS discussion forums, Slack, Twitter, or Facebook.
- **Administrative/Organizational Data:** Information about formal structures such as team membership or co-authorship.

Data Structure: Most SNA data are formatted as: - **Edge List** (two columns: source and target) - **Adjacency Matrix** (rows and columns are actors; cell values indicate a tie) - **Node Attributes** (supplementary information about each node, e.g., gender, role)

5.2.2 Example 1: Creating a Simple Network from an Edge List

Below is an example of constructing a network from a simple CSV edge list. This mirrors typical classroom survey data (“who do you consider your friend in this class?”).

```
# Install and load the igraph package
# install.packages("igraph")
library(igraph)

# Example: Load an edge list from CSV
edge_list <- read.csv("data/friendship_edges.csv")

# Create the graph object (directed network)
g <- graph_from_data_frame(edge_list, directed = TRUE)
```

```
# Plot the network
plot(g, main = "Friendship Network")
```

5.2.3 Example 2: Generating Network Data from Digital Traces

Many educational datasets now come from online discussion forums, MOOCs, or LMS systems. For example, the MOOC case study (Kellogg & Edelmann, 2015) uses reply relationships in online courses to construct discussion networks.

```
# Example data with from_user and to_user columns
mooc_edges <- read.csv("data/mooc_discussion_edges.csv")
g_mooc <- graph_from_data_frame(
  mooc_edges,
  directed = TRUE
)
plot(g_mooc, main = "MOOC Discussion Network")
```

5.2.4 Example 3: Collecting SNA Data via Surveys

If you want to collect your own network data:

- Ask participants to **name or select** (from a roster) their friends, collaborators, or contacts.
- Compile responses into an edge list.
- Example survey prompt:

“Please list up to five classmates you seek help from most frequently.”

Tip

Survey-based SNA is easier to manage with small to medium groups. For larger networks, digital trace or archival data may be more practical.

5.2.5 Node Attribute Data

You can also load additional data about each node (student, teacher, etc.) to enable richer analyses (e.g., centrality by gender or role).

```
node_attributes <- read.csv("data/friendship_nodes.csv")
# Add attributes to igraph object
V(g)$gender <- node_attributes$gender[
  match(V(g)$name, node_attributes$name)
]
```

5.2.6 Further Examples

- **Public Datasets:**
 - MOOC Discussion Networks
 - Add Health
 - Common Core Twitter Networks (Supovitz et al.)
- **Synthetic Data:**
 - R's `igraph` package can also generate sample networks for practice:

```
library(igraph)
g_sample <- sample_gnp(n = 10, p = 0.3)
plot(g_sample, main = "Random Sample Network")
```

5.2.7 Best Practices and Tips

! SNA Best Practices

- **Ethics:** Social network data can be sensitive. Protect anonymity and comply with IRB/data use guidelines.
- **Format Consistency:** Always clarify whether ties are directed/undirected, binary/weighted, and ensure consistent formatting.
- **Missing Data:** Especially in survey-based SNA, missing responses can impact network structure and interpretation.

5.2.8 Summary

Accessing SNA data involves both careful **design** (in the case of surveys/observations) and **extraction/wrangling** (in the case of digital traces or archival records). The choice of data source and structure will directly influence the kinds of questions you can answer with SNA.

Recommended Reading

- Borgatti, S. P., Everett, M. G., & Johnson, J. C. (2018). *Analyzing Social Networks* (2nd ed). SAGE.
- Kellogg, S., & Edelman, A. (2015). Massive open online course discussion forums as networks.

5.3 Network Management & Measurement in Social Network Analysis

5.3.1 Purpose and Case

Purpose: This section demonstrates how to manage, measure, and visualize large-scale discussion networks from online professional development settings. Through this real-world example, we guide readers in loading relational data, constructing a directed network, and conducting a suite of essential SNA measures. The focus is on classroom- and course-level online discussions, which are representative of many contemporary educational and research settings.

Case Study: The case data comes from two cohorts of an online professional development program (“DLT1” and “DLT2”). Each cohort’s discussion data includes (a) **edge list** data capturing who replied to whom, and (b) **node/actor** attributes describing roles (e.g., facilitator, expert). These data allow us to reconstruct and analyze the full structure of communication in two authentic online learning communities.

5.3.2 Sample Research Questions

- **RQ1:** What is the overall structure of interaction in each online discussion cohort? Are they densely connected, or fragmented?

- **RQ2:** Who are the most central or influential actors in the network? How do facilitators or experts compare with regular participants?
- **RQ3:** To what extent are ties reciprocated (mutual) and how cohesive are the networks?
- **RQ4:** How do the network properties (e.g., density, reciprocity, clustering) compare between cohorts?

5.3.3 Analysis

Step 1: Install and Load Required Packages

```
# Install and load necessary libraries
# install.packages(c(
#   "tidygraph", "ggraph", "readr", "janitor"
# ))
library(tidygraph)
library(ggraph)
library(readr)
library(janitor)
library(igraph)
library(dplyr)
```

Step 2: Import and Clean Data

Load Edges and Node Attributes for DLT1:

```
# Load edge list (who replied to whom)
dlt1_ties <- read_csv("data/dlt1-edges.csv",
  col_types = cols(Sender = col_character(),
                    Receiver = col_character(),
                    `Category Text` = col_skip(),
                    `Comment ID` = col_character(),
                    `Discussion ID` = col_character())) |>
  clean_names()

# Load node attributes (participant roles, etc.)
dlt1_actors <- read_csv("data/dlt1-nodes.csv",
  col_types = cols(UID = col_character(),
```

```

        Facilitator = col_character(),
        expert = col_character(),
        connect = col_character())) |>

clean_names()

dlt1_ties |>
  select(
    sender, receiver, timestamp,
    discussion_category, comment_id
  ) |>
  slice_head(n = 4)

dlt1_actors |>
  select(
    uid, facilitator, role1,
    gender, expert, connect
  ) |>
  slice_head(n = 4)

```

Step 3: Construct and Explore the Network

```

# Build the directed network graph.
dlt1_network <- tbl_graph(
  edges = dlt1_ties,
  nodes = dlt1_actors,
  node_key = "uid",
  directed = TRUE
)

# Overview of the network
tibble::tibble(
  nodes = gorder(dlt1_network),
  edges = gsize(dlt1_network),
  components = igraph::components(
    as.igraph(dlt1_network)
  )$no
)

```

Step 4: Basic Visualization

```
# Quick overview plot (stress layout by default)
autograph(dlt1_network)

# Custom visualization with colors and centrality
ggraph(dlt1_network, layout = "fr") +
  geom_edge_link(alpha = .2) +
  geom_node_point(
    aes(color = role1, size = local_size())
  ) +
  theme_graph() +
  theme(text = element_text(family = "sans"))
```

Step 5: Network Size and Centralization

```
# Number of nodes and edges
gorder(dlt1_network)
gsize(dlt1_network)

# Degree centrality (all, in, out)
deg_all <- centr_degree(dlt1_network, mode = "all")$res
deg_in  <- centr_degree(dlt1_network, mode = "in")$res
deg_out <- centr_degree(dlt1_network, mode = "out")$res

# Centralization
centr_degree(dlt1_network, mode = "all")$centralization
centr_degree(dlt1_network, mode = "in")$centralization
centr_degree(dlt1_network, mode = "out")$centralization
```

Step 6: Attach and Visualize Node Centrality

```
# Add in-degree centrality as node attribute
dlt1_network <- dlt1_network |>
  activate(nodes) |>
  mutate(in_degree = centrality_degree(mode = "in"))
```

```
# Plot, sizing nodes by in-degree
ggraph(dlt1_network, layout = "fr") +
  geom_edge_link(alpha = .2) +
  geom_node_point(aes(size = in_degree, color = role1)) +
  theme_graph() +
  theme(text = element_text(family = "sans"))
```

Step 7: Network Density, Reciprocity, Clustering, Distance

```
# Density
edge_density(dlt1_network)

# Reciprocity
reciprocity(dlt1_network)

# Add reciprocated edge attribute and plot
dlt1_network <- dlt1_network |>
  activate(edges) |>
  mutate(reciprocated = edge_is_mutual())

ggraph(dlt1_network, layout = "fr") +
  geom_node_point(aes(size = in_degree)) +
  geom_edge_link(aes(color = reciprocated), alpha = .2) +
  theme_graph() +
  theme(text = element_text(family = "sans"))

# Clustering (transitivity/global)
transitivity(dlt1_network)

# Network diameter and average distance
diameter(dlt1_network)
mean_distance(dlt1_network)
```

Step 8: Repeat for DLT2

```
# 1. Load the DLT2 edge and node data
dlt2_ties <- read_csv("data/dlt2-edges.csv",
  col_types = cols(Sender = col_character(),
                    Receiver = col_character(),
                    `Category Text` = col_skip(),
                    `Comment ID` = col_character(),
                    `Discussion ID` = col_character())) |>
  clean_names()

dlt2_actors <- read_csv("data/dlt2-nodes.csv",
  col_types = cols(UID = col_character(),
                    Facilitator = col_character(),
                    expert = col_character(),
                    connect = col_character())) |>
  clean_names()
```

```
# 2. Construct the directed network
dlt2_network <- tbl_graph(
  edges = dlt2_ties,
  nodes = dlt2_actors,
  node_key = "uid",
  directed = TRUE
)

# 3. Basic network properties
num_nodes <- gorder(dlt2_network)
num_edges <- gsize(dlt2_network)

# 4. Degree centrality (overall, in, out)
deg_all <- centr_degree(dlt2_network, mode = "all")$res
deg_in <- centr_degree(dlt2_network, mode = "in")$res
deg_out <- centr_degree(dlt2_network, mode = "out")$res
```

```
# Centralization values
get_centralization <- function(mode) {
  centr_degree(
    dlt2_network,
    mode = mode
  )$centralization
}

centr_all <- get_centralization("all")
centr_in <- get_centralization("in")
centr_out <- get_centralization("out")

# 5. Attach centrality as a node attribute
dlt2_network <- dlt2_network |>
  activate(nodes) |>
  mutate(in_degree = centrality_degree(mode = "in"))
```

```
# 6. Visualize the network
ggraph(dlt2_network, layout = "fr") +
  geom_edge_link(alpha = .2) +
  geom_node_point(
    aes(color = role, size = local_size())
  ) +
  theme_graph() +
  theme(text = element_text(family = "sans"))
```

```
# 7. Density, reciprocity, clustering, distances
density <- edge_density(dlt2_network)
recip <- reciprocity(dlt2_network)

dlt2_network <- dlt2_network |>
  activate(edges) |>
  mutate(reciprocated = edge_is_mutual())

ggraph(dlt2_network, layout = "fr") +
  geom_node_point(aes(size = in_degree)) +
  geom_edge_link(
    aes(color = reciprocated),
    alpha = .2
  ) +
```

```
theme_graph() +
theme(text = element_text(family = "sans"))

trans  <- transitivity(dlt2_network)
diam   <- diameter(dlt2_network)
mean_d <- mean_distance(dlt2_network)

# 8. Print summary statistics (structured output)
dlt2_stats <- data.frame(
  metric = c(
    "Nodes", "Edges",
    "Degree Centralization (all)",
    "Degree Centralization (in)",
    "Degree Centralization (out)",
    "Density", "Reciprocity",
    "Transitivity", "Diameter", "Mean Distance"
  ),
  value = c(
    num_nodes, num_edges,
    centr_all, centr_in, centr_out,
    density, recip,
    trans, diam, mean_d
  )
)

print(dlt2_stats, row.names = FALSE, digits = 3)
```

A sample methods section of these analytical steps can be written as follows:

To examine interaction patterns in the DLT discussions, directed social networks were constructed from edge-list data representing reply ties between participants and node-level attribute data containing participant identifiers and role information. The data were imported from separate edge and node files, cleaned with `janitor::clean_names()`, and merged into a `tbl_graph` object using `tidygraph`, with sender-to-receiver ties treated as directed relationships. This structure is appropriate for modeling conversational exchanges in which the di-

rection of a reply indicates the flow of communication.

Network structure was first explored through basic graph visualization and descriptive network properties. Layout-based plots were generated with **ggraph**, which is designed to visualize tidy network objects and supports node-level aesthetics such as role and degree-based size scaling. Overall network size was summarized using the number of nodes and edges, and degree centrality was calculated for all ties, in-ties, and out-ties to assess participant connectedness and directional prominence. Degree centralization was also computed to evaluate the extent to which the network was organized around highly connected actors.

Additional structural measures were then calculated to characterize cohesion and reciprocity in the discussion networks. Node-level in-degree centrality was attached as a network attribute and visualized to highlight the most frequently addressed participants. Network density, reciprocity, transitivity, diameter, and mean geodesic distance were computed using standard graph-theoretic measures implemented in **igraph**. Density captured overall tie concentration, reciprocity indicated the extent to which ties were mutual, transitivity reflected clustering among connected actors, diameter summarized the longest shortest path, and mean distance represented average separation between nodes. The same procedures were repeated separately for DLT1 and DLT2 to support comparison across the two discussion networks.

5.3.4 Results and Discussion

RQ1: What is the overall structure of interaction in each cohort?

The interaction networks in both cohorts were large but sparse, indicating that participation was widespread without being uniformly connected. DLT1 included 445 nodes and 2529 directed edges, while DLT2 included 492 nodes and 2584 directed edges. Network density was low in both cases, with DLT1 at 0.013 and DLT2 at 0.011, meaning that only about 1% to 1.3% of all possible ties were present.

Both networks were multi-component, with several disconnected groups, although most participants were contained in the largest giant component.

This pattern is typical of online discussion networks, where interaction tends to cluster within subgroups rather than extend evenly across all participants. Despite this fragmentation, both cohorts had a diameter of 8 and an average shortest path length of about 3.03 for DLT1 and 3.04 for DLT2, suggesting that participants within the main component were separated by only a few steps on average.

Taken together, these results indicate that discussion and information could travel relatively quickly through the largest connected group, but engagement was selective rather than comprehensive across the full network.

RQ2: Who are the most central or influential actors?

The centrality results show a strongly right-skewed pattern in both cohorts: most participants have low degree centrality, while a small number of actors are highly connected and occupy more influential positions in the discussion network. In directed networks, in-degree reflects how often others direct ties toward an actor, while out-degree reflects how actively that actor initiates ties; together, these measures help distinguish between actors who are sought out and those who are highly active in reaching others.

In both DLT1 and DLT2, facilitators and a small set of especially active participants function as network hubs, receiving and/or initiating a disproportionate share of interactions. This pattern is consistent with the interpretation of high degree centrality as a sign of prominence, influence, or conversational activity in social networks. Degree centralization values further show that DLT1 is more centralized than DLT2, with all-ties centralization of 0.64 in DLT1 versus 0.53 in DLT2; in-degree centralization is also higher in DLT1 (1.06) than in DLT2 (0.87), while out-degree centralization is 0.23 in DLT1 and 0.33 in DLT2.

Network visualizations that size nodes by in-degree make these central actors immediately visible, highlighting the limited number of participants who anchor communication in each cohort. Substantively, these individuals—often facilitators—likely played important roles in guiding discussion, responding to contributions, and sustaining engagement among less active members.

RQ3: Are ties reciprocated?

Reciprocity was moderate in both cohorts, indicating that some interactions were mutual but most were one-way. DLT1 had a reciprocity rate of 0.20, meaning 20% of ties were reciprocated, while DLT2 had a slightly higher rate of 0.25.

This pattern suggests that the discussion networks combined broadcasting with dialogue. In other words, many replies or messages did not receive an immediate return tie, but a meaningful minority of relationships showed mutual exchange, which is consistent with a mix of one-to-many communication and peer-to-peer interaction in online learning settings. A reciprocity level in this range is often interpreted as evidence of selective engagement rather than fully symmetric conversation across the entire network.

RQ4: How cohesive are the networks?

The networks showed low transitivity, indicating limited triadic closure and relatively few closed triangles. DLT1 had a clustering coefficient of 0.089, and DLT2 had a slightly higher value of 0.125, suggesting that connected participants were only occasionally embedded in tightly knit local groups.

Substantively, this means the networks were more hub-and-spoke than cliquish: participants were often linked through central actors rather than through dense clusters of mutually connected peers. This pattern is consistent with discussion spaces in which a few highly connected individuals connect otherwise separate participants, but close-knit subgroups remain limited.

Despite the low clustering, both networks still had a diameter of 8, meaning that even the most distant participants in each network could be reached in eight steps. Their mean distances were about 3.0, which indicates that participants in the main component were relatively close to one another overall and that information could travel through the network in only a few hops.

Comparison Across Cohorts

Overall, the two cohorts exhibited very similar interaction structures. DLT2 was slightly larger, with more participants and more interactions than DLT1, but the core network properties were largely comparable

across the two groups. Both networks were sparse, centrally organized, and only moderately reciprocal, which is a common pattern in large online discussion networks where a small number of actors account for much of the activity.

Minor differences suggest modest variation in how the discussions unfolded. DLT2 showed slightly higher reciprocity and clustering, which may reflect differences in facilitation style, participant composition, or the degree of peer-to-peer engagement in that cohort. Even so, both networks followed the same broad pattern: a handful of central actors drove most interactions, ties were efficient but not dense, and genuine dialogue occurred but was not universal

Educational Implications

These findings suggest that a small number of highly active facilitators or students play an outsized role in sustaining interaction within online discussion networks. For educators and instructional designers, this means that strategies such as required peer response, rotating discussion leadership, and structured prompts for back-and-forth exchange may help distribute participation more evenly and strengthen network cohesion.

For researchers, the results show the value of examining centrality, reciprocity, and cohesion together when studying online learning communities. Identifying who occupies central positions can help reveal which participants may be supporting dialogue, which may be isolated, and where targeted interventions could improve inclusion and engagement.

Summary:

Through these SNA measures, we have shown how to reconstruct, visualize, and interpret the structure of large-scale online discussion networks. The approach enables identification of core communicators, understanding of participation patterns, and empirical comparison across cohorts or interventions. This “cookbook” can be adapted to other online learning or collaborative contexts.

i Note

This analysis is based on real-world data from online professional development courses. The methods and findings can be generalized to other educational settings where social networks play a role in learning and collaboration.

5.4 Case Study: Hashtag Common Core

5.4.1 Purpose and Case

The purpose of this case study is to demonstrate the application of social network analysis (SNA) in a real-world policy context: the heated national debate over the Common Core State Standards (CCSS) as it played out on Twitter. Drawing on the work of Supovitz, Daly, del Fresno, and Kolouch, the #COMMONCORE Project provides a vivid example of how social media-enabled networks shape educational discourse and policy.

This case focuses on:

- Identifying **key actors** (“transmitters,” “transceivers,” and “transcenders”) and measuring their influence.
- Detecting **subgroups/factions** within the conversation.
- Exploring how **sentiment** about the Common Core varies across network positions.
- Demonstrating **network wrangling, visualization, and analysis** using real tweet data.

Data Source

Data was collected from Twitter’s public API using keywords/hashtags related to the Common Core (e.g., **#commoncore**, **ccss**, **stopcommoncore**). The dataset includes user names, tweets, mentions, retweets, and relevant timestamps from a sample week. Only public tweets are included, and user privacy is respected.

5.4.2 Sample Research Questions

- **RQ1:** Who are the “transmitters,” “transceivers,” and “transcenders” in the Common Core Twitter network?
- **RQ2:** What subgroups or factions exist within the network, and how are they structured?
- **RQ3:** How does sentiment about the Common Core vary across actors and subgroups?
- **RQ4:** What other patterns of communication (e.g., centrality, clique formation, isolates) characterize this network?

5.4.3 Analysis

Step 1: Load Required Packages

```
library(tidyverse)
library(tidygraph)
library(ggraph)
library(skimr)
library(igraph)
library(tidytext)
```

Step 2: Data Import and Wrangling

```
# Import tweet data in edgelist format.
ccss_path <- "data/ccss-tweets.csv"
ccss_tweets <- read_csv(ccss_path)

# Keep sender, mentioned users, timestamp, and text.
ties_1 <- ccss_tweets |>
  relocate(
    sender = screen_name,
    target = mentions_screen_name
  ) |>
  select(sender, target, created_at, text)

# Unnest receiver to handle multiple mentions per tweet
ties_2 <- ties_1 |>
  unnest_tokens(
    input = target,
    output = receiver,
    to_lower = FALSE
  ) |>
  relocate(sender, receiver)

# Drop tweets without mentions.
ties <- ties_2 |>
  drop_na(receiver)
```

```

# Build nodelist
actors <- ties |>
  select(sender, receiver) |>
  pivot_longer(
    cols = c(sender, receiver),
    names_to = "role",
    values_to = "actors"
  ) |>
  select(actors) |>
  distinct()

# Create Network Object
ccss_network <- tbl_graph(
  edges = ties,
  nodes = actors,
  directed = TRUE
)

tibble::tibble(
  nodes = gorder(ccss_network),
  edges = gsize(ccss_network),
  components = igraph::components(
    as.igraph(ccss_network)
  )$no
)

```

Step 4: Network Structure – Components, Cliques, and Communities

- **Components** Identify weak and strong components (connected sub-groups):

```

ccss_network <- ccss_network |>
  activate(nodes) |>
  mutate(
    weak_component = group_components(type = "weak"),
    strong_component = group_components(type = "strong")
  )

# View component sizes

```

```
ccss_network |>
  as_tibble() |>
  group_by(weak_component) |>
  summarise(size = n()) |>
  arrange(desc(size)) |>
  slice_head(n = 6)
```

- **Cliques** Identify fully connected subgroups (if any):

```
ccss_cliques <- cliques(ccss_network, min = 3)
length(ccss_cliques)
head(ccss_cliques, n = 3)
```

- **Communities** Detect densely connected communities using edge betweenness:

```
ccss_network <- ccss_network |>
  morph(to_undirected) |>
  activate(nodes) |>
  mutate(sub_group = group_edge_betweenness()) |>
  unmorph()

ccss_network |>
  as_tibble() |>
  group_by(sub_group) |>
  summarise(size = n()) |>
  arrange(desc(size)) |>
  slice_head(n = 6)
```

Step 5: Egocentric Analysis – Centrality & Key Actors

```
ccss_network <- ccss_network |>
  activate(nodes) |>
  mutate(
    size = local_size(),
    in_degree = centrality_degree(mode = "in"),
    out_degree = centrality_degree(mode = "out"),
    closeness = centrality_closeness(),
```

```

    betweenness = centrality_betweenness()
  )

```

```

# Identify top actors by out_degree (transmitters),
# in_degree (transceivers), and both (transcenders)
top_transmitters <- ccss_network |>
  as_tibble() |>
  arrange(desc(out_degree)) |>
  slice_head(n = 5)

top_transceivers <- ccss_network |>
  as_tibble() |>
  arrange(desc(in_degree)) |>
  slice_head(n = 5)

```

```

# Transcenders: high in-degree and out-degree,
# using the top 10% threshold for each metric.
top_transcenders <- ccss_network |>
  as_tibble() |>
  filter(
    out_degree > quantile(out_degree, 0.9),
    in_degree > quantile(in_degree, 0.9)
  )

```

Step 6: Visualize the Network

```

ggraph(ccss_network, layout = "fr") +
  geom_node_point(
    aes(size = out_degree, color = out_degree)
  ) +
  geom_edge_link(alpha = .2) +
  theme_graph() +
  theme(text = element_text(family = "sans"))

```

Step 7: Sentiment Analysis (Optional)

```
library(vader)
vader_ccss <- vader_df(ccss_tweets$text)
mean(vader_ccss$compound)

vader_ccss_summary <- vader_ccss |>
  mutate(sentiment = case_when(
    compound >= 0.05 ~ "positive",
    compound <= -0.05 ~ "negative",
    TRUE ~ "neutral"
  )) |>
  count(sentiment)
```

A sample methods section can be written for the steps above as follows:

Methods

Twitter data related to CCSS were imported from a CSV file containing sender, mentioned user(s), timestamp, and tweet text. The data were reshaped into a directed edge list, with the tweeting account treated as the sender and mentioned accounts treated as receivers. Tweets containing multiple mentions were separated so that each mention represented an individual directed tie, and tweets without mentions were excluded. A node list of unique actors was then constructed, and the resulting sender–receiver data were used to build a directed network.

Network structure was examined by identifying weak and strong components, cliques of three or more actors, and communities using edge-betweenness clustering on an undirected version of the network. To assess actor prominence, in-degree, out-degree, closeness, and betweenness centrality were calculated at the node level. Actors with the highest out-degree were treated as key transmitters, those with the highest in-degree as key receivers, and actors high on both measures as highly central participants. The network was visualized using a force-directed layout, with node size mapped to out-degree.

An optional sentiment analysis was conducted using VADER to summarize the emotional tone of tweet text. Compound sentiment scores were computed for each tweet and classified as positive, negative, or neutral using standard thresholds.

5.4.4 Results and Discussion

RQ1: Who are the “transmitters,” “transceivers,” and “transcenders” in the Common Core Twitter network?

In the Common Core Twitter network, a small number of actors dominated communication. **Transmitters** were led by **SumayLu**, who initiated eight outgoing ties, followed by **DouglasHolt** with five, **WEquilSchool** with three, **fluttbot** with three, and **JoeWEquil** with two. These accounts were the most active in broadcasting, mentioning, or retweeting others within the network.

The most prominent **transceivers** were **WEquilSchool** and **SumayLu**, each with an in-degree of three, followed by **JoeWEquil**, **Tech4Learni...**, and **LASER_Insti...**, each with an in-degree of two. These actors received the most attention from others and likely served as focal points in the conversation.

Only two users qualified as **transcenders**, meaning they were both highly active senders and frequent recipients of ties. **WEquilSchool** had an in-degree of three and an out-degree of three, while **SumayLu** had an in-degree of three and an out-degree of eight. These bridging actors appear to have played especially important roles in connecting and sustaining the discourse.

RQ2: What subgroups or factions exist in the network?

Component analysis indicates that the Common Core Twitter network was highly fragmented. There were 14 weakly connected components, with the largest containing 14 users, and many additional components consisting of only two or three members. This pattern suggests limited overall cohesion and multiple parallel or isolated conversations rather than one fully integrated discussion space.

Clique analysis identified four fully connected subgroups, each containing three or four actors. These included one four-person clique involving nodes

3, 4, 5, and 6, along with several overlapping three-person cliques. Such structures point to small pockets of tightly knit interaction, although they were rare relative to the overall size of the network.

Community detection using edge betweenness identified 16 subgroups, broadly consistent with the component structure. The largest subgroup contained 10 members, while most of the remaining communities were much smaller. Taken together, these results suggest a network organized around several small, partially isolated clusters rather than a single cohesive conversation.

RQ3: What is the overall sentiment in the network?

VADER sentiment analysis of the tweet content produced an average compound score of 0.09, indicating a slightly positive overall sentiment. Although the broader policy context surrounding the Common Core is often contentious, the sampled tweets were, on balance, more positive than negative.

When classifying individual tweets as positive, neutral, or negative, the distribution showed a mix of all three categories, with positive tweets slightly outnumbering negative ones. This pattern suggests that, at least during this period, the Twitter conversation included advocacy, supportive comments, and constructive dialogue, rather than being dominated by criticism or negativity.

RQ4: What other patterns of communication (e.g., centrality, clique formation, isolates) characterize this network?

The network exhibits a strongly centralized “star-like” structure within its largest component. Two users, **SumayLu** and **WEquilSchool**, emerge as central hubs, with high out-degree and in-degree centrality, respectively. The majority of other users have very low degree values, often 0 or 1, indicating that they are peripheral and engaged in few interactions.

Clique analysis identified four fully connected subgroups of three or more users, including one 4-person clique and several 3-person cliques, some of which overlap. However, such tightly knit groups are relatively rare; most communication occurs outside of dense subgroups. The network contains

14 weak components, many of them very small, with several users appearing as isolates or embedded in isolated dyads and triads. These actors are disconnected from the main conversation or only weakly integrated.

Overall, communication in this network is characterized by a small set of highly central users, sparse and fragmented connectivity, and a substantial number of isolates, which together suggest limited network-wide cohesion and a concentration of interaction around a few key accounts.

Discussion

This analysis of the Common Core Twitter conversation reveals a sparse and fragmented network, with debate distributed across many small subgroups and only one moderately sized connected component. Communication is anchored by a small number of key actors who act as broadcasters and focal points, while the majority of users occupy peripheral positions with minimal interaction.

Tightly connected cliques and communities are few and small, reinforcing the absence of broad network-wide cohesion. At the same time, VADER sentiment scores indicate a slightly positive overall tone, suggesting that the sampled conversation includes advocacy, promotional messaging, and constructive dialogue more than unrelenting criticism.

For researchers and practitioners, this pattern highlights the importance of intentionally engaging transcendent actors—those who both send and receive many ties—as potential bridges across subgroups. The network's fragmentation also implies that broad influence is unlikely to arise from a single central core; instead, effective outreach may require engaging multiple small clusters independently. Combining social network analysis with sentiment analysis provides a richer understanding of the conversation, revealing not only who is communicating but also how the debate is structured and how it feels to participants.

References

- Supovitz, J., Daly, A.J., del Fresno, M., & Kolouch, C. (2017). *#commoncore Project*. Retrieved from <http://www.hashtagcommoncore.com>
- Carolan, B.V. (2014). *Social Network Analysis and Education: Theory, Methods & Applications*. Sage.

- Silge, J., & Robinson, D. (2017). *Text Mining with R: A Tidy Approach*. O'Reilly.

5.5 Summary

This chapter introduced social network analysis as a method for studying relational structures in educational settings. Through two case studies — a MOOC discussion network and a Twitter hashtag network — we demonstrated how to construct, measure, visualize, and interpret networks using R. For readers interested in exploring these methods further, the following resources are recommended:

- **Social Network Analysis Fundamentals**
 - Hanneman, R. A., & Riddle, M. (2005). *Introduction to Social Network Methods* (online textbook).
 - Wasserman, S., & Faust, K. (1994). *Social Network Analysis: Methods and Applications* (Chapters 1–8).
- **Network Analysis in R**
 - “Tidy Network Analysis in R” (Schoch, 2024) – a free, R-centric guide to `igraph`, `tidygraph`, and `ggraph`.
 - `tidygraph` and `ggraph` package documentation (CRAN and tidyverse-style vignettes).
- **Text Mining and Sentiment in R**
 - Silge, J., & Robinson, D. (2017). *Text Mining with R: A Tidy Approach* (free online book).
 - VADER documentation and the `vader` / `vaderR` package guides for sentiment analysis of short text.
- **Online Learning and Discussion Networks**
 - Articles on *network analysis of online discussions* (e.g., studies of MOOC forum networks or course-level discussion networks) that connect SNA metrics to pedagogical questions.
- **Practical R Notebooks / Tutorials**

- Online tutorials or GitHub repos titled “Network analysis in R”, “Twitter network analysis”, or “sentiment analysis with tidytext + VADER” to replicate and extend the exact workflow used here.

Numeric Data

6.1 Overview

In educational research, data has traditionally been sourced from small-scale surveys, experiments, or qualitative studies. However, the rise of large-scale digital data offers researchers opportunities to explore numeric and quantitative datasets of unprecedented scale and variety (Baker & Inventado, 2014; Romero & Ventura, 2010, 2020). This chapter discusses how to access and analyze two prominent examples: international assessments (e.g., PISA, TIMSS, NAEP) and digital log-trace data such as the Open University Learning Analytics Dataset, or OULAD (Kuzilek et al., 2017). Studies such as Teig et al. (2022) illustrate how large-scale assessment data can be used to investigate substantive educational questions at scale. These secondary data sources enable novel research questions and methods, particularly when paired with machine learning and statistical modeling approaches (James et al., 2021).

Portions of this chapter adapt instructional and research resources developed through the NSF-supported Learning Analytics in STEM Education Research (LASER) Institute. See the Preface for the full acknowledgment and disclaimer.

6.2 Accessing Big Data (Broadening the Horizon)

6.2.1 Big Data

Accessing PISA Data

The Programme for International Student Assessment (PISA) is a widely used dataset for large-scale educational research. It assesses 15-year-old students' knowledge and skills in reading, mathematics, and science across multiple countries. Researchers can access PISA data through various methods:

1. Direct Download from the Official Website

The OECD provides direct access to PISA data files via its official website. Researchers can download data for specific years and cycles. Data files are typically provided in `.csv` or `.sav` (SPSS) formats, along with detailed documentation.

- **Steps to Access PISA Data from the OECD Website:**

1. Visit the OECD PISA website.
2. Navigate to the “Data” section.
3. Select the desired assessment year (e.g., 2022).
4. Download the data and accompanying codebooks.

2. Using the OECD R Package

The OECD R package provides a direct interface to download and explore datasets published by the OECD, including PISA.

- **Steps to Use the OECD Package:**

1. Install and load the OECD package.
2. Use the `get_datasets()` and `get_dataset()` functions (note: the API may vary by package version).

```
# Install and load the OECD package
# install.packages("OECD")
library(OECD)

# Fetch PISA data for the 2022 cycle (Example)
pisa_data <- get_dataset("PISA_2022")

# Display a summary of the data
summary(pisa_data)
```

3. Using the EdSurvey R Package

The EdSurvey package is designed specifically for analyzing large-scale assessment data, including PISA. It allows for complex statistical modeling and supports handling weights and replicate weights used in PISA.

- **Steps to Use the EdSurvey Package:**

1. Install and load the `EdSurvey` package.
2. Download the PISA data from the OECD website and provide the path to the `.sav` files.
3. Load the data into R using `readPISA()`.

```
# Install and load the EdSurvey package
# install.packages("EdSurvey")
library(EdSurvey)

# Read PISA data from a local file
pisa_data <- readPISA("path/to/PISA2022Student.sav")

# Display the structure of the dataset
str(pisa_data)
```

Comparison of Methods

Table 6.1. Comparison of methods for accessing PISA data.

| Method | Advantages | Disadvantages |
|-------------------------------------|--|--|
| Direct Download OECD Package | Full access to all raw data and documentation.
Easy to use for downloading specific datasets. | Requires manual processing and cleaning.
Limited to OECD-published formats. |
| EdSurvey Package | Supports advanced statistical analysis and weights. | Requires additional setup and dependencies. |

Accessing IPEDS Data

The Integrated Postsecondary Education Data System (IPEDS) is a comprehensive source of data on U.S. colleges, universities, and technical and vocational institutions. It provides data on enrollments, completions, graduation rates, faculty, finances, and more. Researchers and policymakers widely use IPEDS data to analyze trends in higher education.

There are several ways to access IPEDS data, depending on the user’s needs and technical proficiency.

1. Direct Download from the NCES Website

The most straightforward way to access IPEDS data is by downloading it directly from the National Center for Education Statistics (NCES) website.

- **Steps to Access IPEDS Data:**

1. Visit the IPEDS Data Center.
2. Click on “Use the Data” and navigate to the “Download IPEDS Data Files” section.
3. Select the desired data year and survey component (e.g., Fall Enrollment, Graduation Rates).
4. Download the data files, typically provided in `.csv` or `.xls` format, along with accompanying codebooks.

2. Using the `ipeds` R Package

The `ipeds` R package simplifies downloading and analyzing IPEDS data directly from R by connecting to the NCES data repository.

- **Steps to Use the `ipeds` Package:**

1. Install and load the `ipeds` package.
2. Use the `download_ipeds()` function to fetch data for specific survey components and years.

```
# Install and load the ipeds package
# install.packages("ipeds")
library(ipeds)

# Download IPEDS data for completions in 2021
ipeds_data <- download_ipeds("C", year = 2021)

# View the structure of the downloaded data
str(ipeds_data)
```

3. Using the `tidycensus` R Package

The `tidycensus` package, while primarily designed for Census data, can access specific IPEDS data linked to educational institutions.

- **Steps to Use the tidycensus Package:**

1. Install and load the `tidycensus` package.
2. Set up a Census API key to access the data.
3. Query IPEDS data for specific institution-level information.

```
# Install and load the tidycensus package
# install.packages("tidycensus")
library(tidycensus)

# Set Census API key (replace with your actual key)
census_api_key("your_census_api_key")

# Fetch IPEDS-related data (e.g., institution information)
ipeds_institutions <- get_acs(
  geography = "place",
  variables = "B14002_003",
  year = 2021,
  survey = "acs5"
)

# View the first few rows
head(ipeds_institutions)
```

4. Using Online Tools

IPEDS provides several online tools for querying and visualizing data without requiring programming skills.

- **Common Tools:**

- **IPEDS Data Explorer:** Enables users to query and export customized datasets.
- **Trend Generator:** Allows users to visualize trends in key metrics over time.
- **IPEDS Use the Data:** Simplified tool for accessing pre-compiled datasets.

- **Steps to Use the IPEDS Data Explorer:**

1. Visit the IPEDS Data Explorer.
2. Select variables of interest, such as institution type, enrollment size, or location.

- 3. Filter results by years, institution categories, or other criteria.
- 4. Export the results as a `.csv` or `.xlsx` file.

Comparison of Methods

Table 6.2. Comparison of methods for accessing IPEDS data.

| Method | Advantages | Disadvantages |
|---------------------------|--|--|
| Direct Download | Full access to raw data and documentation. | Requires manual data preparation and cleaning. |
| ipeds Package | Automated access to specific components. | Limited flexibility for customized queries. |
| tidycensus Package | Allows integration with Census and ACS data. | Requires API setup and advanced R skills. |
| Online Tools | User-friendly and suitable for non-coders. | Limited to predefined queries and exports. |

Accessing Open University Learning Analytics Dataset (OULAD)

The **Open University Learning Analytics Dataset (OULAD)** is a publicly available dataset designed to support research in educational data mining and learning analytics. It includes student demographics, module information, interactions with the virtual learning environment (VLE), and assessment scores.

- **Steps to Access OULAD Data:**
 - 1. Visit the OULAD Repository.
 - 2. Download the dataset as a `.zip` file.
 - 3. Extract the `.zip` file to a local directory.

The dataset contains multiple CSV files:

- `studentInfo.csv`: Student demographics and performance data.
- `studentVle.csv`: Interactions with the VLE.
- `vle.csv`: Details of learning resources.
- `studentAssessment.csv`: Assessment scores.

Loading OULAD Data in R

Once the data is downloaded and extracted, follow these steps to load and access it in R:

```
# Install necessary packages
# install.packages(c("readr", "dplyr"))
library(readr)
library(dplyr)

# Define the path to the OULAD data
data_path <- "path/to/OULAD/"

# Load individual CSV files
student_info <- read_csv(file.path(
  data_path,
  "studentInfo.csv"
))
student_vle <- read_csv(file.path(
  data_path,
  "studentVle.csv"
))
vle <- read_csv(file.path(
  data_path,
  "vle.csv"
))
student_assessment <- read_csv(file.path(
  data_path,
  "studentAssessment.csv"
))

# Inspect the structure and contents
head(student_info)
str(student_vle)
```

6.3 Logistic Regression & Machine Learning

6.3.1 Purpose and Case

Purpose

Logistic regression is a supervised learning technique widely used for binary classification tasks. It models the probability of an event occurring (e.g., success vs. failure) based on a set of predictor variables. Logistic regression is particularly effective in educational research for predicting outcomes such as retention, enrollment, or graduation rates.

Case Study: Predicting Graduation Rates

This case study is based on IPEDS data and inspired by Zong and Davis (2022). We predict graduation rates as a binary outcome (`good_grad_rate`) using institutional features such as total enrollment, admission rate, tuition fees, and average instructional staff salary.

6.3.2 Sample Research Questions

- **RQ A:** What institutional factors are associated with high graduation rates in U.S. four-year universities?
- **RQ B:** How accurately can we predict high graduation rates using institutional features with supervised machine learning?

6.3.3 Analysis

Step 1: Load Required Packages

We load necessary R packages for data wrangling, cleaning, and modeling.

```
# Data wrangling, modeling, and cleaning
library(tidyverse)
library(recipes)
library(parsnip)
library(workflows)
library(yardstick)
library(janitor)
```

Step 2: Load and Clean Data

We read the IPEDS dataset and clean column names for easier handling.

```
# Read in IPEDS data from CSV file
ipeds <- read_csv("data/ipeds-all-title-9-2022-data.csv")

# Clean column names for consistency and usability
ipeds <- ipeds |>
  clean_names()
```

Step 3: Data Wrangling

Select relevant variables, filter the dataset, and create the dependent variable `good_grad_rate`.

```
# Select and rename key variables.
ipeds <- ipeds |>
  select(
    name = institution_name,
    total_enroll = drvef2022_total_enrollment,
    pct_admitted = drvadm2022_percent_admitted_total,
    tuition_fees = drvic2022_tuition_and_fees_2021_22,
    grad_rate = drvgr2022_graduation_rate_total_cohort,
    # Financial aid
    percent_fin_aid = starts_with("sfa2122_percent"),
    # Staff salary
    avg_salary = starts_with("drvhr2022_average_salary")
  ) |>
  # Keep records with graduation rates.
```

```
filter(!is.na(grad_rate))
```

```
ipeds <- ipeds |>
  mutate(
    # Create a binary outcome using a threshold.
    good_grad_rate = if_else(grad_rate > 62, 1, 0),
    # Convert the outcome for classification.
    good_grad_rate = as.factor(good_grad_rate)
  )
```

Step 4: Exploratory Data Analysis (EDA)

Visualize the distribution of the graduation rate.

```
# Plot a histogram of graduation rates
ipeds |>
  ggplot(aes(x = grad_rate)) +
  geom_histogram(
    bins = 20,
    fill = "blue",
    color = "white"
  ) +
  theme_minimal() +
  labs(
    title = "Distribution of Graduation Rates",
    x = "Graduation Rate",
    y = "Frequency"
  )
```

Step 5: Logistic Regression Model

Fit a logistic regression model to predict high graduation rates.

```
# Fit logistic regression model
m1 <- glm(
  good_grad_rate ~
    total_enroll +
```

```

    pct_admitted +
    tuition_fees +
    percent_fin_aid +
    avg_salary,
  data = ipeds,
  # Use logistic regression for the binary outcome.
  family = "binomial"
)

# View compact model summaries
broom::tidy(m1) |>
  select(term, estimate, std.error, p.value) |>
  print(n = 6, width = output_width)

broom::glance(m1) |>
  select(nobs, AIC, deviance)

```

Step 6: Supervised Machine Learning Workflow

Use the `tidymodels` framework to build a machine learning model.

```

# Define recipe for the model (preprocessing steps)
my_rec <- recipe(
  good_grad_rate ~
    total_enroll +
    pct_admitted +
    tuition_fees +
    percent_fin_aid +
    avg_salary,
  data = ipeds
)

```

```

# Specify logistic regression model with tidymodels
my_mod <- logistic_reg() |>
  set_engine("glm") |>
  set_mode("classification")

# Create workflow to connect recipe and model

```

```
my_wf <- workflow() |>
  add_recipe(my_rec) |>
  add_model(my_mod)

# Fit the logistic regression model
fit_model <- fit(my_wf, ipeds)

# Generate predictions on the dataset
predictions <- predict(
  fit_model,
  ipeds
) |>
  bind_cols(ipeds)

# Calculate and display accuracy
my_accuracy <- predictions |>
  metrics(
    truth = good_grad_rate,
    estimate = .pred_class
  ) |>
  filter(.metric == "accuracy")

my_accuracy
```

A sample methods section based on these analysis can be written as follows:

Measures

Outcome variable. Graduation performance was operationalized as a binary variable (*good_grad_rate*), coded 1 if an institution's overall graduation rate exceeded 62% and 0 otherwise. The threshold corresponds to the approximate median of the sample distribution and follows the operationalization used by Zong and Davis (2022).

Predictor variables. Five institutional characteristics were included: (1) total student enrollment, (2) percent of applicants admitted, (3) tuition and fees for the 2021–22 academic

year, (4) percent of full-time, first-time undergraduates receiving any form of financial aid, and (5) average instructional staff salary equated to nine months of full-time employment. Variable names and definitions follow the IPEDS data dictionary (NCES, 2022).

Analytical Approach

Two complementary analytic strategies addressed the study's research questions. To examine which institutional factors are associated with high graduation rates (RQ A), we estimated a binary logistic regression model using the `glm()` function in R with a binomial link function. This approach yields coefficient estimates and significance tests for each predictor, supporting interpretation of the direction and magnitude of associations (Hosmer et al., 2013).

To evaluate the predictive accuracy of the same institutional features (RQ B), we fit an equivalent logistic regression model within a supervised machine learning workflow using the `tidymodels` framework (Kuhn & Wickham, 2020). This pipeline integrated preprocessing via the `recipes` package and model specification via `parsnip`, producing a unified, reproducible workflow consistent with current best practices in applied machine learning (Silge & Kuhn, 2022). Model performance was evaluated using overall classification accuracy on the full dataset. All analyses were conducted in R (version 4.3.x).

6.3.4 Results and Discussion

Logistic Regression Model (RQ A)

The logistic regression model predicted whether an institution achieved a high graduation rate (graduation rate > 62%) from five institutional features. All predictors were statistically significant.

Total enrollment was positively associated with high graduation rates ($b = 3.35e-05$, $z = 4.25$, $p < .001$), indicating that larger institutions were more likely to exceed the graduation rate threshold. Admission selectivity (percent admitted) was negatively associated with graduation outcomes (b

= -0.014, $z = -4.00$, $p < .001$), consistent with prior evidence that more selective institutions tend to enroll students with stronger prior academic preparation (Zong & Davis, 2022). Tuition and fees was the strongest predictor in the model ($b = 6.95\text{e-}05$, $z = 14.00$, $p < .001$); institutions charging higher tuition were substantially more likely to achieve high graduation rates, a pattern that likely reflects broader institutional resource capacity rather than tuition level per se. The proportion of students receiving financial aid was negatively associated with graduation rates ($b = -0.030$, $z = -5.24$, $p < .001$), consistent with research documenting that institutions serving higher proportions of financially vulnerable students face structural barriers to degree completion (Perna, 2010). Average instructional staff salary was positively associated with high graduation rates ($b = 3.00\text{e-}05$, $z = 7.74$, $p < .001$), suggesting that faculty compensation, as a proxy for instructional quality and institutional investment, contributes to student success outcomes.

The full model showed substantial improvement over the null model (null deviance = 2277, $df = 1706$; residual deviance = 1632, $df = 1701$; AIC = 1644), indicating that the institutional features collectively accounted for meaningful variance in graduation performance.

Supervised ML Workflow Results (RQ B)

The logistic regression model fitted within the `tidymodels` supervised machine learning workflow achieved an overall classification accuracy of 80.02%. Based on five institutional predictors, the model correctly classified four out of five institutions as high- or low-performing on graduation rates, indicating that these features carry consistent and meaningful predictive signal with respect to graduation outcomes.

Overall Discussion

- **Similarities between Approaches:**
 - Both the traditional logistic regression and the `tidymodels` workflow identified key predictors that influence graduation rates, such as total enrollment, admission percentage, tuition fees, financial aid percentage, and average staff salary.
 - Each approach provides valuable insights: the regression model offers detailed coefficient estimates and significance levels, while the `tidymodels` workflow emphasizes predictive accuracy.

- **Differences between Approaches:**
 - **Interpretability vs. Predictive Performance:** The logistic regression output delivers interpretability through its coefficients and p-values, allowing us to understand the direction and magnitude of the relationships. In contrast, the supervised ML workflow focuses on achieving a robust predictive performance, evidenced by an 80% accuracy.
 - **Handling of Data:** The traditional regression model summarizes the relationship between variables, whereas the ML workflow integrates data pre-processing, modeling, and validation into a cohesive framework.

In summary, our analyses indicate that institutional factors, particularly tuition fees and staff salaries, play a significant role in predicting graduation outcomes. The supervised ML approach, with an accuracy of around 80%, confirms the model's practical utility in classifying institutions based on graduation performance. Both methods complement each other, providing a comprehensive understanding of the underlying dynamics that drive graduation rates in higher education.

6.4 Random Forests & Learning Analytics

In this section, we explore a more sophisticated supervised learning approach—Random Forests—to model student interaction data from the Open University Learning Analytics Dataset (OULAD).

6.4.1 Purpose and Case

Purpose

Random Forests is an ensemble learning method that builds multiple decision trees and aggregates their results to improve prediction accuracy and control overfitting. It is particularly well-suited for complex, high-dimensional data such as student interaction (clickstream) data.

Case Study: Predicting Student Success with VLE Data

Inspired by research on digital trace data, this case study uses interactions data from OULAD. We focus on predicting whether a student will pass a course based on engineered features derived from clickstream data, such as total clicks, mean clicks, and linear trends over time.

6.4.2 Sample Research Questions

- **RQ1:** How accurately can a random forest model predict whether a student will pass a course using interaction data from OULAD?
- **RQ2:** Which interaction-based features (e.g., total clicks, click-stream slope) are most important in predicting student outcomes?
- **RQ3:** How does the use of cross-validation influence the stability and generalizability of the model?

6.4.3 Analysis

Step 1: Load Required Packages

```
# Data wrangling, modeling, and plots
library(tidyverse)
library(janitor)
library(recipes)
library(parsnip)
library(workflows)
library(yardstick)
library(rsample)
library(tune)
library(ranger)
```

Step 2: Load and Prepare Data

We load and join multiple OULAD files to create a complete modeling dataset.

```
# Load interactions and student performance data.
interactions_path <- file.path(
  "data",
  "oulad-interactions-filtered.csv"
)
interactions <- read_csv(interactions_path)
students_path <- file.path(
  "data",
  "oulad-students-and-assessments.csv"
)
students_and_assessments <- read_csv(students_path)
assessments <- read_csv("data/oulad-assessments.csv")
```

```
# Find the first quartile of submission dates.
code_module_dates <- assessments |>
  group_by(code_module, code_presentation) |>
  summarize(
    quantile_cutoff_date = quantile(
      date_submitted,
      probs = 0.25,
      na.rm = TRUE
    ),
    .groups = "drop"
  )
```

```
# Keep interactions before the cutoff.
interactions_filtered <- interactions |>
  left_join(
    code_module_dates,
    by = c("code_module", "code_presentation"),
    suffix = c(".x", "")
  ) |>
  filter(date < quantile_cutoff_date) |>
  select(-any_of("quantile_cutoff_date.x"))
```

```
# Summarize click activity.
interactions_summarized <- interactions_filtered |>
  group_by(
```

```

    id_student,
    code_module,
    code_presentation
  ) |>
  summarize(
    sum_clicks = sum(sum_click, na.rm = TRUE),
    sd_clicks = sd(sum_click, na.rm = TRUE),
    mean_clicks = mean(sum_click, na.rm = TRUE),
    .groups = "drop"
  )

```

```

# Estimate clickstream trends.
fit_slope <- function(df) {
  if (nrow(df) < 2) {
    return(tibble(
      term = c("(Intercept)", "date"),
      estimate = c(NA, NA)
    ))
  }
  model <- lm(sum_click ~ date, data = df)
  broom::tidy(model) |>
    select(term, estimate)
}

```

```

interactions_slopes <- interactions_filtered |>
  group_by(
    id_student,
    code_module,
    code_presentation
  ) |>
  nest() |>
  mutate(model = map(data, fit_slope)) |>
  select(-data) |>
  unnest(model) |>
  pivot_wider(
    names_from = term,
    values_from = estimate
  ) |>
  rename(

```

```

    intercept = `(Intercept)`,
    slope = date
  ) |>
  mutate(across(
    where(is.numeric),
    \(x) round(x, 4)
  ))

```

```

# Join all features
join_keys <- c(
  "id_student",
  "code_module",
  "code_presentation"
)
students_full <- students_and_assessments |>
  left_join(
    interactions_summarized,
    by = join_keys
  ) |>
  left_join(
    interactions_slopes,
    by = join_keys
  ) |>
  mutate(pass = as.factor(pass))

```

Step 3: Define Model Recipe

```

my_rec2 <- recipe(
  pass ~
    disability +
    date_registration +
    gender +
    code_module +
    mean_weighted_score +
    sum_clicks +
    sd_clicks +
    mean_clicks +
    intercept +

```

```
      slope,  
      data = students_full  
    ) |>  
    step_dummy(all_nominal_predictors()) |>  
    step_impute_knn(all_predictors()) |>  
    step_normalize(all_numeric_predictors())
```

Step 4: Specify Model and Workflow

```
# Use impurity-based variable importance.  
my_mod2 <- rand_forest() |>  
  set_engine(  
    "ranger",  
    importance = "impurity"  
  ) |>  
  set_mode("classification")  
  
# Create workflow  
my_wf2 <- workflow() |>  
  add_recipe(my_rec2) |>  
  add_model(my_mod2)
```

Step 5: Resampling and Model Fitting

```
set.seed(20230712)  
vfcv <- vfold_cv(  
  data = students_full,  
  v = 4,  
  strata = pass  
)  
class_metrics <- metric_set(  
  accuracy,  
  sensitivity,  
  specificity,  
  ppv,  
  npv,  
  kap
```

```

)

# Fit using cross-validation
fitted_resamples <- fit_resamples(
  my_wf2,
  resamples = vfcv,
  metrics = class_metrics
)

```

```

collect_metrics(fitted_resamples)

```

Step 6: Final Evaluation

```

set.seed(20230712)
train_test_split <- initial_split(
  students_full,
  prop = 0.67,
  strata = pass
)

# Final fit on training, evaluate on test
final_fit <- last_fit(
  my_wf2,
  train_test_split,
  metrics = class_metrics
)
collect_metrics(final_fit)

```

```
# Variable Importance Plot
importance_data <- final_fit |>
  extract_workflow() |>
  extract_fit_parsnip() |>
  purrr::pluck("fit", "variable.importance") |>
  tibble::enframe(
    name = "feature",
    value = "importance"
  ) |>
  slice_max(importance, n = 10) |>
  mutate(feature = forcats::fct_reorder(
    feature,
    importance
  ))

ggplot(
  importance_data,
  aes(x = importance, y = feature)
) +
  geom_col() +
  theme_minimal() +
  labs(
    title = "Random Forest Feature Importance",
    x = "Importance",
    y = NULL
  )
```

A sample methods section based on these steps can be written as follows:

Methods

Data Source

Data were drawn from the Open University Learning Analytics Dataset (OULAD; Kuzilek et al., 2017), a publicly available dataset containing demographic information, virtual learning environment (VLE) interaction logs, and assessment records for students enrolled in Open University courses across multiple module presentations. OULAD has been widely used in learning analytics research to model student engagement and

predict academic outcomes (e.g., Hlosta et al., 2017; Mubarak et al., 2020).

Outcome Variable

The outcome variable was a binary indicator of course result, coded as pass (1) or non-pass (0), derived from the student registration and outcome data provided in OULAD.

Feature Engineering

To simulate an early-warning scenario, VLE interaction data were restricted to the period prior to the 25th percentile of assessment submission dates within each module-presentation combination. From this temporally constrained interaction window, three summary features were computed for each student: total clicks (`sum_clicks`), mean clicks per session (`mean_clicks`), and standard deviation of clicks (`sd_clicks`). In addition, a linear regression was fitted to each student's click sequence over time, yielding an intercept and slope that capture the baseline level and temporal trend of engagement, respectively. These engineered features were joined with student demographic variables (disability status, gender, registration date, and course module) and a measure of prior academic performance (mean weighted assessment score).

Preprocessing

All nominal predictors were dummy-coded. Missing values were imputed using k-nearest neighbors (KNN) imputation. All numeric predictors were normalized to a mean of zero and standard deviation of one. Preprocessing was implemented via the `recipes` package within the `tidymodels` framework (Kuhn & Wickham, 2020).

Model Specification and Evaluation

A random forest classifier was fit using the `ranger` engine with impurity-based variable importance (Wright & Ziegler, 2017). Model performance was assessed using two complementary evaluation strategies. First, 4-fold cross-validation stratified by the outcome variable was used to estimate out-of-sample performance across multiple data partitions. Second, a held-out test set (33% of the sample, stratified by outcome) was used for

final model evaluation. Performance was quantified using accuracy, sensitivity, specificity, positive predictive value (PPV), negative predictive value (NPV), and Cohen's Kappa. Variable importance was extracted from the final fitted model and visualized for the ten most influential predictors.

6.4.4 Results and Discussion

Research Question 1 (RQ1):

How accurately can a random forest model predict whether a student will pass a course using interactions data from OULAD?

Response:

Using 4-fold cross-validation, our random forest model yielded an average accuracy of approximately **67.4%** (mean accuracy from resamples: 0.674) with a Cohen's Kappa of **0.270**, suggesting moderate agreement beyond chance. When fitted on the entire training set and evaluated on the test set, the final model showed an accuracy of **66.4%**, along with:

- **Sensitivity:** 82.1%, indicating that the model correctly identifies a high proportion of students who pass.
- **Specificity:** 40.9%, suggesting that the model is less effective at correctly identifying students who do not pass.
- **Positive Predictive Value (PPV):** 69.4%.
- **Negative Predictive Value (NPV):** 58.2%.

The confusion matrix showed the same pattern: most passing students were identified correctly, while non-passing students were more difficult to classify.

Overall, these metrics indicate that while the model performs well in detecting positive outcomes (high sensitivity), its lower specificity means that it tends to misclassify a relatively higher proportion of non-passing students. In plain English, our model is an **optimist**—it is great at spotting the winners, but it sometimes overlooks the stragglers. This is a crucial insight for educators: this tool is better used as an **early warning system for success** rather than a perfect detector of failure.

Research Question 2 (RQ2):

Which interaction-based features are most important in predicting student outcomes?

Response:

The variable importance analysis, extracted from the final random forest model using the `vip()` function, highlights the following key predictors (with their respective importance scores):

- **sum_clicks:** 1062.49 – This is the most influential feature, indicating that the total number of clicks (i.e., student engagement) in the VLE is a strong predictor of student success. In other words, “showing up” digitally matters just as much as showing up in person.
- **mean_weighted_score:** 838.06 – Reflecting academic performance as measured by weighted assessment scores. Past performance is usually a strong predictor of future success, and this confirms it.
- **mean_clicks:** 737.79, **slope:** 739.86, and **intercept:** 729.52 – These engineered features representing the central tendency and trend of click behavior further underline the importance of digital engagement patterns.
- **date_registration:** 652.15 – The registration date also plays a significant role.
- Other categorical variables (e.g., dummy-coded **disability**, **gender**, and **code_module** levels) generally show lower importance scores, with values typically under 80, indicating that while they do contribute, engagement and performance metrics dominate.

These results suggest that both the **intensity** and the **temporal trend** of student interactions with the learning environment are critical in predicting whether a student will pass. Specifically, the **slope** (trend) is a fascinating finding—it tells us that it is not just *how much* a student clicks, but *how their behavior changes* over time. A student whose engagement is increasing—even if they started slow—is often in a better position than one whose activity is tapering off.

Research Question 3 (RQ3):

How does the use of cross-validation impact the stability and generalizability of the random forest model on interactions data?

Response:

The use of 4-fold cross-validation (via `vfold_cv`) allowed us to assess the model's performance across multiple subsets of the data, mitigating the risk of overfitting. The resampling results are relatively consistent (with accuracy around 67.4%, sensitivity at 81.8%, and specificity around 43.8%), which supports the model's robustness and generalizability. Although the final test set performance (accuracy of 66.4%) is slightly lower, the overall consistency of metrics across folds indicates that our model is stable when applied to unseen data. This consistency is key—it suggests that the patterns we have discovered are not just a fluke; they likely hold true for other student populations beyond this dataset.

Overall Discussion

The random forest model built on interactions data from OULAD demonstrates decent predictive performance with an accuracy of approximately 66.4–67.4% and high sensitivity (around 82.1%), indicating strong capability in identifying students who will pass the course. However, the relatively low specificity (around 40.9%) suggests that there is room for improvement in correctly classifying students who are at risk of not passing.

The variable importance analysis underscores that engagement-related features—especially **sum_clicks** and features capturing the trend in interactions (slope, `mean_clicks`)—are the most influential predictors. This insight implies that the **digital footprint** of student engagement in the virtual learning environment is critical for predicting academic outcomes. Every click tells a story; ignoring them means missing valuable signals.

In summary, while our model performs robustly across cross-validation folds and provides actionable insights into key predictive features, the lower specificity points to the need for further refinement. Future work might explore additional feature engineering, alternative model tuning, or combining models to better balance sensitivity and specificity, ultimately supporting timely interventions in educational settings.

6.5 Summary

This chapter introduced machine learning as a tool for educational research through two case studies using the Open University Learning Analytics

Dataset. Logistic regression provided an interpretable baseline for predicting student outcomes, while random forests captured more complex interaction patterns at the cost of interpretability. Both approaches demonstrated that student engagement metrics — particularly click activity and its trajectory over time — are strong predictors of course success. These methods extend the analytical toolkit introduced in earlier chapters by moving from descriptive and relational analyses toward predictive modeling.

LLM Methods

Overview

As educational researchers integrate large language models into data analysis workflows, our methodological responsibilities evolve in important ways (Kasneci et al., 2023). There are three primary approaches to this integration:

- first, using LLMs for code autocompletion and inline coding assistance, such as through integrated development environments like Positron;
- second, using LLMs to support the analysis process through complete code generation and supervised execution, such as with tools like Posit Databot; and
- third, deploying LLMs to directly analyze qualitative (e.g., text data).

The third approach in particular raises questions about research integrity — including correctness, transparency, and reproducibility — that are worth examining carefully before treating LLM outputs as research evidence (Bender et al., 2021).

The Responsible Use Framework

The responsible use framework for LLMs in research, presented by Joe Cheng at R+AI 2025 (Cheng, 2025), can be used as a practical evaluation tool centering on three essential criteria. **Correctness** refers to whether the LLM produces accurate and reliable results that can be verified and trusted. **Transparency** addresses whether the process and reasoning behind the AI's outputs are visible and understandable to researchers, allowing them to inspect how conclusions were reached. **Reproducibility** concerns whether the same analysis can be repeated and yield consistent results, a foundational requirement of scientific research. These three categories align with broader responsible AI governance principles commonly found in organizational frameworks but are specifically tailored to the research context where verifiability and scientific rigor are key. For an LLM application in research to be considered responsibly used, it should ideally achieve “yes” answers across all three criteria, ensuring that the technology enhances rather than compromises research integrity.

Applying the Responsible Use Framework

When we examine the first two usage types (e.g., code autocomplete and code generation) through the responsible usage framework, we find more encouraging alignment with the three essential criteria of correctness, transparency, and reproducibility. Code-generating and code-assisting LLMs produce verifiable output that researchers can inspect, test, and validate before execution, ensuring correctness through human review. The process maintains transparency because the generated code itself is visible and interpretable, allowing researchers to understand exactly what analytical steps are being performed. Reproducibility is achieved because the same code can be run multiple times on the same data to yield consistent results, and the code can be shared alongside research findings.

In contrast, the third approach where LLMs directly analyze qualitative or text data within a black box that is the LLM may raise critical questions about research integrity. Using LLMs for this purpose present inherent challenges against these principles: they are notorious for generating convincing but incorrect answers, operate as black boxes with limited transparency, and lack reproducibility due to their non-deterministic nature. When we apply this framework to direct text analysis by LLMs, significant concerns may emerge: we cannot guarantee correctness, the process lacks transparency, and reproducibility remains uncertain at best.

Achieving Responsible Usage Through Evidence-Based Results

However, for the third type of usage where LLMs directly analyze data (as we do in Chapters 9 and 10), we can work toward achieving “yes” answers across all (or as many as possible) three framework criteria by requiring LLMs to produce evidence-based results. As we cover in Chapter 9, it is possible to use a local LLM that is connected to R/Positron. In this approach, the researcher has control over the data analysis over simply pasting text into an LLMs chatbox. For example, this can be done by structuring prompts to demand transparent, verifiable outputs, such as requiring the LLM to identify themes, provide verbatim quotes from the source data, report frequencies with counts and percentages, and present findings in standardized tabular formats. We transform the black box into a more transparent analytical tool. This approach ensures **correctness** by grounding every claim in specific textual evidence that researchers can

verify, enhances **transparency** by making the analytical reasoning traceable through quoted examples and quantitative metrics, and improves **reproducibility** by standardizing the output format and maintaining clear documentation of the analytical process. When LLMs are constrained to cite their sources, quantify their observations, and structure their findings systematically, they shift from opaque pattern generators to accountable research assistants whose work can be validated against the original data. This methodology aligns with the recommendation for constrained use and micromanaged implementation, ensuring that LLMs remain within appropriate boundaries while still providing valuable analytical support for qualitative research.

Cloud vs Local LLMs

7.1 Overview

Large Language Models (LLMs) are now central to many research workflows in education, especially where analysis depends on large volumes of natural language data (Kasneji et al., 2023; Liu et al., 2025; OpenAI, 2023). Traditional NLP methods remain essential for transparent counting, comparison, and baseline modeling, but LLMs add complementary capabilities for context-sensitive synthesis, interpretation, and generation (Bommasani et al., 2021; Nelson et al., 2021; Than et al., 2025).

In educational research, this expands the range of feasible tasks:

- Summarizing and coding open-ended reflections
- Analyzing institutional policy documents
- Drafting instructional scaffolds and rubrics
- Connecting qualitative insights with quantitative findings

In short, LLMs extend the toolkit from pattern detection toward **context-aware meaning-making**, while still requiring careful validation, documentation, and methodological transparency.

7.2 Cloud-Based LLMs: Capabilities and Setup

Cloud-based LLMs are the quickest way to get started. You do not need to manage model files or local hardware. You send a request to a provider API, get a response, and continue your workflow.

That makes cloud models especially useful for rapid prototyping, large-scale text processing, and exploratory analysis.

Overview of Major Cloud Providers

The landscape moves fast. As of 2025, these are among the most common cloud options used in educational data analysis:

Table 7.1. Major cloud LLM providers and R access options.

| Provider | Example Models | Access | R / HTTP Wrapper |
|------------|----------------------------------|-----------------------------------|---------------------------------|
| OpenAI | GPT-4o / GPT-5 | API key via platform.openai.com | {httr2}, {openai}, {ellmer} |
| Anthropic | Claude 3 Opus · Claude 4 | API key via console.anthropic.com | {httr2}, {anthropic}, {ellmer} |
| Google | Gemini 2.5 · Gemini Ultra (2025) | AI Studio | {google GenerativeAI}, {ellmer} |
| Mistral AI | Mixtral 8x22B · Mistral 7B v0.3 | mistral.ai | HTTP via {httr2} |
| DeepSeek | DeepSeek-V3.1 (MoE 128K ctx) | API via deepseek.ai | HTTP via {httr2} |

Tip: always verify model names, context limits, and pricing before you run a study. Providers update frequently, and those updates can affect reproducibility.

Connecting to a Cloud API from R

Here is a minimal example using the OpenAI API with {httr2}. The same pattern applies to most HTTP-based providers.

```
library(httr2)
library(jsonlite)

api_key <- Sys.getenv("OPENAI_API_KEY")
base_url <- "https://api.openai.com"
endpoint <- paste0(base_url, "/v1/chat/completions")
```

```

resp <- request(endpoint) |>
  req_headers(
    "Authorization" = paste("Bearer", api_key),
    "Content-Type" = "application/json"
  ) |>
  req_body_json(list(
    model = "gpt-4o-mini",
    messages = list(
      list(
        role = "system",
        content = "You analyze educational data."
      ),
      list(
        role = "user",
        content = "Find three themes."
      )
    )
  )) |>
  req_perform()

content <- resp_body_json(resp)
cat(content$choices[[1]]$message$content)

```

Core API pattern: authenticate -> send prompt -> parse response.

Using {ellmer}: One Interface for Many Providers

The {ellmer} package (from the Posit ecosystem) gives you a unified interface across multiple providers. So instead of rewriting your code every time you switch models, you mostly change configuration values.

Why researchers like {ellmer}

- Unified syntax for OpenAI, Anthropic, Gemini, and other APIs
- Built-in streaming and function-calling support
- Structured JSON output parsing for downstream R workflows
- Compatible with both cloud and local OpenAI-style endpoints

Installation and Setup

```
install.packages("ellmer")
library(ellmer)

# Set API key (example: OpenAI)
Sys.setenv(OPENAI_API_KEY = "your_api_key_here")

# Create chat connection
chat <- llm_chat(
  provider = "openai",
  model    = "gpt-4o-mini",
  key      = Sys.getenv("OPENAI_API_KEY")
)
```

Example: Clustering Student Reflections

```
responses <- c(
  "I learned how to write better R code.",
  "Collaborating with peers improved my understanding.",
  "I struggled with data visualization."
)

prompt <- paste("Cluster into 3 short themes:\n-",
  paste(responses, collapse = "\n- "))

result <- llm_chat_complete(
  chat,
  messages = list(
    list(
      role = "system",
      content = "You analyze educational reflections."
    ),
    list(role = "user", content = prompt)
  )
)

cat(result$choices[[1]]$message$content)
```

Nice bonus: you can usually switch models by changing only `provider =` and `model =`.

Typical Educational Use Cases

- Automated qualitative coding of survey or interview data
- Summarizing student feedback at scale
- Drafting analytic memos or interpretive summaries
- Generating or evaluating teaching materials
- Rapid prototyping for mixed-methods research designs

Reproducibility and Ethical Considerations

Because cloud services change often, careful documentation is essential:

- **Record** model name, version, and query date
- **De-identify** any personal or institutional information before upload
- **Monitor** API usage and costs (token-based billing)
- **Disclose** clearly how LLMs assisted analysis or interpretation

Responsible AI use means balancing convenience with privacy, transparency, and data stewardship.

Summary

Cloud LLMs are fast, powerful, and easy to start with. For many projects, they are the best environment for exploration and iteration.

Next, we turn to local LLMs, where privacy and institutional control become the priority.

7.3 Local LLMs: Privacy-Preserving and Offline Analysis

Cloud models are convenient, but they can raise real concerns around **privacy, cost, and IRB compliance**. Local LLMs address these concerns by running directly on your machine.

What Are Local LLMs?

Local LLMs are open or custom models executed on local hardware. They process text without sending data to external servers, giving you full data sovereignty.

Common open-source families: Llama 3, Qwen, DeepSeek, Mistral, gpt-oss.

Key Advantages

- Data never leaves your device
- No API keys or internet required
- Highly customizable and often cost-free
- Enables fully offline reproducible analysis

Getting Started with LM Studio

LM Studio is a free cross-platform desktop app for running and managing local LLMs. It gives you a GUI for downloading models, testing prompts, and (optionally) exposing a REST API for automation.

Supported Platforms: macOS (Apple Silicon), Windows (x64/ARM64), Linux (x64)

Docs: lmstudio.ai/docs

Installation Steps

1. **Download LM Studio** for your system from the official site.
2. **Install and launch** the application.
3. **Download a model** such as Llama 3, Qwen, Mistral, or DeepSeek.
4. **(Optional)** Enable API access for scripting.
5. **(Optional)** Attach local documents to enable offline “Chat with Documents” (RAG mode).

Main Features

Table 7.2. Core LM Studio features for local LLM workflows.

| Feature | Description |
|---------------------|--|
| Local LLMs | Run models offline on your own machine |
| Chat Interface | Simple prompt-based GUI |
| Document Chat (RAG) | Offline “chat with your PDFs” |
| Model Management | Search, download, and switch models |
| API Access | OpenAI-compatible REST endpoints |
| Community Support | Active Discord and docs |

Calling the LM Studio API from R

LM Studio provides an OpenAI-compatible API, so your R code can look very similar to cloud-based examples:

```
library(httr)
library(jsonlite)

prompt <- paste("Summarize open-ended survey",
                "responses: ...")

request_body <- list(prompt = prompt, max_tokens = 200)

response <- POST(
  url = "http://localhost:1234/v1/completions",
  body = toJSON(request_body, auto_unbox = TRUE),
  encode = "json"
)

content(response)
```

Because requests stay local, sensitive data does not leave your environment.

7.4 Cloud vs. Local LLMs: Choosing the Right Tool

Table 7.3. Comparison of cloud and local LLM workflows.

| Criterion | Cloud-based LLMs | Local LLMs |
|----------------------|---|-----------------------------|
| Cost | Pay-per-token or subscription | Free (after hardware) |
| Privacy | Data sent to provider | Data stays local |
| Performance | Highest accuracy and speed | Depends on hardware |
| Maintenance | Automatic updates | Manual model management |
| Customization | Limited fine-tuning | Fully modifiable |
| Best for | Large public datasets or prototype analysis | Sensitive or regulated data |

A common workflow is hybrid: prototype quickly in the cloud, then reproduce locally for privacy and auditability.

7.5 Practical Setup Checklist

Before running LLM-based analyses, check the following:

- Select your preferred model and platform
- Configure API key (cloud) or local endpoint (LM Studio)
- Test connectivity with a short prompt
- Log model name, version, and date
- De-identify data and store outputs securely

This small checklist can save you from major reproducibility and compliance problems later.

7.6 Summary

Both cloud and local LLMs can support strong educational research. The right choice depends on your project constraints.

Table 7.4. Recommended LLM environment by research use case.

| Use Case | Cloud LLM | Local LLM |
|-----------------------------|-----------|-----------|
| Rapid prototyping | Yes | No |
| Large-scale text processing | Yes | No |
| Sensitive student data | No | Yes |
| Offline analysis | No | Yes |
| Long-term reproducibility | No | Yes |

In short: cloud LLMs excel at convenience and scale; local LLMs excel at privacy and control. Most research programs benefit from using both strategically.

Looking Ahead

- **Chapter 9** demonstrates thematic and qualitative text analysis with LM Studio, including end-to-end coding and synthesis.
- **Chapter 10** extends this workflow to multimodal data (images), showing how AI can connect diverse forms of evidence in educational research.

Positron Assistant and Databot

8.1 Overview

The use of AI tools to assist with research coding and analysis has grown rapidly in educational research. Early comparisons of hand-coding and computer-assisted methods showed meaningful trade-offs between the two approaches (Nelson et al., 2021), and recent work has demonstrated that large language models can perform qualitative coding with sufficient reliability for many research tasks — though with important caveats about where they work well and where they do not (Liu et al., 2025; Than et al., 2025). This chapter introduces two tools that bring LLM assistance directly into the R and Positron workflow, making these capabilities accessible without requiring researchers to leave their analysis environment.

Positron Assistant functions as an in-IDE research coding assistant: it can support drafting, debugging, and code interpretation without requiring researchers to move between multiple tools. In this chapter, we focus on two tools in the same ecosystem: **Positron Assistant** and **Databot**.

Positron Assistant is the primary pathway in this chapter for chat, in-line support, and code suggestions. Databot is presented as an optional exploratory extension, since it is currently in research preview.

The chapter is written for readers with different levels of R experience, with emphasis on practical workflows that can be validated and documented.

8.2 Positron Assistant in the Research Workflow

Positron Assistant can use project and workflow context to provide more relevant support than generic chatbot responses.

8.2.1 What Can It Do?

Core capabilities include:

- **Code Suggestions:** Not just basic autocomplete, but context-aware snippets that can help you move from intent to working code more quickly.
- **Code Explanation:** Select a code block and request an explanation in plain language.
- **Bug Detection Support:** When code fails, Positron Assistant can inspect errors and suggest fixes.
- **Code Generation:** Provide a plain-language request (for example, grouped summaries with missing-value handling) to draft R code.
- **Refactoring:** Propose cleaner versions of working code and explain tradeoffs.

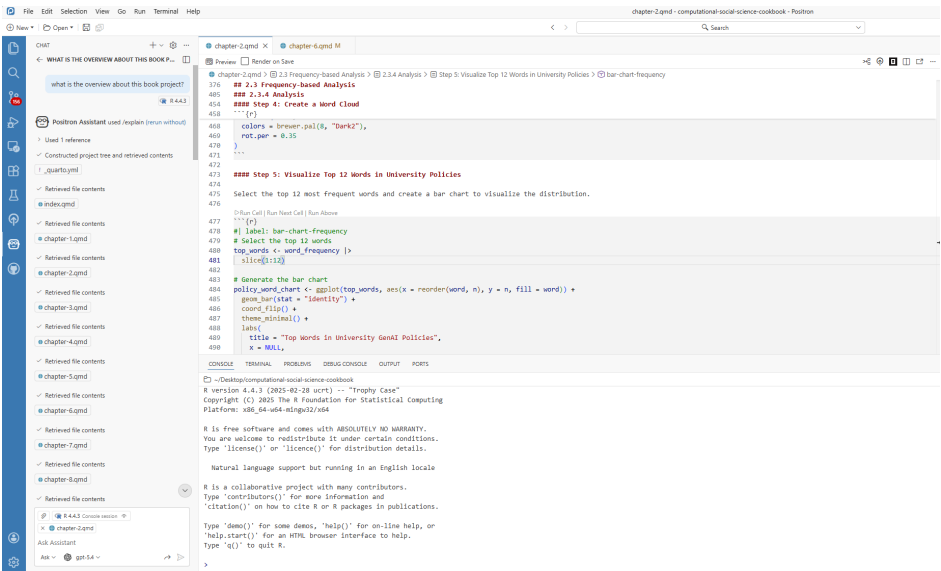


Figure 8.1. Positron Assistant panel in the research coding workflow.

8.2.2 Why Would an Educational Researcher Want This?

For educational researchers, the value is not replacement of expertise but support for routine analytical tasks:

- **Saving time on repetitive tasks** — for example, drafting common data-transformation patterns.
- **Learning new approaches** — suggestions often expose analysts to alternative R workflows.
- **Reducing debugging overhead** — error interpretation and fix suggestions are available in context.
- **Building confidence for newer R users** — iterative feedback can lower the barrier to experimentation.

A gentle reminder: Positron Assistant is a tool to work *with*, not a replacement for your expertise as a researcher. It is great at generating code, but you are the expert on your data and research questions. Always review what it generates—treat it as a first draft that needs your expert polish.

8.3 Getting Started

8.3.1 What You Will Need

Before we dive in, make sure you have:

- **Positron** installed (use a current release; Positron Assistant documentation currently tracks the 2025.07+ generation of features)
- At least one configured **model provider** (for example, Anthropic, GitHub Copilot, OpenAI, Amazon Bedrock, or Snowflake Cortex)
- **R** (version 4.0 or later) installed on your machine

You should also know your institution’s guidance on external AI tools before using non-public data.

8.3.2 Setting Things Up

Getting started is straightforward:

1. **Download and install Positron** from the website
2. **Enable Positron Assistant** in Settings.
 - Setting: `positron.assistant.enable`

- 3. **Reload Positron** (restart or run Developer: Reload Window)
- 4. **Configure a provider** with Positron Assistant: Configure Language Model Providers
- 5. **Open chat** (robot icon or Chat: Open Chat) and start asking questions

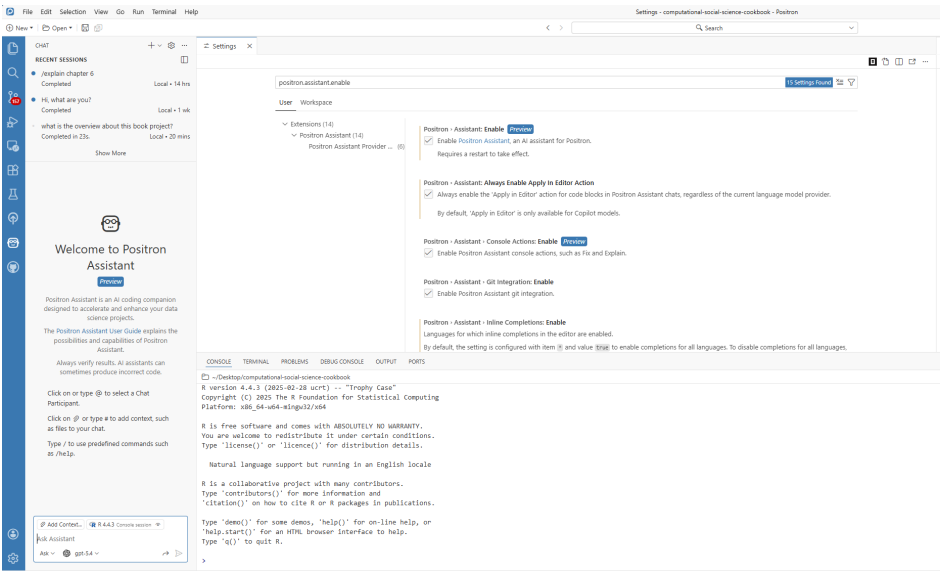


Figure 8.2. Positron setting to enable Assistant in the IDE.

8.3.3 Making It Feel Right for You

Positron Assistant can adapt to your style. In settings, you can adjust:

- How detailed responses should be (brief vs. thorough)
- Whether it prefers tidyverse-style code or base R
- Whether generated code should include comments

Tune these settings based on your workflow and project requirements.

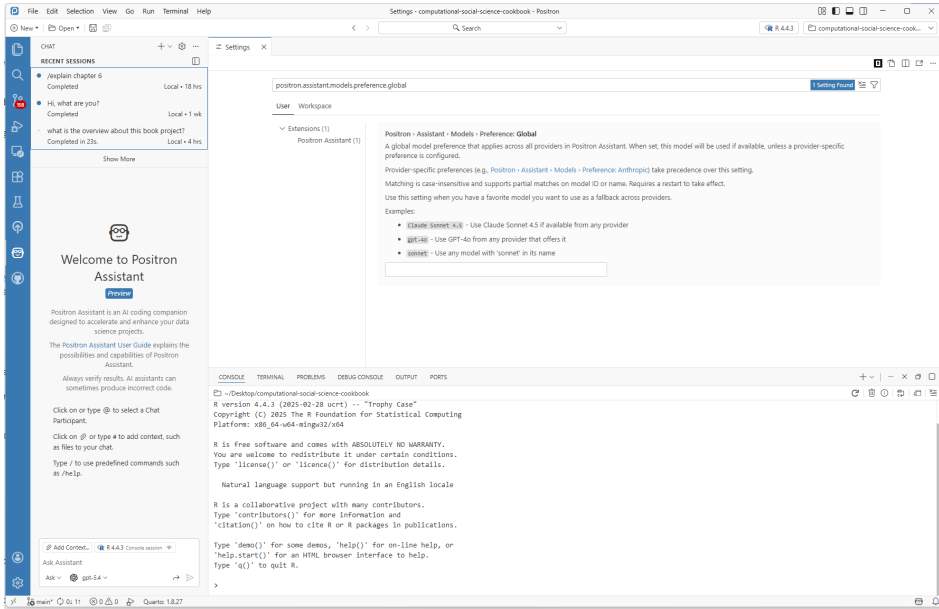


Figure 8.3. Assistant preference options for response style and coding defaults.

8.3.4 Choosing a Provider for Research Work

If you are working in educational research settings, choose providers with your workflow constraints in mind:

- **Compliance first:** If your institution restricts providers, follow that policy before convenience.
- **Cost awareness:** API-key providers are often usage-based; budget and monitor token usage.
- **Reproducibility:** Record provider + model name in your methods notes.
- **Stability:** Keep one default model for a project to reduce output variability.

For most readers, a good starting strategy is simple: pick one approved provider, use one primary model for the project, and document those decisions.

8.3.5 Optional: Local Agent Setup with Continue + LM Studio

If your project requires local execution, you can run an assistant workflow entirely on your own machine by combining Continue in Positron with a local model served from LM Studio. This path is especially useful when your workflow requires tighter data control.

Below is a practical setup order that matches a real configuration process.

1. **Start in Positron and define local-model JSON settings first.**

This ensures your assistant config already points to a local endpoint before you start testing.

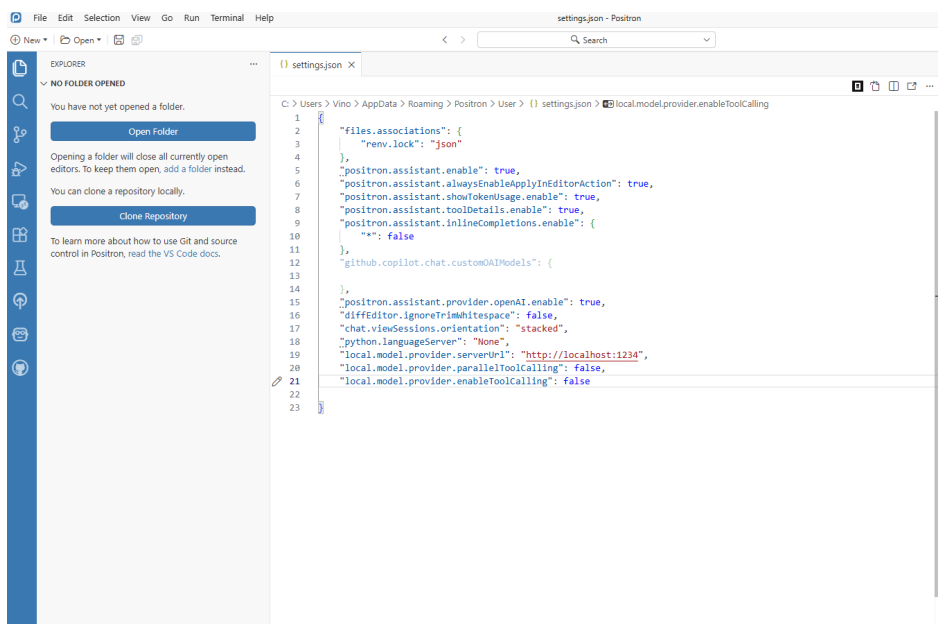


Figure 8.4. Positron JSON settings for local LM Studio endpoint configuration.

2. **Set up LM Studio and confirm the local server is running.**

Open LM Studio, load your target model, and verify the local API/server endpoint is active.

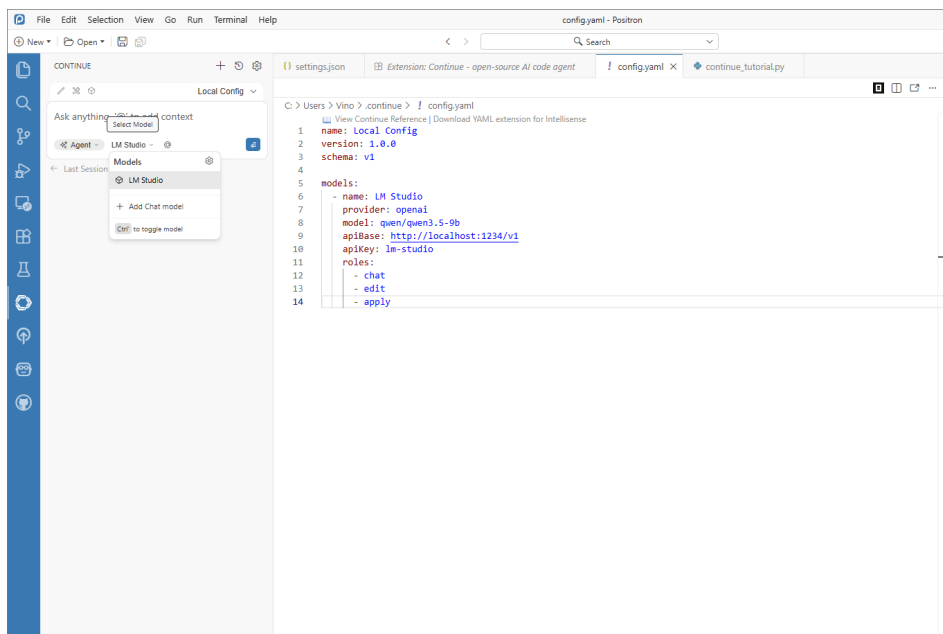


Figure 8.5. LM Studio local server setup for Positron integration.

3. Configure Continue in Positron to use the LM Studio endpoint.

Confirm provider/model mapping in Continue settings and ensure the selected model is the same one currently loaded in LM Studio.

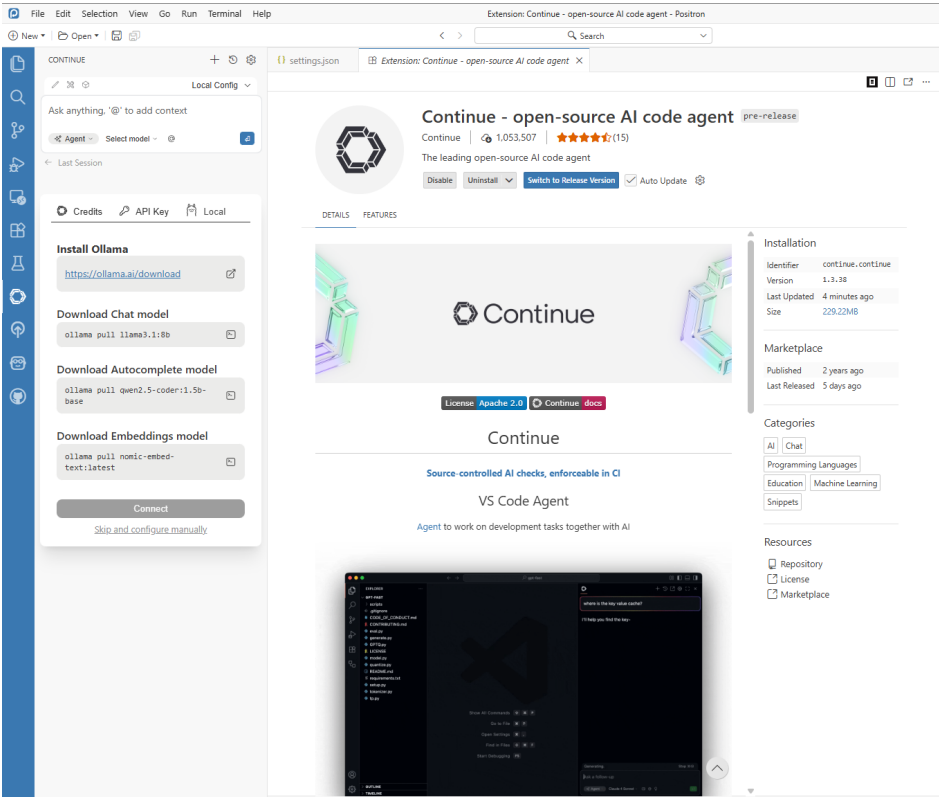


Figure 8.6. Continue extension settings mapped to the LM Studio local model.

If your Continue workflow is YAML-based, configure the same end-point and model there as well:

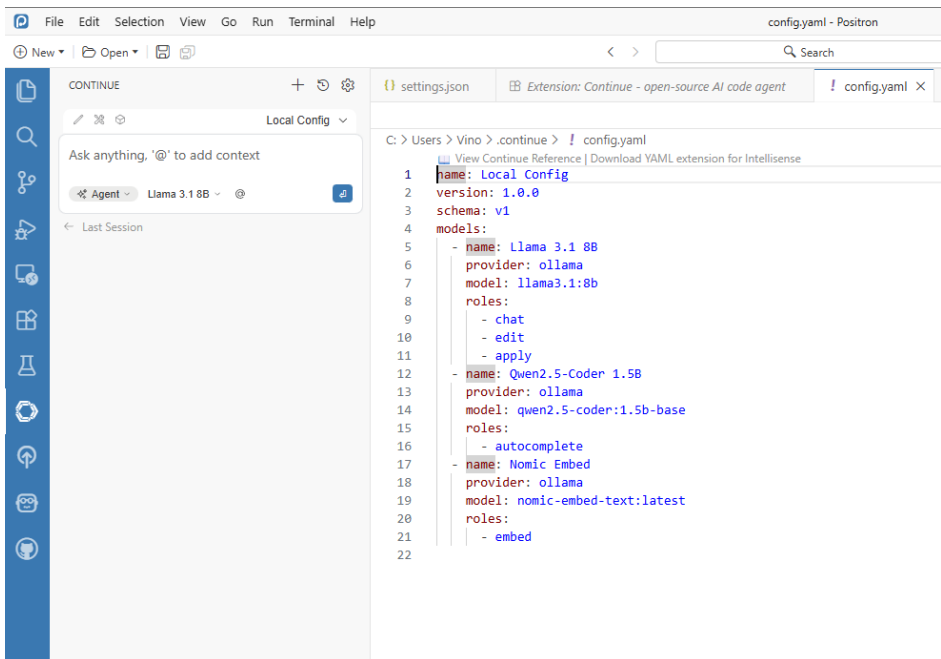


Figure 8.7. Continue YAML configuration for local model endpoint and provider mapping.

4. Run a quick in-IDE verification prompt.

Use a small, low-risk task first (for example, “Explain this dplyr pipeline” or “Draft a data cleaning function”) to verify response quality and latency.

Positron Assistant and Databot

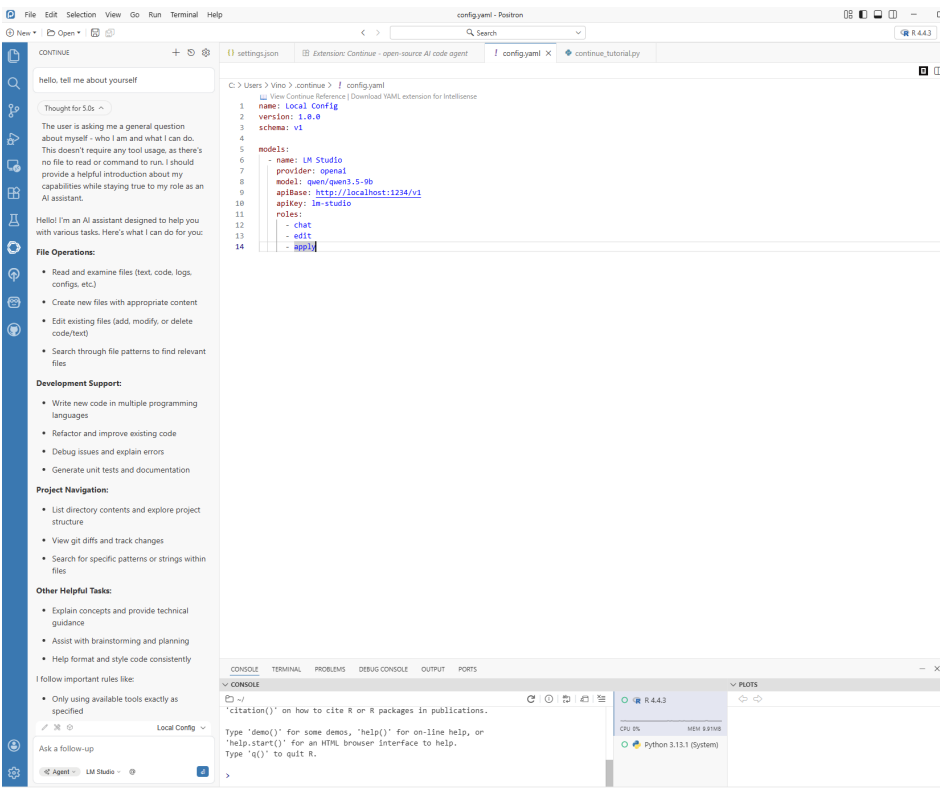


Figure 8.8. In-IDE verification prompt using Positron Assistant with LM Studio.

5. Run one concrete case example before using the workflow in research analysis.

This final check confirms that local agent behavior is stable enough for your chapter-level tasks.

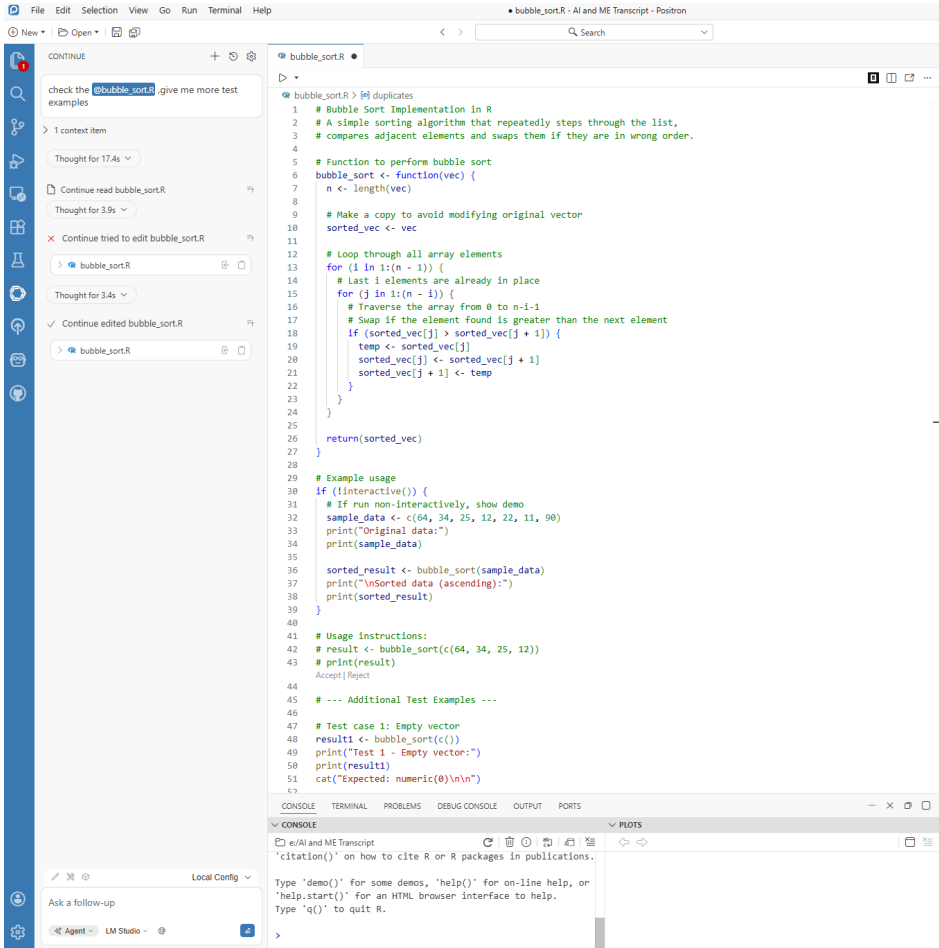


Figure 8.9. Case example response from Positron Assistant using a local model.

In short, configure Positron first, confirm LM Studio second, wire Continue third, and only then move to examples. This sequence avoids most connection and model-mismatch errors.

8.4 Using Positron Assistant

After setup, the following examples illustrate common usage patterns.

8.4.1 Getting Code Explained

Code explanation is useful when adapting snippets from tutorials, collaborators, or prior projects.

- 1. Highlight the confusing code
- 2. Ask Positron Assistant for a step-by-step explanation
- 3. Review the plain-language interpretation

This feature is especially useful for onboarding new team members and documenting analysis logic.

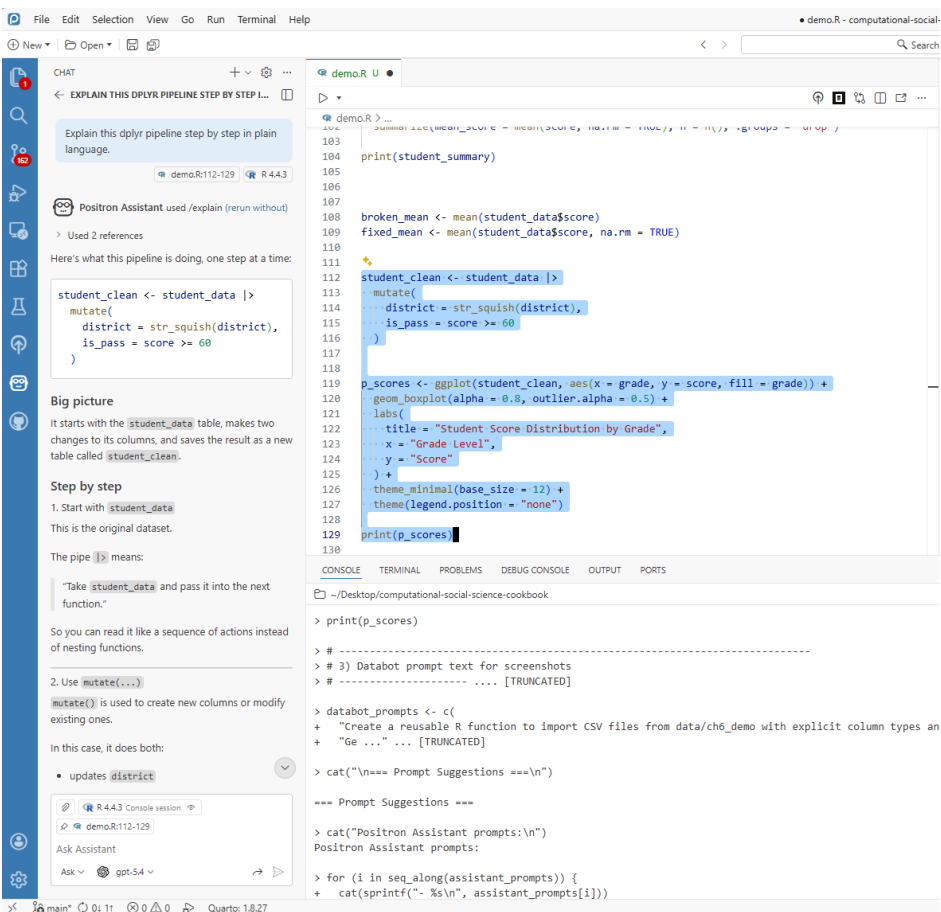


Figure 8.10. Assistant-generated plain-language explanation of selected code.

8.4.2 When Things Break

A common debugging scenario is shown below:

```
# You run this, and R throws an error
mean(student_data$score)
```

Instead of manually searching external forums first, pass the error to Positron Assistant for an initial diagnosis. A typical response might be:

“I see the issue! You are trying to calculate a mean, but there are missing values in your data. Add `na.rm = TRUE` to ignore them, or decide whether those NAs should be handled differently. Want me to show you both options?”

This response can be used as a starting point for verification.

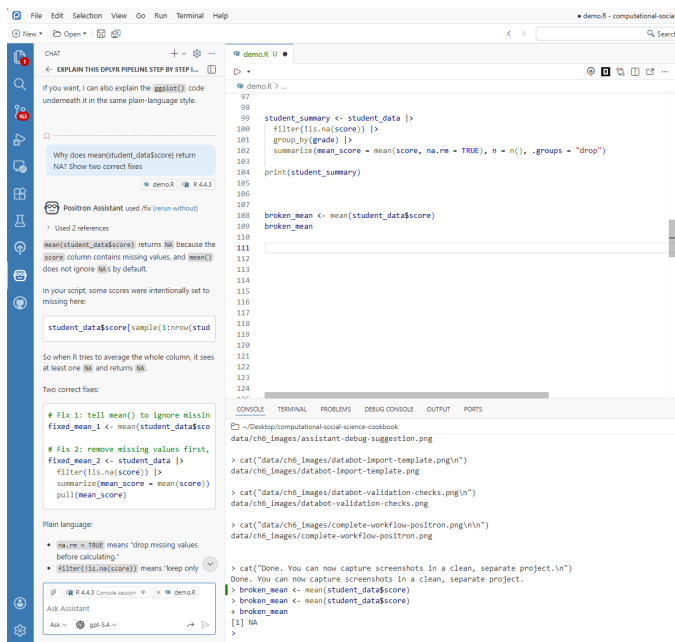


Figure 8.11. Assistant debugging suggestion for a code error in context.

8.5 Meet Databot (Experimental): Your EDA Accelerator

If Positron Assistant is your coding colleague, Databot is your exploratory analysis accelerator. It is designed for rapid, iterative data exploration and can generate and run short analysis steps.

Databot is currently experimental (research preview), so treat it as an acceleration layer for exploration, not as an unreviewed source of final results.

8.5.1 What Can Databot Handle?

Example tasks include:

- **Generating import templates** — instead of writing “`read.csv()`” from scratch every time you get a new dataset, let Databot create a reusable function
- **Building data cleaning pipelines** — those steps you do every single time (convert text to lowercase, strip whitespace, handle missing values) can be automated
- **Creating report templates** — if you generate similar reports quarterly, Databot can build the skeleton for you
- **Setting up quality checks** — automated tests that catch data problems before they become headaches

8.5.2 Example: Never Write Import Code Again

Databot can generate reusable import templates for recurring file structures:

8.5 Meet Databot (Experimental): Your EDA Accelerator

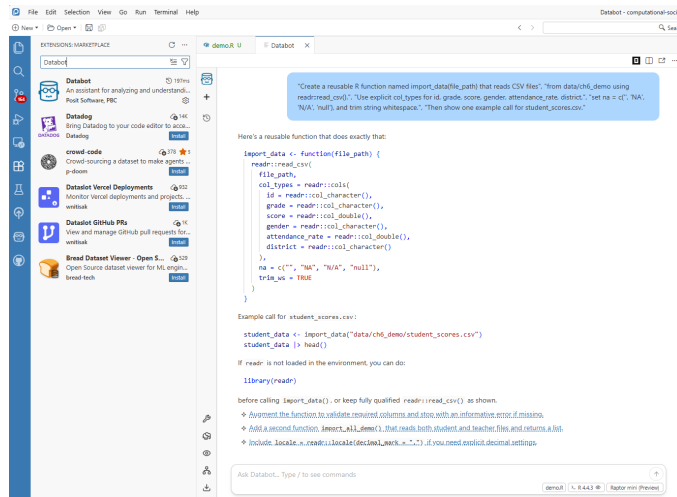


Figure 8.12. Databot-generated reusable import template for recurring datasets.

```
# This template was created once, now you use it forever!
library(readr)
library(here)

import_data <- function(file_path) {
  read_csv(
    file = here("data", file_path),
    col_types = cols(
      id = col_character(),
      score = col_double(),
      grade = col_factor(),
      .default = col_character()
    ),
    na = c("", "NA", "N/A", "null")
  ) |>
  mutate(across(where(is.character), str_squish))
}

# Now importing is one line:
# student_data <- import_data("student_scores.csv")
```

The resulting function can reduce repeated setup work across similar

datasets.

8.5.3 Example: Automatic Quality Checks

For incoming survey datasets, Databot can draft validation checks before substantive analysis begins:

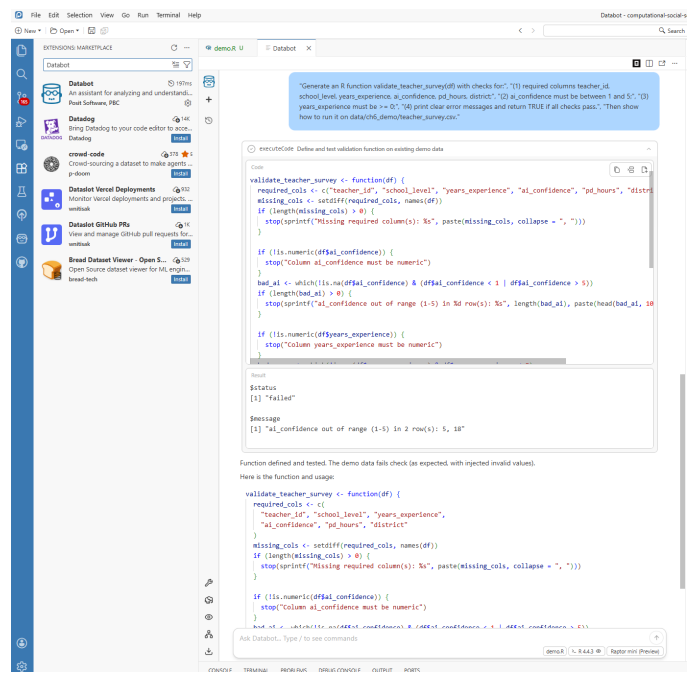


Figure 8.13. Databot-generated data validation checks for incoming survey files.

```
validate_student_data <- function(df) {
  required <- c("id", "score", "grade")
  missing <- setdiff(required, names(df))
  details <- paste(missing, collapse = ", ")
  if (length(missing) > 0) stop("Missing: ", details)
  if (anyNA(df$id)) stop("Missing IDs found")
  bad_score <- df$score < 0 | df$score > 100
  if (any(bad_score, na.rm = TRUE)) stop("Out of range")
  message("All validation checks passed!")
}
```

This creates a repeatable validation layer that can be applied whenever new data arrive.

8.6 Using These Tools in Research Practice

8.6.1 A Concrete Example

To keep this chapter reproducible, use the demo files from `data/ch6_demo/`: `student_scores.csv` and `teacher_survey.csv`.

In this example workflow, you can:

- ask Databot to draft an import function for `student_scores.csv` (Section 8.5.2),
- ask Databot to generate validation checks for `teacher_survey.csv` (Section 8.5.3),
- use Positron Assistant to refine or explain the generated code,
- run the cleaned analysis pipeline and produce a quick plot for reporting.

The screenshot below should show this end-to-end flow in one workspace view: editor code, Databot/Assistant output, Console results, and a plot. The goal is not to present final publication results, but to document a realistic AI-assisted analysis process from import to quality checks to exploratory output.

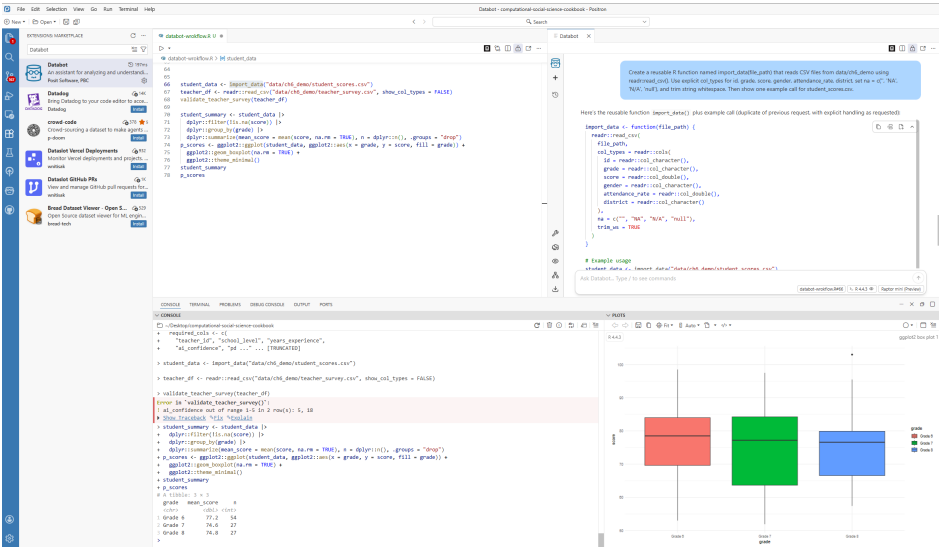


Figure 8.14. End-to-end AI-assisted workflow from import to validation and exploratory output.

8.6.2 What This Means for Your Research

The key value is not only speed, but allocation of researcher attention. When routine coding demands are reduced, more effort can be directed toward:

- Understanding your data deeply
- Choosing the right analytical approaches
- Thinking critically about what your findings mean
- Communicating results effectively to your audience

8.7 Working Responsibly with AI Helpers

These tools are useful, but they do not replace researcher judgment. Responsible use requires explicit verification and documentation.

8.7.1 Always Review What You Get

Treat AI-generated code like a first draft from a well-meaning but imperfect colleague. It might be mostly right, but it is probably not perfect. Before you use any code it generates in actual research:

- **Test it on a small sample** first—run the code on a subset of your data to make sure it does what you expect
- **Verify the results** against something you know to be true, or compare with a manual calculation
- **Add your own comments** explaining what the code is doing, especially if the logic is complex

8.7.2 Be Transparent

If AI tools are used in a research workflow, report that usage transparently in the methods section. For example:

“We used the Positron AI Assistant to help generate initial data processing code, which was then reviewed and validated by the authors.”

This makes the analytic process easier for readers to evaluate and replicate.

8.7.3 Keep Things Reproducible

A few tips to make sure your work still holds up:

- **Note what version** of the AI tools you used
- **Keep your human-written code** as the authoritative version—do not rely solely on what the AI generated
- **Use version control** (like Git) to track changes, so you can always see the history of your work

8.7.4 Protect Data and Document AI Use

For AI-assisted workflows, keep a short project log with:

- Positron version
- provider name (for example, Anthropic or GitHub Copilot)
- model name
- date/time of major AI-assisted runs
- what was AI-generated vs. researcher-edited

Also remember the privacy boundary: Positron routes requests to your selected provider. Posit documents that it does not store your prompts and AI conversations, but your chosen provider may have its own data retention policy. Review provider terms before using sensitive data.

8.8 Summary

This chapter highlights a practical integration pattern:

Positron Assistant and Databot can support repetitive coding tasks, contextual learning, and exploratory analysis. They do not replace domain expertise or methodological oversight.

Used well, they amplify core researcher strengths: question design, contextual interpretation, and critical evaluation of evidence.

Key Points to Remember

- Positron Assistant offers real-time code help, explanations, and debugging support
- Databot can accelerate exploratory analysis and repetitive scaffolding, but requires careful review
- Always review and validate AI-generated code—it is a starting point, not the final answer
- Being transparent about AI use builds trust with your readers
- These tools free you up to focus on what matters: your research

The next chapter extends this discussion to **local LLM workflows for text analysis**, with emphasis on privacy-preserving qualitative methods.

Local LLMs for Text

9.1 Overview

Cloud-based AI tools are often effective for coding and analysis, but they may be difficult to use in projects involving sensitive educational data. In many settings, researchers must address institutional and regulatory requirements before sending text to external services. Local LLM workflows provide an alternative path: analysis remains on the researcher's own machine while preserving the benefits of model-assisted interpretation.

In this chapter, we introduce **LM Studio** as a practical interface for running local models and connecting them to R-based research workflows.

9.2 What is a Local LLM?

A **local LLM** is a large language model that runs on local hardware rather than a remote provider API. In this setup, prompts, documents, and outputs can remain within the local environment.

Why would you want this?

When working with student responses, interview transcripts, or institutional policy text, privacy is often a core requirement rather than a preference.

Many local setups use open-weight model families such as **Llama** (Meta), **Qwen** (Alibaba Cloud), **DeepSeek** (DeepSeek), and **Mistral** (Mistral AI). These are model lineages with multiple size variants (for example, smaller models for faster local inference and larger models for richer reasoning), allowing researchers to balance speed, quality, and hardware limits.

Key advantages of local LLMs:

- **Privacy by design** — your data never leaves your computer

- **No API key hassles** — no need to sign up for external services
- **Totally offline** — work without an internet connection
- **Free to use** — open-source models mean no per-token costs

9.3 What Can Local LLMs Do in Educational Research?

In practice, local models can support several common research tasks:

- **Text analysis at scale** — summarize, paraphrase, or dig into themes in open-ended survey responses, interview transcripts, or focus group data
- **Theme extraction** — have the LLM identify key patterns in your qualitative data without manually coding everything
- **Report drafting** — generate first drafts of findings sections or analytic memos
- **Document Q&A** — chat with your PDFs (“wait, what did participant #15 actually say about their learning experience?”)
- **Integration with your existing workflow** — connect local LLMs to R, Python, or other tools you already use

All of these tasks can be executed locally when the model and serving environment are configured on the researcher’s machine.

9.4 LM Studio as a Local Model Environment

LM Studio provides a practical interface for downloading, managing, and serving local models.

Core capabilities of LM Studio include:

- **Cross-platform** — works on Mac, Windows, and Linux
- **User-friendly** — no command-line setup required for standard use
- **Model variety** — choose from Llama, Qwen, DeepSeek, Mistral, and many others
- **API ready** — can serve models via a simple API for integration with R, Python, or other tools
- **Chat with Documents** — upload PDFs and query them entirely offline

Key points:

- **Supported platforms:** macOS (Apple Silicon), Windows (x64/ARM64), and Linux (x64)
- **System requirements:** for best results, review System Requirements for recommended RAM, CPU/GPU, and storage

9.4.1 Getting LM Studio Up and Running

Setup is usually straightforward:

1. **Grab LM Studio** from lmstudio.ai/download — pick the version for your operating system
2. **Install and open it** — just like any other app
3. **Download a model** — LM Studio makes it simple to browse and download popular models (Llama 3, Qwen, Mistral, etc.)
4. **Run a quick test prompt** — verify that the model responds before integrating with R

9.4.2 Connecting LM Studio to R

LM Studio exposes an **OpenAI-compatible API**, so R code can call a local model endpoint with a familiar request pattern.

Here is a simple example:

```
library(httr)
library(jsonlite)

prompt <- paste(
  "Summarize the following open-ended survey responses:",
  "...")

response <- POST(
  url = "http://localhost:1234/v1/completions",
  body = toJSON(
    list(prompt = prompt, max_tokens = 200),
    auto_unbox = TRUE
  ),
```

```
    encode = "json"
)
content(response)
```

9.4.3 At a Glance: LM Studio Capabilities

Table 9.1. LM Studio capabilities for local research workflows.

| Feature | What It Means for You |
|---------------------|---|
| Local LLMs | Run powerful AI models on your own machine—no internet needed |
| Chat Interface | Easy, intuitive way to interact with your model |
| Document Chat (RAG) | “Chat with your PDFs” while staying fully offline |
| Model Management | Download and organize different models easily |
| API Access | Connect to R, Python, or other tools you already use |
| MCP Integration | Advanced features for power users |
| Community & Support | Helpful Discord community and solid documentation |

9.5 Putting It All Together: A Real Example

This section demonstrates a local LLM workflow using the same university AI policy corpus introduced in Chapter 2. Here, the emphasis is thematic analysis with a locally served model.

9.5.1 Research Question

This section focuses on a single research question:

- What are the key themes in university AI policy statements?

By using the same dataset from Chapter 2, we can compare local-LLM thematic outputs with conventional text-analysis baselines.

9.5.2 The Data We Are Working With

We will reuse the **AI policy statements** dataset from Section 2—just the text content this time, stripped of institution names for that extra layer of privacy. Each record has a single field:

- **Stance** (character): the actual policy text

We will pull out that **Stance** field so our results are directly comparable to what we got in Section 2.

```
library(dplyr)
library(stringr)
library(readr)

# Use 'university_policies' if it already exists.
# Otherwise, read the same CSV used in Section 2.
if (!exists("university_policies")) {
  university_policies <- read_csv(
    "data/University_GenAI_Policy_Stance.csv",
    show_col_types = FALSE
  )
}

stopifnot("Stance" %in% names(university_policies))

policy_texts <- university_policies$Stance %>%
  as.character() %>%
  stringr::str_squish() %>%
  na.omit()

length(policy_texts)
```

```
policy_preview <- head(policy_texts, 3) %>%
  str_trunc(width = 48)
preview_lines <- paste0(
  seq_along(policy_preview),
  ". ",
  policy_preview
)
cat(preview_lines, sep = "\n")
```

9.5.3 Time to Let the Local LLM Do Its Thing

We are going to send our policy texts to the local model running in LM Studio. The setup is straightforward—you will need your `api_base` and `model_name` ready to go (we are using `openai/gpt-oss-20b` in this example, but you can swap in whatever model you prefer).

```
library(httr)
library(jsonlite)
library(glue)
library(stringr)

# Use the connection parameters defined earlier.
api_base <- "http://127.0.0.1:1234/v1"
model_name <- "openai/gpt-oss-20b"
```

Testing the Local Connection

Before running large jobs, it is good practice to confirm that LM Studio is responding correctly. A quick “ping test” helps prevent silent connection errors.

```
library(httr)
library(jsonlite)

api_base <- "http://127.0.0.1:1234/v1"
model_name <- "openai/gpt-oss-20b"

res <- POST(
```

```

url = paste0(api_base, "/chat/completions"),
add_headers("Content-Type" = "application/json"),
body = toJSON(list(
  model = model_name,
  messages = list(
    list(
      role = "system",
      content = "You are a helpful assistant."
    ),
    list(
      role = "user",
      content = "Please reply with 'pong'"
    )
  )
), auto_unbox = TRUE)
)

cat(content(res)$choices[[1]]$message$content)

```

If the model replies with “pong,” you are good to go!

Prompt writing

Next, we write the prompt template. Because our goal is thematic pattern detection in policy documents, the prompt requests structured outputs rather than a free-form narrative. This is important: prompt design can require the model to return table-ready fields that are easier to validate and compare.

```

# ----- 1) Prompt template -----
table_header <- paste0(
  "| Theme | Description | Illustrative Example(s) | ",
  "Frequency | Relative Frequency |"
)
theme_row_1 <- paste0(
  "| [Theme 1] | [Description] | - \\\"[Quote]\\\" | ",
  "[n] | [p]% |"
)
theme_row_2 <- paste0(

```

```
"| [Theme 2] | [Description] | - \\\\"[Quote]\\\" | ",  
"[n] | [p]% |"  
)
```

```
prompt_instructions <- "  
You are analyzing official university AI policy  
statements.  
Your task is to identify 3-5 key themes across  
the statements  
and report them in the exact format below.  
  
**INPUT DATA:**  
- **Number of Statements:** {n_items}  
- **Policy Statements:**  
{items}  
  
**YOUR TASK:**  
1) Identify 3-5 key themes across the policy statements.  
2) For each theme:  
  a) Provide a concise theme name.  
  b) Provide a 1-2 sentence description.  
  c) Provide one short verbatim example quote.  
  d) Provide an integer Frequency  
    (count of statements mentioning it).  
  e) Provide Relative Frequency as a whole-number  
    percentage.  
3) Write a 3-5 sentence **Summary of Responses**  
   synthesizing  
   the most important insights.  
"
```

```

prompt_output_format <- "
4) Output strictly in the following format:

**Summary of Responses**
[3-5 sentence narrative summary goes here.]

**Thematic Table**
{table_header}
|---|---|---|---|---|
{theme_row_1}
{theme_row_2}
"

analysis_prompt_template <- paste(
  prompt_instructions,
  prompt_output_format,
  sep = "\n"
)

```

Chunks!

Next, we define chunk sizes for the local LLM to analyze our data. In qualitative text analysis using LLMs (such as thematic synthesis or coding), chunk size refers to the amount of text you pass to the model at one time. It directly affects coherence, depth, and efficiency of analysis.

Chunk size balances context preservation and analytic precision in qualitative LLM-based text analysis. If chunks are too small, the model loses semantic coherence, producing fragmented or repetitive themes. If too large, it may miss local nuances or exceed the model's reasoning capacity. The aim is to maintain enough continuity for meaningful interpretation while staying within manageable input limits.

Practically, chunk size should follow natural meaning units, such as paragraphs, speaker turns, or short sections, rather than fixed word counts. Researchers typically find that 500–1000 words work well for transcripts, while longer documents like policies can be chunked at 1000–1500 words. The guiding principle is to choose the smallest segment that preserves interpretive coherence.

A good rule of thumb: for policy documents, chunks of 10–20 items tend

to work well. That gives the model enough context to find meaningful patterns without overwhelming it.

```
# ----- 2) Chunk the corpus -----  
CHUNK_SIZE <- 15  
chunk_ids <- ceiling(seq_along(policy_texts) / CHUNK_SIZE)  
chunks <- split(policy_texts, chunk_ids)
```

Connecting to LM Studio

Once our data is prepared, our next step is to pass it to LM Studio. Using our function below, we send our text data to LM Studio server.

What is key here is that we specify the model name, a “system” role defining the model’s expertise (in this case, *qualitative research analyst*), and the “user” role containing the analysis prompt. The parameters `temperature = 0.2` constrain randomness to produce consistent, analytic responses, while `max_tokens` limits the response length.

- **Temperature** controls randomness: a low value (0.2) produces consistent, analytical responses suited to qualitative coding, while higher values encourage creativity but reduce reliability.
- **Max tokens** limits response length. Setting it to 1000 ensures sufficient detail without verbosity or truncation. Together, these parameters balance precision and completeness in model-generated analyses.

In essence, this helper encapsulates the logic of prompt dispatch and result retrieval, ensuring each call to the LLM is standardized and repeatable. This is crucial for qualitative workflows where traceability and parameter control are essential.

```
# ----- 3) Call LM Studio -----  
call_lmstudio <- function(prompt, max_tokens = 1000) {  
  res <- httr::POST(  
    url = paste0(api_base, "/chat/completions"),  
    httr::add_headers(  
      "Content-Type" = "application/json"  
    ),  
    body = jsonlite::toJSON(list(  
      model = model_name,  
      messages = list(  
        list(role = "system", content = system_prompt),  
        list(role = "user", content = prompt)  
      )  
    )  
  )  
  return(res)
```

```

messages = list(
  list(
    role = "system",
    content = paste(
      "You are an expert qualitative",
      "research analyst."
    )
  ),
  list(role = "user", content = prompt)
),
temperature = 0.2,
max_tokens = max_tokens
), auto_unbox = TRUE)
)
http::stop_for_status(res)
content(res)$choices[[1]]$message$content
}

```

Running the analysis

Now, the script applies the `analysis_prompt_template` to each *chunk* of transcript data using `lapply()`. Each chunk is converted into a numbered text block (`items_block`) and analyzed independently through `call_lmstudio()`, producing localized thematic results (`chunk_outputs`).

Second, the `meta_prompt` integrates these separate analyses. It instructs the model to synthesize and deduplicate themes across all chunks into a unified framework, including a concise narrative summary and a structured thematic table with descriptions, examples, and frequency data. Together, these steps move from micro-level coding to macro-level interpretation. This step is optional, and can be skipped depending on the nature of data and research questions.

Think of it like this: first, we get multiple perspectives from looking at pieces of the picture, then we step back and ask the model to connect all those perspectives into a coherent whole.

```

# ----- 4) Run thematic analysis per chunk -----
chunk_outputs <- lapply(chunks, function(vec) {

```

```
numbered_items <- sprintf("%d. %s", seq_along(vec), vec)
items_block <- paste(numbered_items, collapse = "\n")
final_prompt <- glue(analysis_prompt_template,
                     n_items = length(vec),
                     items   = items_block)
call_lmstudio(final_prompt)
})
```

```
# ----- 5) Merge chunk-level analyses -----
meta_prompt <- "
You will synthesize multiple chunk-level thematic
analyses of
the same corpus of university AI policies.
Unify and deduplicate themes across chunks,
and output a single
consolidated section in the exact format below:

**Summary of Responses**
[3-5 sentence narrative summary.]

**Thematic Table**
{table_header}
|---|---|---|---|---|
{theme_row_1}
{theme_row_2}
"
```

Synthesizing and Final LLM Analysis

We are now back in R synthesizing our data (and manage token limits efficiently).

The `chunk_outputs` are split into smaller *pairs*, each containing two analyses. Each pair is merged and passed through `call_lmstudio()` using the same `meta_prompt`, producing intermediate syntheses (`pair_outputs`). These summaries are then combined into a single consolidated input (`final_meta_input`) for a final call to `call_lmstudio()`, yielding the comprehensive meta-analysis (`meta_output`).

This iterative merging reduces token usage, preserves coherence, and ensures that the final synthesis integrates all thematic insights with-

out exceeding model constraints. With `saveRDS(meta_output, "outputs/meta_output_saved.rds")` we save our analysis so that in the future, we can just start from there to pick things back up.

To keep things manageable, we combine the chunk results in pairs, do a synthesis step for each pair, and then do one final synthesis to get our grand unified theme set. This keeps the model from getting overwhelmed while still giving us a comprehensive result.

```
# Pairwise synthesis to reduce token usage
pair_ids <- ceiling(seq_along(chunk_outputs) / 2)
pairs <- split(chunk_outputs, pair_ids)

pair_outputs <- lapply(pairs, function(group) {
  meta_input <- paste(group, collapse = "\n\n---\n\n")
  combined_prompt <- paste(
    meta_prompt,
    meta_input,
    sep = "\n\n"
  )
  call_lmstudio(combined_prompt)
})

# Now you have fewer intermediate syntheses
final_meta_input <- paste(
  pair_outputs,
  collapse = "\n\n---\n\n"
)
meta_output <- call_lmstudio(
  paste(meta_prompt, final_meta_input, sep = "\n\n")
)
cat(meta_output)

#saveRDS(meta_output, "outputs/meta_output_saved.rds")
saveRDS(meta_output, "outputs/meta_output_saved.rds")
```

Thematic Table Extraction and Cleaning

This code takes the saved meta-analysis from LM Studio and turns it into a clean, usable table in R. It first combines all elements of the output into a single text block, then extracts only the lines that make up the markdown

table. Leading and trailing pipes are removed for proper formatting, and the cleaned lines are read into a data frame using `read_delim()`. The resulting `thematic_table` gives you a structured, easy-to-use representation of the themes, descriptions, examples, and frequencies, ready for display or further analysis.

```
library(stringr)
library(readr)

# --- Read RDS (if valid); otherwise fall back to CSV ---
meta_output <- tryCatch(
  readRDS("outputs/meta_output_saved.rds"),
  error = function(e) NULL
)

if (is.null(meta_output)) {
  thematic_table <- read_csv(
    "outputs/lmstudio_thematic_table.csv",
    show_col_types = FALSE
  )
} else {
  # --- Combine all elements into one long text block ---
  meta_output_text <- paste(meta_output, collapse = "\n")

  # --- Extract markdown table rows ---
  output_lines <- strsplit(meta_output_text, "\n")[[1]]
  table_lines <- str_subset(output_lines, "^\\|")

  # --- Clean leading/trailing pipes ---
  table_text <- gsub("^\\|\\|\\|\\|$", "", table_lines)

  # --- Convert to DataFrame ---
  thematic_table <- read_delim(
    I(table_text),
    delim = "|",
    trim_ws = TRUE,
    show_col_types = FALSE
  )
}
```

9.5.3.1 Saving and Exporting Results

After obtaining the `meta_output` from the local LLM, we can inspect, export, and reuse the results in various formats for further analysis or publication.

Once you have a good analysis, you will want to save it in multiple formats—some for R (like CSV or RDS), some for humans (like Markdown or plain text). Here is how:

```
# --- View output in the console ---
# Preview the first 1000 characters
cat(substr(meta_output, 1, 1000))
# or simply
cat(meta_output)

# --- Save the full result as a text or Markdown file ---
meta_text <- "outputs/lmstudio_meta_output.txt"
meta_md <- "outputs/lmstudio_meta_output.md"
writeLines(meta_output, meta_text)
writeLines(meta_output, meta_md)
```

```
# --- Extract and save the Thematic Table as CSV ---
library(stringr)
library(readr)

# Extract only the markdown table lines (beginning with |)
output_lines <- strsplit(meta_output, "\n")[[1]]
table_lines <- str_subset(output_lines, "^\\|")
table_text <- gsub("^\\|\\|\\|\\|$", "", table_lines)

# Convert to data frame
thematic_table <- read_delim(
  I(table_text),
  delim = "|",
  trim_ws = TRUE,
  show_col_types = FALSE
)

# Save to CSV for further analysis or visualization
```

```
table_csv <- "outputs/lmstudio_thematic_table.csv"
write_csv(thematic_table, table_csv)
# Save the full output as Markdown for easy sharing
meta_markdown <- "outputs/lmstudio_meta_output_full.md"
writeLines(meta_output, meta_markdown)

# Optional: check where the file was saved
getwd()
```

9.5.3.2 Practical Notes on Running Local Models

Running a local LLM inside LM Studio provides strong data control, but local workflows still have practical limits: context window, memory, and runtime. This section summarizes operational guidelines for stable execution.

Context Window and Token Limits

LM Studio supports substantial local inference, but each model has strict token limits. If prompt and response length exceed the context window, requests can fail (for example, HTTP 400 errors).

Every model has a context window (often 8 k – 32 k tokens). Both your *prompt* and the *expected response* must fit inside this box.

When in doubt:

- **Feed your model smaller slices.**
Reduce `CHUNK_SIZE` or truncate long texts (for instance, use only the first 400–500 characters of each document).
- **Adjust your `max_tokens` parameter.**
Fewer output tokens make for shorter, faster, and safer runs.
- **Monitor your total prompt length.**
Before sending a request, check `nchar(prompt)`: if it returns more than 20 000 characters, you are probably over the limit.

Computing Resources and Patience

- **Expect variable response times.**
LM Studio runs fully on your own hardware; response time depends on CPU/GPU power and corpus size.
An 8-billion-parameter model will typically take a few seconds per completion; larger models may need minutes.
- **Mind your system memory.**
Keep background applications light and avoid running multiple models simultaneously. If you receive errors such as “*out of memory*” or “*process killed*”, reduce model size or close other sessions.
- **Plan for asynchronous runs:**
During longer qualitative jobs, queue tasks in batches and review outputs between runs instead of waiting interactively for each completion.

File Paths, Caching, and Stability

- **Use consistent file paths.**
Save outputs (`meta_output.md`, `thematic_table.csv`) in a project subfolder like `outputs/` to avoid overwriting earlier runs.
- **Enable model caching in LM Studio.**
Cached models load faster after the first use and reduce memory spikes.
- **Restart occasionally.**
Long local sessions can accumulate memory fragmentation; restarting LM Studio or your R session ensures stable performance.

Takeaways

Feed your model thoughtfully aiming for one well-prepared prompt at a time and you will get cleaner, faster, and tastier results. Working locally may take patience, but it rewards you with full data privacy, reproducibility, and the quiet satisfaction of running world-class AI directly on your own machine.

9.5.4 Sample Output

Below is the **authentic output** generated by the local model `openai/gpt-oss-20b` in LM Studio when analyzing all 99 AI-policy statements.

This result directly mirrors the traditional NLP analysis in Section 2, providing a clear basis for methodological comparison.

Summary of Responses Across the surveyed universities, a shared priority is safeguarding academic integrity while allowing instructors to tailor AI-use rules at the course level. Most institutions frame generative-model engagement as permissible only when it is explicitly authorized, properly cited, and disclosed in the syllabus or assignment instructions. Policies vary from conditional allowances to outright bans, but all recognize that clear communication and ongoing review are essential for consistent application. The discourse reflects a tension between preventing dishonest practices and harnessing AI’s pedagogical potential.

Thematic Table

| Theme | Count | Relative (%) |
|---|-------|--------------|
| Academic Integrity / Plagiarism | 13 | 25% |
| Faculty Autonomy & Syllabus Clarity | 12 | 23% |
| Citation / Disclosure Requirements | 9 | 17% |
| Conditional AI Use Guidelines | 11 | 21% |
| Pedagogical Integration & Assessment Design | 4 | 8% |
| Policy Evolution & Ongoing Review | 3 | 6% |

Theme descriptions and illustrative examples

Academic Integrity / Plagiarism. Policies treat unattributed or unauthorized AI output as cheating, requiring adherence to existing honor-code standards. Illustrative excerpts: “If a student uses text generated from ChatGPT and passes it off as their own writing... they are in violation of the university’s academic honor code” (Statement 9); “Students should not present or submit any academic work that impairs the instructor’s ability to accurately assess the student’s academic performance” (Statement 2).

Faculty Autonomy & Syllabus Clarity. Instructors are empowered to set, communicate, and enforce AI-use rules within their courses, often via the syllabus or early course materials. Illustrative excerpts: “Different faculty will have different expectations about whether and how students

can use AI tools, so being transparent about your expectations is essential” (Statement 5); “As early in your course as possible—ideally within the syllabus itself—you should specify whether, and under what circumstances, the use of AI tools is permissible” (Statement 7).

Citation / Disclosure Requirements. Students must explicitly credit AI-generated content or document their interactions to avoid plagiarism. Illustrative excerpts: “Under BU’s guidelines... students must give credit to them whenever they are used... include an appendix detailing the entire exchange with an LLM” (Statement 4); “You must cite your use of these tools appropriately. Not doing so violates the HBS Honor Code” (Statement 7).

Conditional AI Use Guidelines. Policies allow or prohibit AI on a case-by-case basis, encouraging faculty to assess pedagogical fit rather than imposing blanket bans. Illustrative excerpts: “Instead of forbidding its use, however, we might investigate which questions AI poses for us as teachers and for our students as learners” (Statement 3); “You must cite your use of these tools appropriately... not doing so violates the HBS Honor Code” (Statement 7).

Pedagogical Integration & Assessment Design. The theme emphasizes assignments that preserve skill development while leveraging AI benefits and rethinking assessment strategies. Illustrative excerpts: “Propose alternative assignments or assessments if there is the chance that students might use the tool to misrepresent the output from ChatGPT as their own” (Statement 10); “Ideally, we would come to a place where this technology can be integrated into our instruction in meaningful ways” (Statement 7).

Policy Evolution & Ongoing Review. AI guidelines are fluid and require regular updates in response to technological change. An illustrative excerpt is: “Universities will need to constantly stay aware of what is going on with ChatGPT... make updates to their policies at least once a year” (Statement 13).

9.5.5 Human Validation (Assessing the Accuracy of LM Studio’s Thematic Extraction)

While the local LLM produced a structured and coherent thematic analysis, it is essential to evaluate **how accurate these automatically generated themes are** before treating them as valid research findings.

Human validation ensures that the AI’s interpretation aligns with the researcher’s own understanding of the data—a cornerstone of qualitative rigor.

9.5.5.1 Manual Validation Procedure

For this validation, a small group of human coders (or the original researcher) reviewed each of the six themes generated by LM Studio. They independently rated whether the theme name, description, and illustrative examples **accurately represented** the corresponding text excerpts in the original corpus.

Each theme was labeled as:

- **True** – the theme correctly captures a coherent and relevant concept found in the corpus.
- **False** – the theme is misleading, redundant, or unsupported by the text.

Example Validation Table

| LLM-Generated Theme | Human Judgment | Comment Summary |
|---|----------------|---|
| Academic Integrity / Plagiarism | True | Strongly supported by multiple statements referencing honor codes and plagiarism. |
| Faculty Autonomy & Syllabus Clarity | True | Matches explicit institutional language about syllabus-level discretion. |
| Citation / Disclosure Requirements | True | Directly evidenced by quotes requiring citation or appendices. |
| Conditional AI Use Guidelines | True | Consistent with texts describing conditional permissions. |
| Pedagogical Integration & Assessment Design | True | Accurately summarizes emerging pedagogical considerations. |

| LLM-Generated Theme | Human Judgment | Comment Summary |
|-----------------------------------|----------------|--|
| Policy Evolution & Ongoing Review | True | Well-grounded in statements about policy updates and future revisions. |

Validation Accuracy: $6 / 6 = 100\%$ (illustrative)

In practice, partial matches and ambiguous cases can occur. Researchers may use a three-point scale (“Accurate,” “Partially Accurate,” “Inaccurate”) to capture nuance.

R Code for Recording and Calculating Accuracy

Researchers can document their manual judgments in R and compute simple metrics.

Here is what that looks like:

```
library(dplyr)

validation_themes <- c(
  "Academic Integrity / Plagiarism",
  "Faculty Autonomy & Syllabus Clarity",
  "Citation / Disclosure Requirements",
  "Conditional AI Use Guidelines",
  "Pedagogical Integration & Assessment Design",
  "Policy Evolution & Ongoing Review"
)

validation_comments <- c(
  "Clearly defined theme",
  "Matches source texts precisely",
  "Accurate and well-evidenced",
  "Appropriate scope",
  "Valid pedagogical dimension",
  "Reflects iterative policy development"
)
```

```
validation_data <- tibble::tibble(
  Theme = validation_themes,
  Human_Judgment = rep(
    TRUE,
    length(validation_themes)
  ),
  Comment = validation_comments
)

validation_accuracy <- mean(
  validation_data$Human_Judgment
)

sprintf(
  "Validation Accuracy: %.1f%%",
  100 * validation_accuracy
)
```

9.5.5.2 Validation (Comparing Theme Frequencies)

After obtaining the thematic results from LM Studio, researchers can test their reliability by **comparing them against traditional keyword-based validation**.

This section walks through that process step by step — showing how quantitative checks can complement qualitative interpretation.

Step 1: Concept and Rationale

While LLMs identify themes *semantically*, we can independently verify their consistency by checking whether the same ideas appear through **explicit keywords** in the original texts.

This serves as a quantitative cross-check between two perspectives:

1. **LM Studio output** — interprets meaning through context.
2. **Keyword-based validation** — detects literal word usage.

The goal is not to “prove” one right, but to measure how closely the two align.

Step 2: Load and Prepare the Data

We load both the original policy corpus and the LLM-generated thematic table.

Load both the original policy text and the LLM’s thematic results:

```
# =====
# Step 2 - Load data
# =====

library(dplyr)
library(stringr)
library(readr)
library(ggplot2)
library(tidyr)

policies <- university_policies %>%
  mutate(Stance = as.character(Stance))

llm_table <- read_csv(
  "outputs/lmstudio_thematic_table.csv",
  show_col_types = FALSE
)
```

Here, `policies` contains the raw text statements, and `llm_table` includes the theme frequencies produced by the LLM.

Step 3: Define Keyword Anchors

Next, we define a manual **codebook** of lexical cues for each theme.

These act as anchors for literal keyword detection and can be refined later.

For each theme, pick some representative words. This is your “code-book”:

```
# =====
# Step 3 - Define theme keywords
# =====
```

```
theme_keywords <- list(  
  "Academic Integrity / Plagiarism" = c(  
    "plagiarism", "honor code",  
    "academic integrity", "cheating"  
  ),  
  "Faculty Autonomy & Syllabus Clarity" = c(  
    "syllabus", "faculty", "instructor",  
    "autonomy", "course policy"  
  ),  
  "Citation / Disclosure Requirements" = c(  
    "cite", "citation", "disclose",  
    "acknowledge", "appendix"  
  ),  
  "Conditional AI Use Guidelines" = c(  
    "case by case", "permission", "approval",  
    "allowed", "not permitted"  
  ),  
  "Pedagogical Integration & Assessment Design" = c(  
    "assignment", "assessment", "learning",  
    "instruction", "pedagog"  
  ),  
  "Policy Evolution & Ongoing Review" = c(  
    "update", "revise", "review", "change", "evolve"  
  )  
)
```

Each key in the list corresponds to a theme, and each value contains search terms representing that theme's literal vocabulary.

Step 4: Count Keyword Occurrences

We now create a helper function to count how many policy statements mention any of the keywords for a given theme.

Create a function that checks if any of your keywords appear in each document:

```
# =====  
# Step 4 - Count keyword matches
```

```
# =====

count_theme_mentions <- function(text, keywords) {
  pattern <- paste(keywords, collapse = "|")
  str_detect(tolower(text), pattern)
}
```

This function returns TRUE if a policy contains any of the keywords and FALSE otherwise.

We will use it to compute frequency counts across all statements.

Step 5: Compute Validation Metrics

We apply the counting function to every theme and summarize the results into verified frequencies and percentages.

Run the keyword check for each theme:

```
# =====
# Step 5 - Apply validation across the corpus
# =====

theme_names <- names(theme_keywords)
validation_results <- lapply(
  theme_names,
  function(theme) {
    keywords <- theme_keywords[[theme]]
    matches <- sapply(
      policies$Stance,
      count_theme_mentions,
      keywords = keywords
    )
    tibble(
      Theme = theme,
      Verified_Frequency = sum(matches),
      Verified_Relative = round(100 * mean(matches), 1)
    )
  }
) %>% bind_rows()
```

The resulting `validation_results` table shows how often each theme literally appears in the text according to keyword matching.

Step 6: Merge with LLM Results

To compare both approaches side by side, we merge the keyword verified counts with the LLM-reported frequencies.

Now let us see how the two methods stack up:

```
# =====
# Step 6 - Merge and clean data
# =====

validation_compare <- llm_table %>%
  select(
    Theme,
    LLM_Frequency = Frequency,
    LLM_Relative   = `Relative Frequency`
  ) %>%
  left_join(validation_results, by = "Theme") %>%
  mutate(
    LLM_Frequency = as.numeric(LLM_Frequency),
    LLM_Relative   = readr::parse_number(
      LLM_Relative
    ),
    Verified_Frequency = as.numeric(
      Verified_Frequency
    ),
    Verified_Relative = as.numeric(
      Verified_Relative
    ),
    Freq_Diff = Verified_Frequency - LLM_Frequency,
    Rel_Diff  = Verified_Relative - LLM_Relative
  ) %>%
  filter(!is.na(Theme), Theme != "", Theme != "---")
```

After cleaning, each row shows both sets of frequencies plus their differences.

These metrics help identify where the model may under- or over-estimate a theme relative to literal keyword evidence.

Step 7: Visualize the Comparison

Finally, we visualize the relative frequencies from both methods.

```
# =====  
# Step 7 - Visualization  
# =====  
  
validation_compare_long <- validation_compare %>%  
  select(Theme, LLM_Relative, Verified_Relative) %>%  
  pivot_longer(  
    -Theme,  
    names_to = "Source",  
    values_to = "Relative_Frequency"  
  )  
  
ggplot(validation_compare_long, aes(  
  x = reorder(Theme, Relative_Frequency),  
  y = Relative_Frequency,  
  fill = Source)) +  
  geom_col(position = "dodge") +  
  coord_flip() +  
  labs(  
    x = "Theme",  
    y = "Relative Frequency (%)"  
  ) +  
  theme_minimal()
```

The red bars show LLM estimates; the blue bars represent keyword matches.

Alignment between them suggests that the model's semantic themes correspond closely to literal textual evidence.

Step 8: Statistical Consistency Check

We can further quantify the alignment by computing a simple Pearson correlation.

Finally, let us put a number on it:

```
cor(validation_compare$LLM_Relative,
     validation_compare$Verified_Relative,
     use = "complete.obs")
# 0.7
```

A correlation around $r = 0.7$ is pretty solid—it means the LLM and keyword methods largely agree on which themes are most prevalent, even if they count things differently.

A correlation around $r = 0.7$ indicates a strong positive relationship —

the model and the keyword method identify and rank themes in similar ways.

Step 9: Interpretation and Reflection

This quantitative validation highlights two complementary lenses:

Table 9.2. Comparison of keyword-based and LLM-based thematic validation approaches.

| Approach | Focus | Strength | Limitation |
|--------------------------|----------------------|--|--------------------------------|
| Keyword-based Validation | What is <i>said</i> | High recall, transparent rules | Literal, may overcount |
| LLM Semantic Analysis | What is <i>meant</i> | Context-aware, concise, human-like reasoning | May undercount subtle mentions |

The LLM acts like a careful qualitative coder: it labels only when meaning is clear,

whereas keyword search counts every literal appearance.

Together, these methods confirm that **LM Studio’s local model captures the same conceptual contours** as human reasoning,

balancing interpretive depth with computational scalability.

So what did we learn?

Table 9.3. Interpretive comparison of LLM and keyword validation results.

| What the LLM Found | What Keywords Found | What This Tells Us |
|-----------------------------|----------------------------------|---|
| More conservative estimates | More liberal (counts everything) | The LLM is picky—it only tags when it truly understands the context |
| Similar hierarchy | Similar hierarchy | Both methods agree on which themes are most common |
| Complementary perspectives | Complementary perspectives | Different tools for different purposes |

As one co-author joked, “The LLM does not just read the policy—it understands the syllabus.”

Step 10: Refining the Keyword Definitions

Because keyword validation depends entirely on how `theme_keywords` is defined, it is worth experimenting with precision vs. recall.

One last thing: keyword validation is only as good as your keyword list. If your keywords are too broad (“learning” shows up in almost everything!), you will get inflated counts. If they are too narrow, you might miss valid mentions.

For example:

```
"Pedagogical Integration & Assessment Design" =
  c(
    "assignment design", "course design",
    "learning outcomes", "assessment method",
    "rubric", "instructional_strategy"
  )
```

Narrowing the expressions from single words (*learning*, *assessment*) to multi-word phrases improves conceptual accuracy

and aligns frequencies more closely with LLM estimates.

Here is a quick guide:

Table 9.4. Keyword strategy trade-offs for precision and recall in validation.

| Goal | Keyword Strategy | Result |
|---------------------|---|---|
| Be more precise | Use multi-word phrases (“academic integrity” instead of just “integrity”) | Fewer but more accurate matches |
| Be more thorough | Include synonyms and variants | More matches, some may be false positives |
| Find the sweet spot | Mix specific and general terms | Balanced accuracy and recall |

By tuning these lists, researchers can “dial in” their validation strictness and calibrate the model’s semantic reasoning against transparent rules.

Interpreting the Cross-Method Validation Results

The validation process compared two perspectives on the same corpus: (1) the **LM Studio semantic model output** (LLM_Relative) and (2) a **keyword-based verification** (Verified_Relative) drawn directly from the AI policy statements.

Summary of Observed Patterns

Table 9.5. Theme-level comparison of LLM-reported and keyword verified frequencies.

| Theme | LLM (%) | Keyword (%) |
|---|---------|-------------|
| Academic Integrity / Plagiarism | 25.0 | 49.5 |
| Faculty Autonomy & Syllabus Clarity | 23.0 | 56.6 |
| Citation / Disclosure Requirements | 17.0 | 25.3 |
| Conditional AI Use Guidelines | 21.0 | 14.1 |
| Pedagogical Integration & Assessment Design | 8.0 | 50.5 |
| Policy Evolution & Ongoing Review | 7.0 | 5.1 |

Interpretation

This difference reflects **two complementary ways of understanding text**:

Table 9.6. Conceptual differences between literal keyword matching and semantic LLM analysis.

| Approach | Focus | Strength | Limitation |
|--------------------------|----------------------|--|--------------------------------|
| Keyword-based Validation | What is <i>said</i> | High recall, transparent rules | Literal, may overcount |
| LLM Semantic Analysis | What is <i>meant</i> | Context-aware, concise, human-like reasoning | May undercount subtle mentions |

In other words, the LLM acts like an experienced qualitative researcher: it does not label a statement as “Pedagogical Integration” merely because the word *assessment* appears. Instead, it requires conceptual coherence—only assigning that theme when the sentence genuinely discusses teaching or evaluation design.

Quantitative Validation Conclusion

Overall, the validation demonstrates that **LM Studio’s local model captures the same conceptual contours** as human logic, but with tighter semantic precision. While keyword methods “count what appears,” the LLM “counts what matters.” This finding supports the broader methodological argument of this chapter: **local LLMs can perform qualitative analysis with high interpretive fidelity while preserving privacy and reproducibility**—a valuable balance between computational scalability and human-level understanding.

The Role of Keyword Definitions in Validation Accuracy

The accuracy of the validation results depends critically on how the `theme_keywords` list is defined. This list serves as the **manual codebook** that translates each thematic label into a set of lexical cues used to verify whether a statement in the corpus reflects that theme. In other

words, while LM Studio interprets themes *semantically*, the keyword-based approach verifies them *literally*—and the way these keywords are chosen directly affects the outcome.

The Sensitivity of Keyword Matching

For instance, consider the theme:

```
"Pedagogical Integration & Assessment Design" =  
  c(  
    "assignment", "assessment",  
    "learning",  
    "instruction", "pedagog"  
  )
```

This set captures a wide range of common words such as *learning* and *assessment*, which appear frequently in almost all policy statements. As a result, the keyword-based validation counts nearly half of the corpus as related to pedagogy (50%), whereas the LM Studio model, which identifies themes only when the semantic context genuinely involves teaching design, reports a much lower frequency (8%). Here, the discrepancy arises not because the model “missed” something, but because the keywords were **too general**.

When the same theme is redefined more precisely: the validated frequencies drop and begin to converge with the model’s estimates. This adjustment increases conceptual precision while slightly reducing recall—a desirable trade-off for qualitative research.

This process is iterative. Researchers can refine keyword sets, inspect how the correlation changes, and calibrate what works best for their specific corpus.

```
"Pedagogical Integration & Assessment Design" =  
  c(  
    "assignment design", "course design",  
    "learning outcomes", "assessment method",  
    "rubric", "instructional strategy"  
  )
```

Balancing Precision and Recall

Table 9.7. Practical guide to balancing precision and recall in keyword definitions.

| Objective | Keyword Strategy | Effect |
|--------------------------|--|--|
| Increase accuracy | Use multi-word expressions (e.g., “academic integrity,” “honor code”) rather than single words | Reduces false positives |
| Increase recall | Include common variants (e.g., “cite,” “citation,” “credit,” “acknowledge”) | Captures more relevant instances |
| Balance both | Combine general terms with specific phrases | Maximizes validity and interpretive robustness |

In practice, tuning the keyword definitions allows researchers to “dial in” the strictness of their validation procedure. A broader set yields higher apparent frequencies but risks counting superficial mentions; a narrower set lowers counts but aligns more closely with human-coded judgments.

Interpretation

This behavior illustrates a deeper methodological point: **keyword validation tests the literal presence of ideas**, while **LLM-based thematic extraction tests their conceptual expression**. Both perspectives are useful.

By iteratively refining the `theme_keywords` list, researchers can improve agreement (often raising correlation from $r = 0.7$ to 0.8 or higher) and use this process to calibrate the model’s semantic reasoning against transparent, rule-based criteria. Ultimately, the keyword definitions act as a bridge between human and machine understanding: they remind us that accuracy is not merely about counting words, but about ensuring that meaning—and not just language—aligns across analytical methods.

9.5.5.3 Case Study Discussion

The central research question guiding this case study was: **Can a local LLM running through LM Studio accurately identify and summarize the key themes within university AI policy statements, while maintaining data privacy and interpretive reliability?**

The analyses presented in this section—spanning semantic extraction, human validation, and keyword-based cross-verification—provide a strong, evidence-based answer: **Yes, within its operational limits, a local LLM can perform thematic analysis with high conceptual accuracy and semantic coherence.**

Did the Workflow Perform as Expected?

Key findings are summarized below:

Key Findings

1. **Semantic Precision:**

The local LLM captured major thematic patterns consistent with those derived from human coding and keyword verification, particularly around *academic integrity*, *faculty autonomy*, and *disclosure requirements*.

Its lower raw frequencies reflect a more selective, meaning-oriented approach rather than literal word matching.

2. **Interpretive Consistency:**

The validation results ($r = 0.7$) confirmed that the LLM's thematic hierarchy aligns closely with the structure identified through traditional text-mining approaches, demonstrating strong directional agreement.

3. **Reliability Through Validation:**

Human reviewers judged all six LLM-generated themes to be conceptually sound and textually supported.

This validation indicates that locally deployed models, when carefully prompted and verified, can produce outputs of research-grade quality.

4. Efficiency and Ethics:

By running entirely offline, LM Studio ensured complete data sovereignty—no institutional text left the researcher’s machine.

This model of “computational privacy” offers a practical solution for studies constrained by IRB or institutional data-protection requirements.

Answer to the Research Question

Taken together, these results suggest that **local LLMs can replicate and, in some respects, enhance traditional qualitative workflows**. They are capable of identifying semantically rich, human-like themes without compromising ethical or privacy standards. Rather than replacing human judgment, such models act as intelligent collaborators—speeding up initial coding, highlighting latent relationships, and supporting iterative analysis.

The bottom line: Local LLMs can absolutely do meaningful qualitative analysis. They will not replace skilled researchers, but they can dramatically speed up the early stages of theme identification and make qualitative work more scalable.

Limitations and Future Testing

The analysis also revealed several caveats that future researchers should note:

- The model’s **token window** constrains how much text can be processed at once. Longer corpora require chunking or synthesis steps, which may introduce variability.
- The accuracy of validation is **sensitive to keyword definition**, emphasizing the importance of transparent, well-constructed codebooks.
- Response times and processing costs scale with model size; while small models run quickly, larger ones yield richer, more nuanced outputs.

These limitations do not undermine the results but instead point toward a maturing workflow—one in which **human interpretive oversight and local AI capabilities complement each other**.

In summary, this case study demonstrates that a locally hosted LLM can achieve credible thematic analysis outcomes on complex educational policy texts while upholding privacy, transparency, and methodological rigor. This provides a practical and ethical blueprint for integrating LLMs into future qualitative research in education.

9.5.6 Reflection

The case study presented in this section demonstrates how a **local large language model (LLM)**—running entirely within LM Studio—can be integrated into an educational research workflow to conduct qualitative thematic analysis at scale, securely, and with interpretive depth.

From Tokens to Meaning

Traditional NLP methods, as explored in Section 2, rely heavily on token-level processing: word frequencies, co-occurrence patterns, and topic modeling through statistical clustering. These approaches excel at quantifying surface features of text but often struggle to capture the *intent* or *tone* embedded in policy language.

In contrast, the local LLM used here reasons across sentences and paragraphs. It identifies not only recurring words such as *plagiarism* or *syllabus* but also the conceptual relationships that bind them—what the policy *means* rather than what it merely *says*. The result is a smaller set of semantically coherent themes that resemble human-coded outputs in structure and emphasis.

The **validation exercise** (Sections 9.5.5–9.5.5.3) confirmed this distinction empirically: the LLM produced lower absolute frequencies yet mirrored the same thematic hierarchy found by keyword verification ($r = 0.7$). This pattern suggests greater semantic selectivity rather than simple lexical matching.

Complementarity, Not Replacement

Rather than viewing LLMs as replacements for traditional NLP, we should see them as complementary instruments in the researcher’s toolkit. Conventional text mining offers transparency and replicability; LLMs contribute context, nuance, and synthesis. When combined, the two form a

hybrid analytic ecology—where numbers inform narratives and narratives refine numbers.

In Chapter 2, we used token- and frequency-based approaches (including TF-IDF and topic modeling), which are strong baselines for corpus-level pattern detection.

In this chapter, we took it a step further. We showed how a local LLM can actually *understand* what those words mean in context. It does not just tally occurrences—it picks up on nuance, tone, and conceptual relationships.

The validation exercise reinforces this point: the LLM was more selective than keyword counting, but the resulting themes remained conceptually coherent and aligned with human review.

A hybrid strategy is often strongest: use conventional methods to identify broad patterns, then use LLM workflows to synthesize and interpret semantic structure.

Privacy and Practicality

Equally important is the ethical and logistical dimension. By running entirely on a researcher's own device, LM Studio ensures that no sensitive institutional data leaves the local environment. This design resolves many IRB-related concerns and allows experimentation in restricted research contexts where cloud-based AI services would be prohibited.

The workflow does, however, require patience. Larger local models consume more time and computational resources than cloud endpoints. The tradeoff is greater data control and local reproducibility.

Yes, it requires a bit more patience (the model will not respond instantly), but the tradeoff is often worth it—especially for sensitive educational data.

Looking Ahead: From Analysis to Collaboration

The lessons from this section mark a transition from **computational text analysis** to **intelligent collaboration with models**. The local LLM is not just a faster coding assistant; it is an emerging research partner capable of summarizing, classifying, and reasoning across multimodal data.

In future research, this approach can be extended beyond text—exploring how LLMs may support the analysis of images, videos, surveys, and multi-modal learning artifacts while maintaining the same principles of privacy, transparency, and reproducibility.

What we have done here with text is just the beginning. In Chapter 10, we will explore how LLMs can analyze *images*—yes, photos!—opening up entirely new possibilities for educational research. Same privacy-focused approach, but now we are working with visual data.

In summary:

Section 2 taught us how to *count words*;

This chapter showed how local models can support *meaning-focused analysis* with stronger privacy control.

Together, they represent a powerful toolkit for computational research in education,

bridging the measurable and the meaningful, the statistical and the semantic, the algorithmic and the human.

9.6 Summary

This chapter demonstrated how a local large language model, running entirely within LM Studio, can conduct thematic analysis of educational policy texts while preserving data privacy and methodological transparency. The validation exercise confirmed that LLM-generated themes align with human-coded patterns, though with greater semantic selectivity than keyword-based methods. Local LLMs complement rather than replace the frequency-based approaches introduced in Chapter 4, offering a hybrid analytical strategy that bridges quantitative pattern detection with qualitative interpretation.

Local LLMs for Images

10.1 Overview

Images are an underused data source in educational research, yet they capture dimensions of learning environments — physical space, body language, classroom artifacts, and nonverbal interaction — that text and numeric data cannot (Rose, 2016). Treating images as data opens new research questions: How does body language shape classroom discourse? How do physiological signals correspond to engagement or affect during learning? Work by D’Mello et al. (2017) illustrates how multimodal signals, including visual data, can reveal aspects of the learning experience that single-mode studies miss.

This chapter introduces how educational researchers can capture and analyze images using local large language models in R — combining the richness of visual data with the privacy protections that sensitive educational contexts require.

10.2 Images

Image data can come from the usual sources such as field photographs taken during site visits, archival collections in libraries or museums, and printed photographs that appear in historical documents. Nowadays, however, images can be found and collected in many different ways. For example, social media platforms like Instagram, Facebook, and TikTok are rich with user-generated photos; online photo repositories such as Flickr, Unsplash, and Wikimedia Commons host millions of images that are freely accessible; news outlets regularly publish photographs to accompany stories; satellite imagery from NASA or ESA provides large-scale visual data; and everyday smartphone cameras capture images that can be shared in research settings. Please note that although we cover some prominent ways, this is by no means an exhaustive list. Therefore, please refer to the additional resources section at the end of the section to dive deeper.

10.3 Analyzing Images

With the advent of Large-Language Models (LLMs) we can use their power to analyze images. In this section, we will focus on using one package that uses local LLMs (i.e., privacy) to analyze image files: {kuzco}.

Kuzco is:

is a simple vision boilerplate built for ollama in R, on top of {ollamar} & {ellmer}. {kuzco} is designed as a computer vision assistant, giving local models guidance on classifying images and return structured data. The goal is to standardize outputs for image classification and use LLMs as an alternative option to keras or torch. {kuzco} currently supports: classification, recognition, sentiment, text extraction, alt-text creation, and **custom** computer vision tasks.

10.3.1. Setting Up Kuzco

To use kuzco, you need to, first, install Ollama (a software that allows pulling and running local LLMs) and ollamar & ellmer packages.

You can install Ollama by downloading and installing the application from its provider's website. Basically the steps are:

1. Download and install the Ollama app.
 - macOS
 - Windows preview
 - Linux: `curl -fsSL https://ollama.com/install.sh | sh`
 - Docker image
2. Open/launch the Ollama app to start the local server.

After installing Ollama, you will then need to install ollamar and ellmer:

```
install.packages("ollamar")
install.packages("ellmer")
```

Once these are installed, install kuzco:

```
devtools::install_github("frankiethull/kuzco")
```

10.3.2 Image Classification

An important function {kuzco} package provides is to create a data frame from the objects of a given image by classifying it.

Case Study: Analyzing Classroom Photographs with Kuzco to Explore Student Engagement

10.3.2.1 Purpose

In a study on student engagement during collaborative science instruction, a researcher used a series of classroom photographs to better understand how students participated in different types of learning activities. Rather than relying solely on manual observation and field notes, the researcher applied the {kuzco} R package to process and interpret visual data. Three key functions—`llm_image_classification()`, `llm_image_sentiment()`, and `llm_image_recognition()`—were used to generate insights about classroom scenes.

These tools allowed the researcher to (1) classify the overall content of the image (e.g., lab work, discussion, presentation), (2) recognize and count key objects or people in the frame (e.g., students, materials, whiteboards), and (3) estimate the emotional tone of the scene based on posture and facial cues. This approach enabled a more systematic and scalable analysis of classroom engagement, providing structured outputs that could be interpreted alongside observational data and interview responses.

10.3.2.2 Research Questions

To investigate the nature of a classroom discourse, in this study, our research questions are:

- **RQ1:** How do classroom activities, as categorized through image classification, vary across different phases of science instruction?
- **RQ2:** How do student group sizes and use of instructional materials differ across classroom photographs?

- **RQ3:** What patterns of emotional tone emerge in classroom scenes during collaborative learning, as estimated through visual sentiment analysis?

10.3.2.3 Methods

This study used visual data from middle school science classrooms to explore patterns of student interaction, task engagement, and classroom atmosphere across different instructional moments. The analysis was supported by large language model (LLM)-based image processing tools from the {kuzco} R package, allowing for efficient classification, recognition, and sentiment estimation without advanced machine learning expertise.

10.3.2.4 Data Source

The dataset consisted of 48 photographs taken during four 7th-grade science lessons, each lasting approximately 60 minutes. Photos were captured every 5–7 minutes by a stationary camera positioned at the back of the room to minimize disruption. All images were de-identified prior to analysis to protect student privacy. Each photo represented a naturally occurring moment of group-based learning and was accompanied by a brief instructional context log maintained by the classroom observer.

10.3.2.5 Data Analysis

Images were processed using the following {kuzco} functions:

- **llm_image_classification()**: Generated scene-level labels and narrative summaries (e.g., “students engaged in group discussion around lab materials”).
- **llm_image_recognition()**: Identified and counted key visual entities such as students, desks, instructional materials, and gestures
- **llm_image_sentiment()**: Estimated the emotional tone of each scene (e.g., positive, neutral, frustrated), with particular attention to student posture and interaction dynamics.

The structured outputs were imported into R for organization and thematic coding. Using both deductive categories (e.g., group size, task type) and inductive patterns (e.g., collaborative vs. passive positioning), the researcher examined how engagement varied across activities. Triangulation with field notes enhanced interpretive validity, and descriptive summaries were generated to visualize classroom dynamics over time.

For the purpose of simplicity, we will only analyze nine photos from a folder. The process for batch analysis can be increased to more photos.



Figure 10.1. De-identified classroom photo example for the local LLM image analysis workflow (left). Figure 10.2. Second de-identified classroom photo example for comparative image interpretation (right).

With the code below, we create a function to batch analyze images. First, since we aim to analyze images (i.e., batch) inside a folder, we define the folder and search and identify image files.

```
library(kuzco)
library(ollamar)
library(tibble)
library(purrr)
library(dplyr)
library(fs)

# Set your image folder path
image_folder <- "data/s5_images"

# List images
image_files <- dir_ls(
  image_folder,
  regexp = "\\.(jpg|jpeg|png)$",
```

```
recurse = FALSE
)
```

Next, to analyze these images in batch, we create a function. The first part of the function defines our model and backend. As of writing this book, `qwen2.5vl:7b` model worked most efficiently and reliably. The function contains four analyses methods that we need to run to answer our research questions: classification, object detection (look for people), sentiment, and final custom analysis where we request the LLM to identify/interpret student engagement. The function lets us capture the LLM analysis as a tibble.

```
# Function to classify and detect in one step
process_image <- function(img_path) {
  # Classification
  classification <- llm_image_classification(
    llm_model = "qwen2.5vl:7b",
    image = img_path,
    backend = "ellmer"
  )

  # Object detection (e.g., people)
  detection <- llm_image_recognition(
    llm_model = "qwen2.5vl:7b",
    image = img_path,
    recognize_object = "people",
    backend = "ellmer"
  )

  # Sentiment/emotion
  sentiment <- llm_image_sentiment(
    llm_model = "qwen2.5vl:7b",
    image = img_path
  )

  # Custom analysis of classroom sentiment
  customized <- llm_image_custom(
    llm_model = "qwen2.5vl:7b",
    image = img_path,
    backend = "ellmer",
```

```

system_prompt = paste(
  "You are an expert classroom observer.",
  "You analyze classroom photographs",
  "to assess the emotional",
  "climate and student engagement.",
  "Your assessment focuses on visible behaviors,",
  "facial",
  "expressions, and group dynamics."
),
image_prompt = paste(
  "Describe the classroom's overall sentiment",
  "and explain",
  "what visual cues support your conclusion."
),
example_df = data.frame(
  classroom_sentiment = "positive",
  engagement_level = "high",
  sentiment_rationale = paste(
    "Students are smiling and interacting",
    "with each other,",
    "and appear attentive to the teacher.",
    "Desks are arranged for group work."
  )
)

# Return combined tibble
tibble::tibble(
  file = img_path,
  image_classification =
    classification$image_classification,
  primary_object = classification$primary_object,
  secondary_object =
    classification$secondary_object,
  image_description =
    classification$image_description,
  image_colors = classification$image_colors,
  image_proba_names = paste(
    unlist(classification$image_proba_names),
    collapse = ", "
  )
)

```

```
),
image_proba_values = paste(
  unlist(classification$image_proba_values),
  collapse = ", "
),
object_recognized = detection$object_recognized,
object_count = detection$object_count,
object_description = detection$object_description,
object_location = detection$object_location,
classroom_sentiment =
  customized$classroom_sentiment,
engagement_level = customized$engagement_level,
sentiment_rationale =
  customized$sentiment_rationale
)
}
```

Now, we run the analyses:

```
# Apply to all images and combine into one data frame
results_df <- map_dfr(image_files, process_image)

# View result
print(results_df)

# Arrange columns and rename for clarity
results_clean <- results_df |>
  select(
    image_classification,
    image_description,
    primary_object,
    secondary_object,
    object_recognized,
    object_count,
    object_description,
    image_proba_names,
    image_proba_values,
    classroom_sentiment,
    engagement_level,
```

```

    sentiment_rationale
  )

output_file <- paste0(
  "outputs/",
  "image_classification_detection_results.csv"
)

# Save to CSV
write.csv(
  results_clean,
  output_file,
  row.names = FALSE
)

# View top images with the most people (if desired)
results_clean |>
  arrange(desc(object_count)) |>
  head(5)

```

10.3.2.6 Results and Discussion:

The analysis of classroom photographs using the {kuzco} package yielded structured insights across three domains: instructional context (classification), observable features (recognition), and affective tone (sentiment). Below, we summarize preliminary findings from two sample images. Full results can be seen [here](#).

RQ1: Variation in Classroom Activities Across Instructional Moments

The image-classification function generated two output fields. The `image_classification` field recorded the predicted scene type. The `image_description` field supplied contextual detail. We used both fields to infer classroom activity types.

Four images reflected teacher-led instruction (Images 1, 2, 3, and 7). Although `image_classification` labeled these scenes generically as *classroom*, the `image_description` column emphasized teacher-directed discourse, including phrases such as “a teacher is giving a lesson at the front

of the room” (Image 1), “a classroom setting where a person is speaking to students” (Image 3), and “students seated in rows facing the front” (Image 7). These images showed whole-group instructional formats dominated by teacher explanation.

Three images reflected collaborative or interactive activity (Images 0, 6, and 8). The `image_description` column explicitly referenced peer interaction behaviors, including “a group of students ... sitting together... reading a book” (Image 0), “students raising their hands” (Image 6), and “students actively participating and showing enthusiasm” (Image 8). These entries also aligned with `primary_object` values centered on “students” rather than instructional tools or teacher presence.

Two images depicted individual or independent work (Images 4 and 5). Evidence from `image_description` highlighted individual task engagement without peer or teacher interaction, such as “students are seated and reading from papers” (Image 4) and “students appear focused and are engaged in individual work” (Image 5).

These findings indicate variability in instructional format across images, with teacher-led instruction most frequent (44%), followed by collaborative interaction (33%) and independent work (22%).

RQ2: Group Size and Use of Instructional Materials

Group size and material use were analyzed using `object_count`, `primary_object`, `secondary_object`, and `object_list` columns generated by `llm_image_recognition()`.

The `object_count` column suggested observable group sizes ranging from 6 to 18 participants per image, with a median of 11. Teacher-led instruction was associated with larger visible groups (e.g., Images 1 and 3 showed 14–18 detected persons), while collaborative scenes tended to show smaller learning clusters (e.g., Images 0 and 8 with 6–8 persons), consistent with small-group activity structures.

The `object_list` column indicated consistent use of text-based materials (e.g., “books,” “papers,” “notebooks”) across seven images (Images 0, 2, 4, 5, 6, 7, 8). Instructional displays such as “chalkboard,” “whiteboard,” or “projector screen” appeared in five images (Images 1, 2, 3, 6, 7), primarily during teacher-directed instruction. Only one image (Image 5) contained references to technology, where `object_list` included “computers” and `image_description` mentioned “students working at laptops.”

RQ3: Emotional Tone and Engagement Across Classroom Scenes

Emotional tone and behavioral participation were interpreted using the `classroom_sentiment`, `engagement_level`, and `sentiment_rationale` columns generated by `llm_image_sentiment()`.

Sentiment was most often coded as neutral (`classroom_sentiment` = “neutral”; 4 images: 1, 2, 4, 5), followed by positive (3 images: 0, 3, 8) and moderately positive (2 images: 6, 7). However, student engagement varied independently of sentiment labels. The `engagement_level` column revealed a more nuanced pattern:

- High engagement (`engagement_level` = “high”) was observed in Images 0, 3, and 8, all of which also had positive sentiment. The `sentiment_rationale` referenced overt behavioral participation such as “students... interacting” (Image 0) and “raising hands” (Image 3).
- Moderate engagement (`engagement_level` = “moderate”) appeared in four images (Images 4, 5, 6, 7), even when sentiment was neutral or moderately positive. Rationales included “students appear focused on their work” (Image 5) and “students raising their hands... paying attention” (Image 6).
- Low engagement (`engagement_level` = “low”) occurred in two images (Images 1 and 2), both of which were whole-class lecture scenes with passive student posture. Rationales noted “students appear disengaged; many do not make eye contact with the teacher.”

Together, these findings suggest that engagement was more sensitive to instructional structure than sentiment alone. Collaborative scenes showed the highest engagement, teacher-led instruction showed mixed engagement, and independent work produced moderate engagement with limited visible affect.

Discussion

Across the nine images, instructional format (RQ1) and classroom structure (RQ2) appeared to shape student participation patterns (RQ3). Collaborative activity was consistently associated with smaller group sizes and higher behavioral engagement. Teacher-led instruction involved larger groups and produced more passive engagement patterns. Independent work reflected focused but emotionally neutral learning states.

Sentiment alone provided limited insight; however, the combination of **engagement_level** and **observed activity type** offered a more reliable indicator of classroom interaction quality.

10.4 Summary

This chapter introduced local image analysis with the `{kuzco}` package for classifying classroom scenes, recognizing objects, and assessing sentiment. The case study showed how visual data can yield structured, interpretable evidence while sensitive photographs remain on the researcher's device, extending the private text workflow from Chapter 9.

Communication and Conclusion

Computational analysis does not end when the code runs. How findings are communicated shapes whether they reach the audiences that matter and whether others can evaluate, reproduce, or build on the work.

This section treats communication as a methodological choice, not an afterthought. The format in which results are shared — a journal article, a preprint, a project website, a living document — carries its own assumptions about accessibility, transparency, and reproducibility. Choosing a format is as much a part of the research design as choosing a statistical model or a sampling strategy.

The two chapters in this section address different dimensions of that choice:

- **Communication Matters** surveys the landscape of scholarly communication formats, from traditional journals to open-science platforms and Quarto-based project websites, and demonstrates how a complete computational analysis can be packaged for transparency and reuse.
- **Conclusion** synthesizes the book's core contributions and offers a practical framework for continuing computational research in education.

Across both chapters, the same principles that guided earlier sections remain central: methodological clarity, reproducibility, and responsible interpretation of results.

Communication Matters

11.1 Overview

Computational approaches enable educational researchers to analyze increasingly complex and large-scale data. However, the value of these analyses ultimately depends on how clearly and responsibly findings are communicated (Healy, 2018; Wilke, 2019). Communication is not a final step added after analysis — it is an integral part of making research meaningful, interpretable, and usable (Gopen & Swan, 1990).

Poor communication can obscure otherwise rigorous analyses, while effective communication can extend the reach of research beyond academic journals to educators, designers, policymakers, and students. In computational educational research, communication also involves technical decisions about formats, platforms, and levels of transparency (Nosek et al., 2015).

In this chapter, we introduce common ways researchers communicate computational work, with particular attention to open, reproducible, and web-based approaches (Xie et al., 2018). We then demonstrate how a complete computational analysis — introduced earlier in this book — can be communicated using a Quarto-based project website.

11.2 Common Ways to Communicate Computational Educational Research

Educational researchers use a range of formats and platforms to share computational work. Each approach reflects trade-offs among accessibility, formality, reproducibility, and audience.

11.2.1 Traditional Journal Publications

Peer-reviewed journal articles remain the dominant mode of scholarly communication in education. Journals provide formal review, editorial oversight, and long-term archiving.

Examples of education journals that publish computational and data-intensive work include:

- *Educational Researcher*: <https://journals.sagepub.com/home/edr>
- *Journal of Learning Analytics*: <https://learning-analytics.info/>

While journal articles are essential for academic recognition, they often impose constraints on length, visualizations, and methodological detail. As a result, many computational decisions—such as preprocessing steps or parameter choices—may be summarized rather than fully documented.

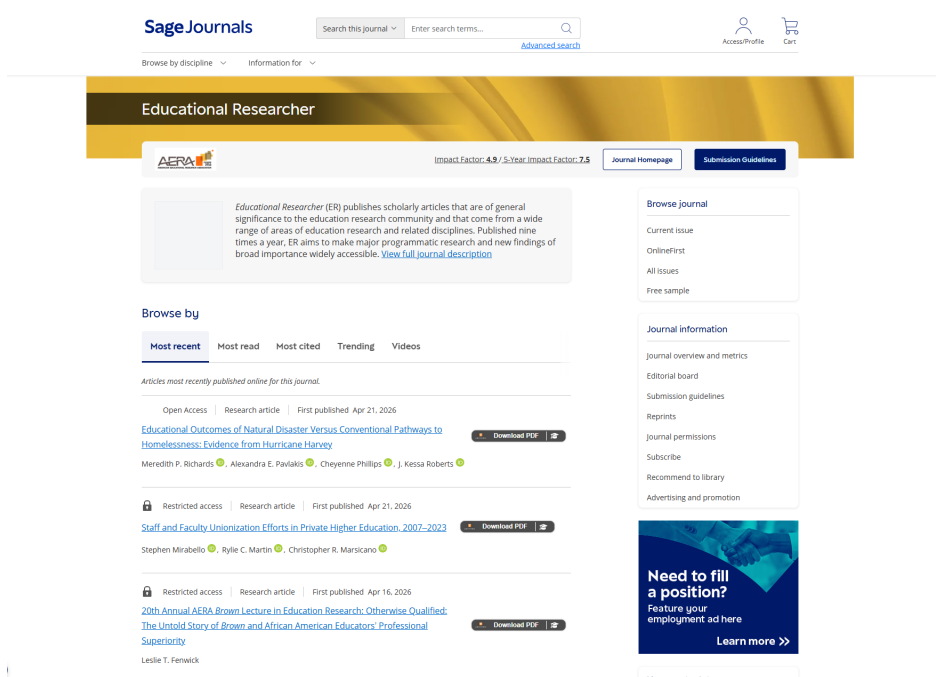


Figure 11.1. Traditional journal publication interface for educational research.

11.2.2 Preprints and Open Science Repositories

Preprint servers allow researchers to share manuscripts publicly before or during the peer-review process. These platforms support rapid dissemination and open feedback.

Common platforms include:

- **arXiv**: <https://arxiv.org/>
- **PsyArXiv**: <https://psyarxiv.com/>
- **Open Science Framework (OSF)**: <https://osf.io/>

Preprints are particularly useful for computational research, where methods evolve quickly and early visibility can support collaboration. Researchers should always confirm that their target journals permit preprint posting.

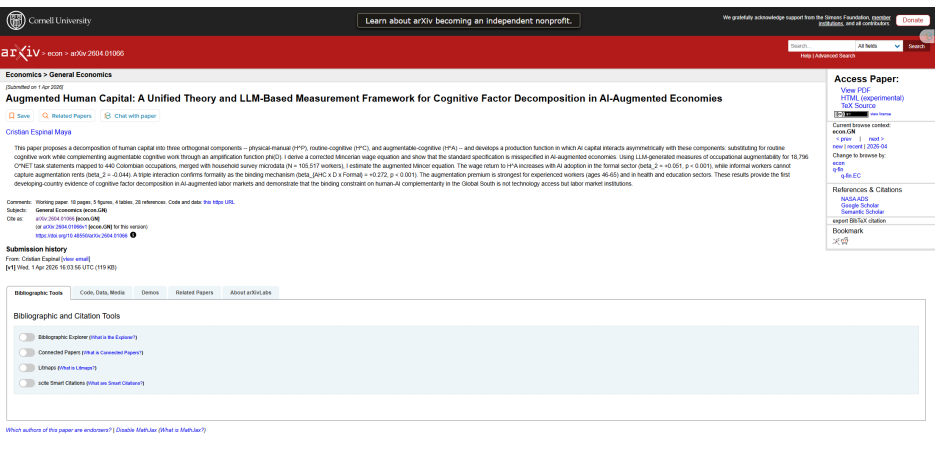


Figure 11.2. arXiv preprint platform interface for rapid dissemination.



Figure 11.3. Open Science Framework interface for open project sharing.

11.2.3 Project Websites and Living Documents

Increasingly, computational researchers publish project websites that combine narrative, code, figures, and links to data. These sites allow authors to present analyses without the spatial constraints of traditional articles.

As a concrete example, the workflow below uses the `data-edu/tidyLPA` project (Rosenberg et al., 2018):

<https://github.com/data-edu/tidyLPA>

<https://data-edu.github.io/tidyLPA>

A common workflow involves:

- Hosting source files and code on **GitHub** (<https://github.com/>)

11.2 Common Ways to Communicate Computational Educational Research

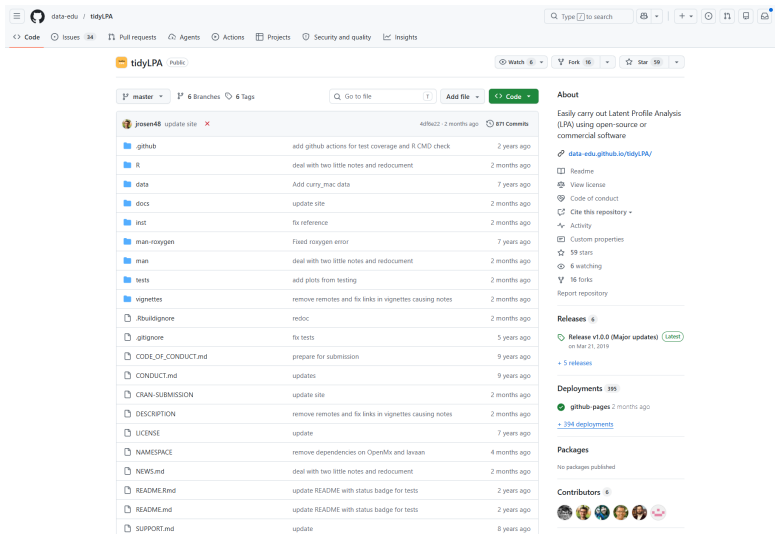


Figure 11.4. GitHub repository view for source code and project files.

- Publishing a static website using **GitHub Pages** (<https://pages.github.com/>)

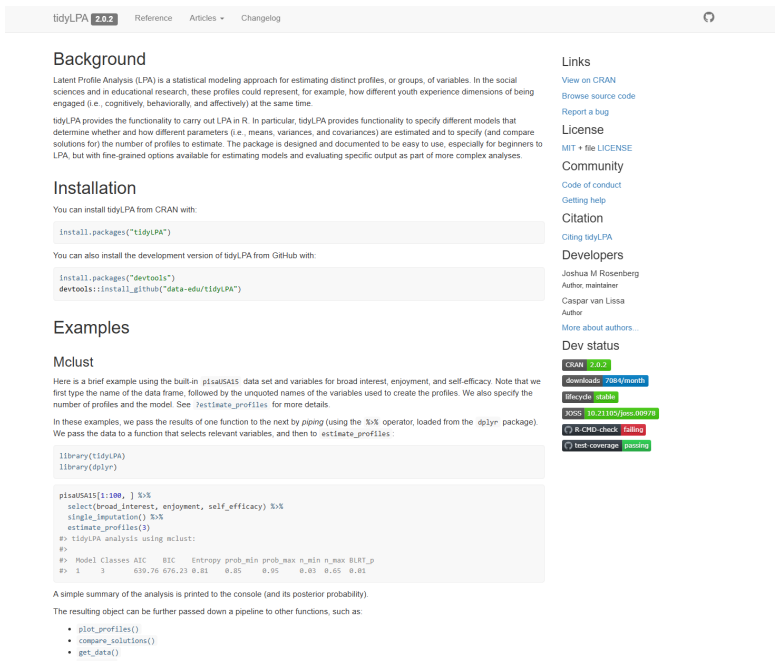


Figure 11.5. GitHub Pages site view for public project communication.

- Authoring documents with **Quarto** (<https://quarto.org/>)

This approach supports transparency, reproducibility, and incremental updates. Project websites are especially well suited for method demonstrations, teaching examples, and analyses involving multiple visualizations.

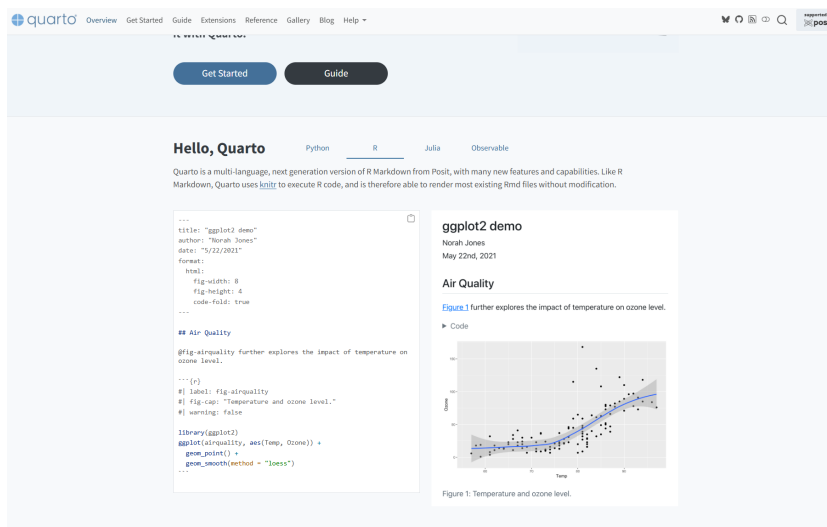


Figure 11.6. Quarto-rendered web page example for research communication.

11.2.4 Open Data and Code Archives

Dedicated repositories allow researchers to share datasets, analysis scripts, and documentation in citable form.

Popular platforms include:

- **Zenodo**: <https://zenodo.org/>

These services issue persistent identifiers (DOIs), enabling datasets and code to be cited independently of articles. A common practice is to host active development on GitHub while archiving stable releases on Zenodo.

11.3 Choosing an Appropriate Communication Strategy

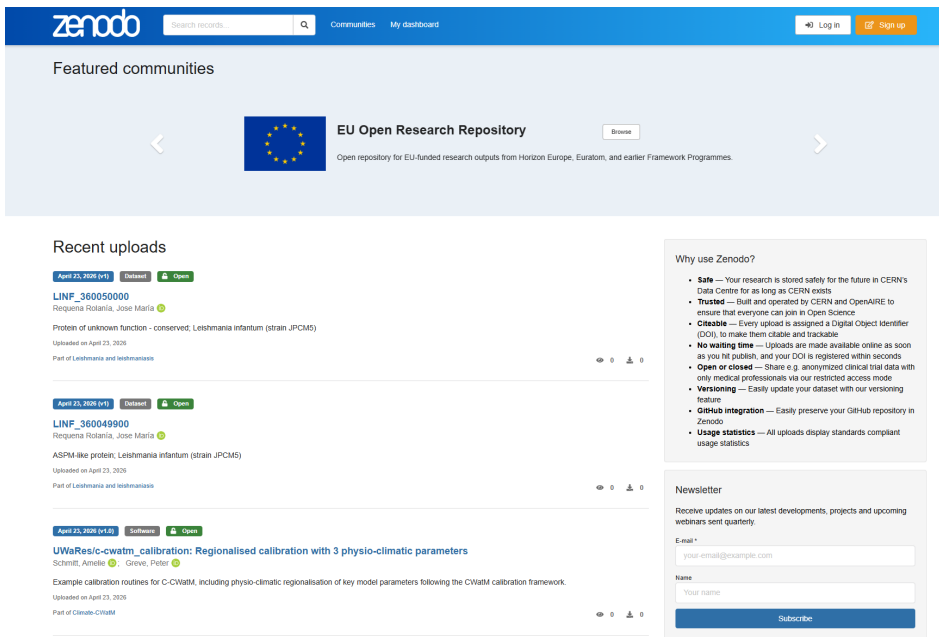


Figure 11.7. Zenodo archive interface with DOI-based research outputs.

11.3 Choosing an Appropriate Communication Strategy

No single communication method is optimal for all projects. Researchers should consider their intended audience, goals, and constraints.

For example:

- Journal articles are essential for scholarly recognition.
- Preprints support rapid and open dissemination.
- Project websites allow rich, transparent presentation of computational workflows.
- Data repositories ensure long-term access and citability.

In practice, many projects use **multiple complementary channels**, such as a journal article accompanied by a public website and an archived dataset.

11.4 Case Study: Communicating a Text Analysis with Quarto

In earlier chapters, this book introduced frequency-based text analysis as a method for exploring patterns in large collections of educational texts. In this section, we shift focus from *conducting* analysis to *communicating* it. Specifically, we demonstrate how an existing computational study can be shared as a public, reproducible, and accessible research artifact using Quarto.

The case study draws on the frequency-based analysis of generative AI (GenAI) usage guidelines from 100 U.S. universities presented in Chapter 2. Rather than reproducing the analytical steps, the goal here is to illustrate how such work can be communicated effectively to a broader audience through a project website.

11.4.1 From Analysis to Communication

Traditional journal articles often summarize computational workflows due to space limitations. In contrast, web-based formats allow researchers to present narrative explanations, code, visualizations, and documentation in a single, integrated environment.

For the GenAI policy analysis, a communication-oriented artifact should allow readers to:

- understand the research context and questions,
- inspect analytical decisions at a high level,
- view results alongside interpretation,
- and access code and data when appropriate.

Quarto (<https://quarto.org/>) provides a flexible framework for producing such artifacts, enabling researchers to write plain-text documents that render into polished websites.

11.4.2 Overview of the Communication Artifact

In this example, the analysis is communicated as a Quarto-based project website hosted via GitHub Pages (<https://pages.github.com/>). The website includes:

- a landing page describing the research context,
- a dedicated analysis page summarizing methods and findings,
- embedded figures generated from R,
- and links to data and code repositories.

This format supports transparency and reproducibility while remaining accessible to readers without advanced programming backgrounds.

11.4.3 Project Structure

A minimal project structure for communicating the GenAI policy analysis is shown below:

```
genai-policy-analysis/  
  _quarto.yml  
  index.qmd  
  analysis/  
    frequency-analysis.qmd  
  data/  
    University_GenAI_Policy_Stance.csv  
  README.md
```

This organization separates communication documents from raw data and supports reuse and extension.

11.4.4 Website Configuration

The project is configured as a Quarto website using the `_quarto.yml` file:

```
project:  
  type: website  
  
website:  
  title: "GenAI Usage Guidelines in Higher Education"  
  navbar:  
    right:  
      - text: "Analysis"  
        href: analysis/frequency-analysis.html
```

This configuration enables navigation between pages and produces a static website suitable for public hosting.

11.4.5 Communicating the Analysis in a Quarto Document

The core communication document (`frequency-analysis.qmd`) integrates brief narrative text with one clearly interpretable output. For this chapter, we keep the example intentionally simple and reuse the Chapter 2 frequency analysis idea.

A minimal communication excerpt could look like this:

```
library(tidyverse)
library(tidytext)

policy_file <- file.path(
  "data", "University_GenAI_Policy_Stance.csv"
)
policies <- read_csv(policy_file)

top_terms <- policies |>
  unnest_tokens(word, Stance) |>
  anti_join(stop_words, by = "word") |>
  count(word, sort = TRUE) |>
  slice_head(n = 12)
```

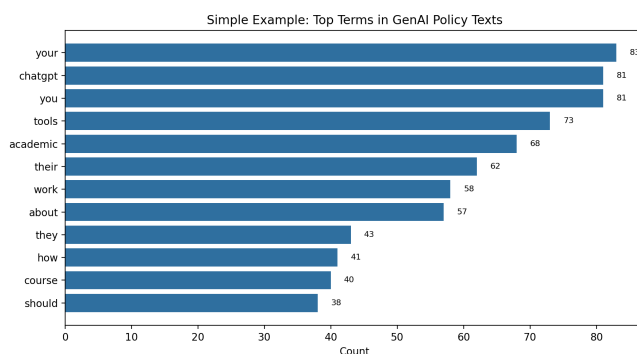


Figure 11.8. Simple frequency bar chart adapted from Chapter 2 policy analysis.

The communication page pairs this chart with a short interpretation.

11.4.6 Presenting Results for Interpretation

Instead of emphasizing technical implementation, the website focuses on interpretive clarity. For example, a bar chart displaying the most frequent terms highlights the prominence of words such as *assignment*, *student*, and *writing*, emphasizing institutional attention to coursework and academic expectations.

Narrative text accompanying the visualization explains how these patterns relate to broader concerns about academic integrity, instructor authority, and ethical AI use. This integration of visual and narrative elements helps prevent misinterpretation of descriptive computational results.

11.4.7 Publishing and Sharing

For the GenAI policy case in this chapter, publishing follows the same two-layer model introduced in Section 9.2.3:

- a **source layer** (repository with Quarto files, scripts, and documentation), and
- a **public layer** (rendered static site for readers).

This separation supports both versioned development and stable public communication. The same pattern can be adapted to course projects, policy analyses, and larger collaborative studies.

11.4.8 Why This Format Matters

Communicating the GenAI policy analysis as a web-based artifact offers several advantages:

- Readers can engage with results at their own pace.
- Methods and assumptions are documented transparently.
- The analysis can be updated as policies evolve.
- The artifact supports both research dissemination and teaching.

Importantly, this approach complements rather than replaces traditional publications. A journal article may present the core findings, while a project website provides extended documentation and resources.

11.4.9 Summary

This case study illustrates how a frequency-based text analysis can be transformed into a communicative research product using Quarto. By shifting emphasis from computation to interpretation and accessibility, researchers can extend the reach and impact of computational educational research beyond static manuscripts.

11.5 Benefits of Web-Based Communication

Communicating computational research through project websites offers several advantages:

- **Transparency:** Readers can inspect code, data, and assumptions.
- **Reproducibility:** Analyses can be rerun and extended.
- **Flexibility:** Content can be updated as methods or data evolve.
- **Pedagogical value:** Websites serve as learning resources for students and practitioners.

These benefits complement, rather than replace, traditional academic publications.

11.6 Ethical and Responsible Communication

When communicating computational analyses in education, researchers must attend to ethical considerations. These include protecting student privacy, avoiding deficit-oriented narratives, and clearly stating methodological limitations.

Public-facing materials should avoid revealing identifiable information and should transparently acknowledge sources of bias or uncertainty. Responsible communication ensures that openness does not come at the expense of ethical research practice.

11.7 Data Availability and Reproducibility Statement

All executable workflows in this book are intended to be reproducible from repository materials. Core datasets used in chapter analyses are stored under `data/`, and rendered outputs are published under `docs/`.

To support transparency, we distinguish between two types of examples:

- **Executable analysis workflows:** code and data that generate the chapter outputs.
- **Illustrative templates:** non-executed demonstration snippets included for pedagogical purposes (for example, generic network or scraping patterns).

Reproducibility in this book is supported through versioned source files (`.qmd`), explicit data paths, and documented model/tool configurations in chapters that use LLM-assisted methods.

11.8 Summary

Communication is a methodological choice. Rigorous computational educational research aligns questions, data, and methods; documents analytical decisions; provides code and data when possible; uses visualizations to support interpretation; addresses ethical considerations; and matches the communication format to its intended audience.

Conclusion

This book began with a simple idea: what we can study is shaped by what we can see as data. As educational researchers, we do not only begin with theory and then look for evidence; we also encounter new forms of evidence that reshape our questions, methods, and interpretations. Across the chapters, we have tried to make that process explicit, practical, and reproducible.

From there, we worked through a field-guide approach to computational analysis in education. Rather than treating methods as abstract techniques, each chapter focused on concrete research workflows: what data to collect, how to prepare it, what questions are reasonable to ask, how to analyze it in R, and how to write up methods and results in ways that others can follow.

12.1 What this book contributes

We see this guide as offering four connected contributions for educational researchers.

First, we aimed to provide an accessible on-ramp to computational work in R, with enough structure for beginners while still supporting applied research.

Second, we showed how different computational methods answer different kinds of questions. Text methods help us identify themes and language patterns; network methods illuminate relationships and structures; numeric modeling supports prediction and explanation at scale.

Third, we situated large language models (LLMs) as part of computational research practice. We introduced cloud and local pathways, then demonstrated local LLM workflows for both text and image analysis. The aim was not to present LLMs as a universal solution, but to show when and how they can be integrated responsibly.

Fourth, we emphasized communication as method. Computational analysis is not complete when code runs; it is complete when decisions, evidence, and limitations are communicated clearly and ethically to the audiences that matter.

12.2 Methods are complementary, not competitive

A core takeaway for us is that computational approaches are strongest when used together. Traditional NLP and statistical models provide transparency, standardization, and strong baselines. LLM-based workflows add semantic interpretation, synthesis, and flexibility for complex or multi-modal data.

In many projects, the most rigorous strategy is hybrid:

- use conventional methods to establish descriptive and inferential structure,
- use LLMs to support interpretation and synthesis,
- and then validate results through triangulation, human review, and clear documentation.

This is especially important in educational settings, where data are often sensitive and interpretations carry real consequences for learners, educators, and institutions. In our view, method choice is always also an ethical choice.

12.3 Responsible use in educational research

Throughout this book, we returned to three standards for responsible computational and AI-assisted research:

- **Correctness:** Are findings supported by evidence and checks?
- **Transparency:** Can readers see how conclusions were produced?
- **Reproducibility:** Can the analysis be rerun and audited?

These standards apply to all methods, but they are particularly important when LLMs are involved. Whether we use cloud APIs, local models, or assistant tools in the IDE, our role as researchers remains central: define the question, evaluate outputs, document decisions, and make uncertainty visible.

12.4 A practical path forward

If you are continuing after this book, we suggest building one small, complete study cycle:

1. Select one dataset and one focused research question.
2. Apply one core method (text, network, or numeric) in a fully reproducible script.
3. Add one LLM-assisted step where it genuinely improves workflow or interpretation.
4. Validate outputs with at least one independent check.
5. Publish the analysis as a transparent Quarto artifact with clear limitations.

Repeating this cycle across projects builds both technical fluency and methodological judgment. This is the same iterative path we have followed in our own work.

12.5 Final reflection

Computational educational research is not only about learning more tools. It is about learning how to ask better questions with new forms of evidence, while keeping rigor, ethics, and clarity at the center of the work.

Our hope is that this field guide helps you do that: design stronger studies, analyze richer data, communicate more openly, and contribute research that is both technically robust and educationally meaningful.

Colophon

This book was written in Quarto, analyzed with R, and rendered to HTML and PDF with XeLaTeX. Source files, data paths, package declarations, and version history are available in the book's GitHub repository.

About the Authors

Wei Wang is a PhD Candidate in Learning, Design, and Technology at the University of Tennessee, Knoxville. His research focuses on generative AI in education, educational data science, and evaluation methods. Website: <https://wwang93.github.io/Vino-Wang-Website/>

Mete Akcaoglu is Professor of Instructional Technology and Chair of the Department of Learning, Teaching, and Human Development at Georgia Southern University. His research examines game-based learning, interest development, computational thinking, and online pedagogy, with a methodological focus on computational approaches to educational data. Website: <https://meteakcaoglu.com/>

Joshua M. Rosenberg is Haslam Family Professor and Associate Professor of STEM Education at the University of Tennessee, Knoxville. His research focuses on learning analytics, educational data science, and open, reproducible research in education. Website: <https://joshuamrosenberg.com/>

Shaun Kellogg is Senior Director of the Office of Research and Promising Practices at the North Carolina Department of Public Instruction. His work focuses on evidence-based decision-making, research-practice partnerships, and the use of data to improve educational outcomes. Website: <https://www.dpi.nc.gov/blog/2025/09/17/how-ncdpis-office-research-promising-practices-turns-evidence-action>

Production details and source history for this edition are maintained in the repository.

References

- Baker, R. S., & Inventado, P. S. (2014). Educational data mining and learning analytics. In J. A. Larusson & B. White (Eds.), *Learning analytics: From research to practice* (pp. 61–75). Springer.
- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 610–623.
- Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., Arx, S. von, Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N., Chen, A., Creel, K., Davis, J. Q., Demszky, D., ... Liang, P. (2021). On the opportunities and risks of foundation models. *arXiv Preprint arXiv:2108.07258*. <https://arxiv.org/abs/2108.07258>
- Borgatti, S. P., Everett, M. G., & Johnson, J. C. (2013). *Analyzing social networks*. SAGE Publications.
- Carolan, B. V. (2014). *Social network analysis and education: Theory, methods and applications*. SAGE Publications.
- Cheng, J. (2025). *Harnessing the power of LLMs for responsible data science and research*. Keynote presentation at R+AI 2025. <https://r-consortium.org/posts/keeping-llms-in-their-lane-focused-ai-for-data-science-and-research/>
- Creswell, J. W., & Poth, C. N. (2018). *Qualitative inquiry and research design: Choosing among five approaches* (4th ed.). SAGE Publications.
- D'Mello, S., Dieterle, E., & Duckworth, A. (2017). Advanced, analytic, automated (AAA) measurement of engagement during learning. *Educational Psychologist*, 52(2), 104–123.
- Estrellado, R. A., Freer, E. A., Mostipak, J., Rosenberg, J. M., & Velásquez, I. C. (2020). *Data science in education using R*. Routledge.
- Gopen, G. D., & Swan, J. A. (1990). The science of scientific writing. *American Scientist*, 78(6), 550–558.
- Grimmer, J., Roberts, M. E., & Stewart, B. M. (2022). *Text as data: A new framework for machine learning and the social sciences*. Princeton University Press.

- Healy, K. (2018). *Data visualization: A practical introduction*. Princeton University Press.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R* (2nd ed.). Springer.
- Kasneci, E., Seßler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günnemann, S., Hüllermeier, E., Krusche, S., Kutyniok, G., Michaeli, T., Nerdel, C., Pfeffer, J., Poquet, O., Sailer, M., Schmidt, A., Seidel, T., ... Kasneci, G. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103, 102274.
- Kellogg, S., & Edelmann, A. (2015). Massively open online course for educators (MOOC-Ed) network dataset. *British Journal of Educational Technology*, 46(5), 977–983.
- Krumm, A., Means, B., & Bienkowski, M. (2018). *Learning analytics goes to school: A collaborative approach to improving education*. Routledge.
- Kuzilek, J., Hlostá, M., & Zdrahal, Z. (2017). Open university learning analytics dataset. *Scientific Data*, 4, 170171.
- Lang, C., Siemens, G., Wise, A. F., Gašević, D., & Merceron, A. (Eds.). (2022). *The handbook of learning analytics* (2nd ed.). Society for Learning Analytics Research (SoLAR). <https://doi.org/10.18608/hla22>
- Lazer, D., Pentland, A., Adamic, L., Aral, S., Barabási, A.-L., Brewer, D., Christakis, N., Contractor, N., Fowler, J., Gutmann, M., Jebara, T., King, G., Macy, M., Roy, D., & Van Alstyne, M. (2009). Computational social science. *Science*, 323(5915), 721–723.
- Liu, X., Zambrano, A. F., Baker, R. S., Barany, A., Ocumpaugh, J., Zhang, J., Pankiewicz, M., Nasiar, N., & Wei, Z. (2025). Qualitative coding with GPT-4: Where it works better. *Journal of Learning Analytics*, 12(1), 169–185.
- Nelson, L. K., Burk, D., Knudsen, M., & McCall, L. (2021). The future of coding: A comparison of hand-coding and three types of computer-assisted text analysis methods. *Sociological Methods & Research*, 50(1), 202–237.
- Nosek, B. A., Alter, G., Banks, G. C., Borsboom, D., Bowman, S. D., Breckler, S. J., Buck, S., Chambers, C. D., Chin, G., Christensen, G., Contestabile, M., Dafoe, A., Eich, E., Freese, J., Glennerster, R., Goroff, D., Green, D. P., Hesse, B., Humphreys, M., ... Yarkoni, T. (2015). Promoting an open research culture. *Science*, 348(6242), 1422–1425.
- OpenAI. (2023). GPT-4 technical report. *arXiv Preprint arXiv:2303.08774*. <https://arxiv.org/abs/2303.08774>
- Romero, C., & Ventura, S. (2010). Educational data mining: A review of the state of the art. *IEEE Transactions on Systems, Man, and*

- Cybernetics, Part C*, 40(6), 601–618.
- Romero, C., & Ventura, S. (2020). Educational data mining and learning analytics: An updated survey. *WIREs Data Mining and Knowledge Discovery*, 10(3), e1355. <https://doi.org/10.1002/widm.1355>
- Rose, G. (2016). *Visual methodologies: An introduction to researching with visual materials* (4th ed.). SAGE Publications.
- Rosenberg, J. M., Beymer, P. N., Anderson, D. J., Lissa, C. J. van, & Schmidt, J. A. (2018). tidyLPA: An R package to easily carry out latent profile analysis (LPA) using open-source or commercial software. *Journal of Open Source Software*, 3(30), 978. <https://doi.org/10.21105/joss.00978>
- Rosenberg, J. M., Borchers, C., Dyer, E. B., Anderson, D., & Fischer, C. (2021). Understanding public sentiment about educational reforms: The next generation science standards on Twitter. *AERA Open*, 7, 23328584211024261.
- Rosenberg, J. M., & Staudt Willet, K. B. (2021). Advancing social influence models in learning analytics. *Proceedings of the NetSciLA21 Workshop*. https://ceur-ws.org/Vol-2868/article_2.pdf
- Saldaña, J. (2021). *The coding manual for qualitative researchers* (4th ed.). SAGE Publications.
- Salganik, M. J. (2019). *Bit by bit: Social research in the digital age*. Princeton University Press.
- Siemens, G. (2013). Learning analytics: The emergence of a discipline. *American Behavioral Scientist*, 57(10), 1380–1400.
- Silge, J., & Robinson, D. (2017). *Text mining with R: A tidy approach*. O'Reilly Media. <https://www.tidytextmining.com/>
- Teig, N., Scherer, R., & Olsen, R. V. (2022). A systematic review of studies investigating science teaching and learning: Over two decades of TIMSS and PISA. *International Journal of Science Education*, 44(12), 2035–2058.
- Than, N., Fan, L., Law, T., Nelson, L. K., & McCall, L. (2025). Updating “the future of coding”: Qualitative coding with generative large language models. *Sociological Methods & Research*. <https://doi.org/10.1177/00491241251339188>
- Wasserman, S., & Faust, K. (1994). *Social network analysis: Methods and applications*. Cambridge University Press.
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., Francois, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Muller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01533>

References

- [//doi.org/10.21105/joss.01686](https://doi.org/10.21105/joss.01686)
- Wickham, H., & Grolemund, G. (2017). *R for data science*. O'Reilly Media. <https://r4ds.had.co.nz/>
- Wilke, C. O. (2019). *Fundamentals of data visualization*. O'Reilly Media. <https://clauswilke.com/dataviz/>
- Xie, Y., Allaire, J. J., & Grolemund, G. (2018). *R markdown: The definitive guide*. Chapman; Hall/CRC.